从 C++到 AI 计算：第三册实践与代码卷

本地量化推理引擎、完整源码、KV Cache、后端优化与验收

CPU Performance Study

本卷是《从 C++ 到 AI 计算：第三册》的配套实践与代码卷，围绕 `linux_cpu_inference` 贯穿项目展开：模型资产、tokenizer、张量布局、int8 kernel、reference decoder、KV cache、trace、7B 账本、服务调度、CPU/GPU 后端边界和工程验收都要落到可构建代码、完整源码、测试和报告。

前言

第三册原理卷把 AI 推理系统拆成模型资产、typed graph、张量布局、量化、KV cache、后端、服务运行时和验收体系。本实践与代码卷只做一件事：把这些概念压到同一个可运行项目里，并把完整源码阅读路径并入同一目录，让读者知道每个概念对应哪段代码、哪条命令、哪种输入、哪份报告和哪条失败路径。

贯穿项目是 `books/cpu-volume-3/labs/linux_cpu_inference`。它不是玩具目录，而是第三册已经持续维护的本地 CPU 量化推理切片：包含教学模型格式、tokenizer、safetensors manifest gate、int8 linear、packed/AVX2 路径、tiny reference decoder、GPT-2 reference path、KV cache trace、7B compute ledger、serving SLO probe、自研 GPT-2 smoke、真实 small 开源模型 smoke、CLI、benchmark 和 CTest。本卷不维护第二份实现；源码章节通过 `lstinputlisting` 直接读取同一份工程文件，所有练习、讲解和完整代码都回到同一份源码。

本卷的学习顺序是从小证据到大系统：先让一个 tiny MLP 算对，再解释张量地址流和 int8 累加；再让 reference decoder、sampler 和 KV cache trace 固定语义；然后接模型导入、shape verifier、memory planner 和 IR lowering；最后用 CPU/GPU 后端边界、工业专项和回归门禁收束。代码走读不再被放到最后当附录，而是插入到相同主题部分中：讲完项目入口就读构建文件和调用链，讲完量化就读公共合同、tiny MLP 和 int8 kernel，讲到真实模型时读 tokenizer、safetensors 和 GPT-2，收束验收时读工具、benchmark、smoke 和测试。每章都要求读者交付代码运行结果或报告字段，而不是只抄概念定义。

核心思想

> 第三册实践的目标不是“写一个看起来像大模型的 demo”，而是建立一条能被复查的推理证据链：资产能校验，shape 能验证，kernel 能对照，trace 能定位，性能能降级，服务指标能解释，回归门禁能长期维护。

常见缩写和术语先导

缩写	全称	本卷中的含义
AI	Artificial Intelligence	这里特指本地推理工程，不讨论泛泛应用口号。
LLM	Large Language Model	decoder-only 文本生成模型是本卷主线。
KV Cache	Key/Value Cache	attention 历史 K/V 状态，属于请求生命周期。
GEMM	General Matrix Multiply	矩阵乘，是预填充和训练常见核心算子。
GEMV	General Matrix Vector Multiply	矩阵向量乘，是单 token decode 的核心形态。
SIMD	Single Instruction Multiple Data	一条指令处理多个 lane 的 CPU 执行方式。
AVX2	Advanced Vector Extensions 2	x86-64 上的 256-bit SIMD 指令集之一。
FMA	Fused Multiply-Add	乘加融合指令，常出现在浮点 kernel 中。
SLO	Service Level Objective	服务目标，例如 TTFT、TPOT、p95/p99。
TTFT	Time To First Token	从请求进入到首 token 输出的时间。
TPOT	Time Per Output Token	decode 阶段每个输出 token 的平均或尾部时间。
IR	Intermediate Representation	编译器内部表示，用于 pass、lowering 和计划生成。
RAG	Retrieval-Augmented Generation	检索增强生成，需要把外部文档和 prompt 编排起来。
MoE	Mixture of Experts	多专家模型，会改变路由、负载和内存账本。
LoRA	Low-Rank Adaptation	低秩适配权重，常用于微调和多租户加载。

这些缩写不要死记。每个缩写在正文里都要落到一个代码对象或报告字段：KV cache 对应 trace 里的 slot 和 position，SLO 对应 serving probe 的 CSV，IR 对应 executable plan，SIMD/AVX2 对应 kernel 反汇编和 benchmark。

本卷结构

本卷按第三册原书的四个大块组织，但目录不再采用“前面做实践、后面贴源码”的割裂方式。每个大块都把实践任务、代码走读和验收证据放在同一条主线里：`linux_cpu_inference`。

第一部分从模型资产到量化 kernel。你会运行 tiny MLP，先读构建入口、项目边界和调用链总图，再检查 tokenizer 和 safetensors manifest，手算张量地址，比较 scalar、packed 和 AVX2 路径，并在公共合同、tiny MLP 与 int8 kernel 源码里解释 scale、accumulator 和 requantize 的边界。

第二部分进入 Transformer、KV cache、sampler 和服务运行时。你会跑 reference decoder trace，解释每个 checkpoint 的 shape、stride、dtype 和 layout，构造 KV cache 状态表，分析 admission、queue wait、TTFT、TPOT、取消和拒绝。

第三部分处理模型导入、真实模型加载闭环、memory planner 和 IR lowering。你会把 manifest、shape verifier、tensor role、state edge、workspace、liveness 和 executable plan 写成可检查报告，并在同一部分中读 reference decoder、tokenizer、safetensors 和 GPT-2 reference path 的完整实现，显式运行 LCQI 自研 GPT-2 safetensors smoke 和一个 small 级别开源 GGUF 模型 smoke，而不是只停留在 tiny fixture。

第四部分收束到后端优化和验收。你会比较 CPU kernel 证据、GPU 后端边界、工业专项能力和回归门禁，并直接读工具、smoke、benchmark、ledger、serving probe 和测试源码。最终交付不是“跑通一次”，而是可复查的工程档案：命令、输入、输出、trace、benchmark、不可验证边界和回归规则。

目录

第零部分

推理链路、张量账本与量化

第 1 章实践：端到端推理链路与项目总入口

本章对应原书中的第三册“端到端链路：从模型文件到下一个 token”。不要把它当作概念复述，本章要把模型目录、manifest、tokenizer、typed graph、backend plan、request session、KV cache、oracle 和 profiler 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `tiny_mlp_i8.txt`，核心命令或测试入口是 `lcqi_cli`。

Listing 1.1 第 1 章实践：端到端推理链路与项目总入口的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / tiny_mlp_i8.txt
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：只跑一个 tiny MLP 推理请求。读者要先手写预期结果，再运行项目观察输出。动作是：把 `readfiles->tokenize->run_inference->logits->predicted_class` 串起来。如果输出里没有 `predicted_class` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型文件存在不等于推理链路可信；只有 CLI 输出、测试断言、输入合同和失败路径同时成立，才算完成入口闭环这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：最终大作业的总目录：每章报告都放入 `reports/volume3-practice/`，第一章提交项目地图、命令记录和一次成功/失败输入对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：构建入口、项目边界与调用链总图

源码阅读覆盖范围

LCQI 的源码边界由三个层次组成。第一层是仓库构建入口，它决定 CMake 怎样把第三册的实验代码纳入构建和测试。第二层是 `labs/linux_cpu_inference`，它包含真正的库、CLI、benchmark、trace、ledger、serving probe 和测试目标。第三层是工具脚本，它们下载或复用外部模型资产，生成 smoke 和 benchmark 报告。

源码阅读任务

> 先读本章的 CMake 文件，再读 README。读者要能回答三个问题：一是 `lcqi` 静态库包含哪些实现文件；二是哪些可执行文件只依赖自研代码，哪些依赖可选外部库；三是真实 GPT-2 和 SmolLM2 smoke 为什么不进入默认 CTest。

根构建入口

第三册根目录的 CMake 文件只做项目级选择：是否构建第三册 labs，以及如何把 `labs/linux_cpu_inference` 接入。它应该保持薄层，避免把具体目标定义分散到多个地方。

Listing 1.2 cpu-volume-3/README.md

```
# 从 C++ 到 AI 计算：第三册

第三册用于承接 AI、AI 编译器、算子开发和本地大模型推理引擎。第二册只讨论硬件原理、多核、高性能计算、并发、锁、无锁数据结构、并行算法和分布式计算，不再直接进入 AI。

本册贯穿项目是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并把 AI 编译器作为引擎内部主线：模型图导入、typed tensor IR、算子融合、内存规划、CPU/GPU lowering、kernel selection 和 runtime dispatch 都要能落回编译原理与底层机器账本。工程路线从 CPU 上可解释的 reference 和量化切片开始，逐步扩展到 tokenizer、模型加载、Transformer 层、KV cache、CPU SIMD/多线程后端、GPU 后端、正确性报告和性能对标。

正式书稿工程位于：

```text
source/latex/
```

构建命令：

```bash
make -C source/latex check
make -C source/latex pdf
make -C source/latex epub
```

生成文件：

```text
```
```

````
source/latex/main.pdf
source/latex/main.epub
```

当前保留的第一阶段切片：

```text
labs/linux_cpu_inference/
```

该目录目前包含最小 MLP/int8 权重教学切片、tiny reference decoder、GPT-2 reference path、int8 scalar baseline、packed layout、AVX2 SIMD 路径和 shape sweep benchmark，用来验证张量、线性层、量化权重、tokenizer、safetensors、GPT-2 forward、KV cache、oracle 和 benchmark 的基本边界。当前 x86-64 AVX2 结果见：

```text
results/lcqi-x86-avx2-benchmark-2026-06-25.csv
results/lcqi-x86-avx2-decode4096-2026-06-25.csv
```

自研 GPT-2 smoke 入口如下：

```bash
make cpu3-gpt2-smoke
make cpu3-gpt2-benchmark-compare
```

第一条命令下载 `openai-community/gpt2` 的标准 safetensors 资产到用户缓存目录，用 LCQI C++ reference path 生成 1 个 token，并把报告写入：

```text
results/lcqi-gpt2-smoke.txt
```

第二条命令用同一个 GPT-2 F32 模型对比 LCQI full-prefix 和 cached-KV 生成路径，并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考，报告写入：

```text
results/lcqi-gpt2-benchmark-compare.txt
```

它仍然不是生产级 AI Infra。生产级门禁见：

```text
reports/ai-infra-maturity-audit.md
```

第三册后续必须把 lab 推进到“小型 CPU 推理 runtime + 手写高性能算子 + 严格 benchmark 报告 + 真实系统对标”，否则只能证明系统学习，不能证明生产级能力。
````
````

Listing 1.3 cpu-volume-3/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(CPUVolume3 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)

if(CPU_VOLUME_3_BUILD_LABS)
    enable_testing()
    add_subdirectory(labs/linux_cpu_inference)
endif()
```

实践与代码卷也提供一个薄 CMake 入口，它把同一份 LCQI 源码作为可测试对象接入。这样本卷不会复制实现，只复用第三册主工程。

Listing 1.4 cpu-volume-3-practice/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(CPUVolume3Practice LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

enable_testing()
add_subdirectory(../cpu-volume-3/labs/linux_cpu_inference linux_cpu_inference)
```

本卷自身的 Makefile 负责构建 PDF/EPUB，并通过 `test` 目标回到第三册真实 LCQI 工程做 CMake/CTest 验证。实践与代码卷如果只排版、不测试，就不能宣称“完整源码可运行”。

Listing 1.5 cpu-volume-3-practice/Makefile

```
LATEX_DIR := source/latex

.PHONY: all check pdf epub test clean

all: pdf epub

check:
    $(MAKE) -C $(LATEX_DIR) check

pdf:
    $(MAKE) -C $(LATEX_DIR) pdf

epub:
    $(MAKE) -C $(LATEX_DIR) epub

```

```
test:
    cmake -S . -B build/lcqi-debug -DCMAKE_BUILD_TYPE=Debug
    cmake --build build/lcqi-debug
    ctest --test-dir build/lcqi-debug --output-on-failure

clean:
    $(MAKE) -C $(LATEX_DIR) clean
```

Listing 1.6 cpu-volume-3-practice/source/latex/Makefile

```
LATEXMK ?= latexmk
ENGINE ?= -xelatex
MAIN ?= main.tex
EPUB_BUILDER ?= ../../../cpu-volume-1/source/latex/scripts/build_epub.py

.PHONY: all pdf epub clean check

all: pdf epub

pdf:
    $(LATEXMK) $(ENGINE) -interaction=nonstopmode -halt-on-error $(MAIN)

epub:
    @BOOK_TITLE="从 C++ 到 AI 计算：第三册实践与代码卷" \
    BOOK_IMPORT_TITLE="CPU Volume 3 Practice Code" \
    BOOK_ID_SEED="cpu-volume-3-practice-code-weread-20260629" \
    EPUB_ASCII_TITLES=1 \
    BOOK_SUBTITLE="本地量化推理引擎、完整源码、KV Cache、后端优化与验收" \
    BOOK_AUTHOR="CPU Performance Study" \
    python3 $(EPUB_BUILDER) --book-dir . --output main.epub --no-cover-page --forbid-images --linearize-tables --wechat-compatible --no-embed-fonts

clean:
    $(LATEXMK) -C

check:
    @python3 scripts/check_inputs.py
    @command -v xelatex >/dev/null 2>&1 || { echo "missing xelatex"; exit 1; }
    @command -v latexmk >/dev/null 2>&1 || { echo "missing latexmk"; exit 1; }
```

LCQI 目标定义

`linux_cpu_inference/CMakeLists.txt` 是代码走读中的第一份核心文件。这里能看到 `lcqi` 库、`lcqi_cli`、`lcqi_bench`、`lcqi_trace`、`lcqi_gpt2`、`lcqi_ledger`、`lcqi_serving_probe`、可选 `lcqi_onednn_bench` 和 `lcqi_tests` 的完整关系。

Listing 1.7 linux_cpu_inference/CMakeLists.txt

```
find_package(Threads REQUIRED)

add_library(lcqi STATIC
    src/gpt2_reference.cpp
```

```
    src/inference.cpp
    src/int8_kernels.cpp
    src/model.cpp
    src/reference_decoder.cpp
    src/safetensors.cpp
    src/tokenizer.cpp
)

if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
    if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
        target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
        set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↩
    ↪ COMPILE_OPTIONS "-mavx2;-mfma")
        target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
    endif()
endif()

if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
    message(WARNING "LCQI is written for the book's local Linux CPU target; ↩
    ↪ current system is ${CMAKE_SYSTEM_NAME}.")
endif()

target_include_directories(lcqi
    PUBLIC
        ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_features(lcqi PUBLIC cxx_std_20)
target_link_libraries(lcqi PUBLIC Threads::Threads)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_cli
    src/main.cpp
)

target_link_libraries(lcqi_cli PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_bench
    src/bench.cpp
)

target_link_libraries(lcqi_bench PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
```

```
endif()

add_executable(lcqi_trace
    src/trace.cpp
)

target_link_libraries(lcqi_trace PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_gpt2
    src/gpt2_cli.cpp
)

target_link_libraries(lcqi_gpt2 PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_gpt2 PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_ledger
    src/ledger.cpp
)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_executable(lcqi_serving_probe
    src/serving_probe.cpp
)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
endif()

find_package(dnnl QUIET)
if(dnnl_FOUND)
    add_executable(lcqi_onednn_bench
        src/onednn_bench.cpp
    )
    target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
    target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
    if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
        target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra -
    ↪ Wpedantic)
    endif()
endif()

add_executable(lcqi_tests
```

```
    tests/lcqi_tests.cpp
)

target_link_libraries(lcqi_tests PRIVATE lcqi)

if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
    target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
endif()

add_test(NAME lcqi_tests COMMAND $<TARGET_FILE:lcqi_tests>)
add_test(NAME lcqi_gpt2_tiny COMMAND $<TARGET_FILE:lcqi_gpt2>)
set_tests_properties(lcqi_gpt2_tiny PROPERTIES
    PASS_REGULAR_EXPRESSION "generated_ids 1 2 3 3 3"
)
add_test(NAME lcqi_trace COMMAND $<TARGET_FILE:lcqi_trace>)
set_tests_properties(lcqi_trace PROPERTIES
    PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
)
add_test(NAME lcqi_ledger COMMAND $<TARGET_FILE:lcqi_ledger>)
set_tests_properties(lcqi_ledger PROPERTIES
    PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
)
add_test(NAME lcqi_serving_probe COMMAND $<TARGET_FILE:lcqi_serving_probe>)
set_tests_properties(lcqi_serving_probe PROPERTIES
    PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
)
```

项目说明与模型资产

README 是复现实验的入口。它写清楚当前能力、后续边界、构建命令、tiny 推理命令、benchmark 字段、reference trace 字段、GPT-2 smoke、7B ledger、serving SLO probe 和 SmolLM2 small smoke。读源码前先读 README，可以避免把 tiny fixture、GPT-2 reference path 和外部 llama.cpp smoke 混成同一个能力等级。

Listing 1.8 linux_cpu_inference/README.md

```
# Linux CPU Quantized Inference

这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache、CPU/GPU 后端、正确性报告和性能对标。

目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变当前切片的语义合同。

当前版本支持：

- 教学文本模型格式 `LCQI_MODEL_V1`
```

```
- 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
- safetensors header manifest 解析：metadata、tensor name、dtype、shape、data offsets、offset 边界和 dtype/shape 字节数校验
- 两层 MLP 教学切片
- int8 权重加 per-layer scale
- float 输入和激活
- 量化线性层、ReLU、分类 argmax
- int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
- tiny reference decoder：RMSNorm、float linear、RoPE、GQA KV cache、decode attention、SwiGLU、lm head
- GPT-2 reference path：byte-level BPE tokenizer、`config.json`、F32/F16/BF16 safetensors 张量读取、HuggingFace GPT-2 name mapping、LayerNorm、绝对位置嵌入、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation、KV cache greedy generation 和分阶段 benchmark
- reference trace CSV：从 prompt/tokenizer 合同到 logits/sampler/token，逐 checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values 摘要
- 7B ledger CSV：输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限
- serving SLO probe CSV：输出 admission、batch state、KV 预算、queue wait、TTFT、TPOT、取消回收和拒绝率
- 自研 GPT-2 冒烟：加载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors`，在 LCQI C++ reference path 上生成 1 个 token，并把文件 hash、prompt ids、generated ids 和文本输出写入报告
- 自研 GPT-2 benchmark 对比：同一个 GPT-2 F32 safetensors 模型分别跑 full-prefix 和 cached-KV，并在可用时附带 llama.cpp/SmolLM2 Q4_K_M 作为成熟引擎参考
- 自研 GPT-2 优化 A/B：从指定 baseline commit 建临时 worktree，当前实现和 baseline 都通过 `books/cpu-volume-3-practice` 的 Release CMake 入口构建，再用同一个 GPT-2 F32 模型多轮测量 full-prefix 与 cached-KV
- 真实 small 开源模型冒烟：通过 llama.cpp 加载 SmolLM2-135M-Instruct 的 GGUF 量化文件，在本地 CPU 上生成一段 assistant 文本，并把模型 hash、命令、输出和性能行写入报告
- CLI 推理和简单 benchmark
- CTest 正确性测试

后续扩展方向：

- 真实 7B 级模型 metadata、tokenizer 和量化权重导入
- SentencePiece tokenizer、GGUF 权重导入、safetensors 分片加载和更多模型方言 name mapping
- GPT-2 reference path 的 Transformers/PyTorch golden logits 对齐报告
- KV cache 分页、内存规划和可选量化
- CPU packed weight、SIMD、NUMA 和线程亲和优化
- GPU backend adapter 和 device kernel
- 与现有框架的 prefill/decode/内存/误差对标报告

构建：

```bash
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -DCMAKE_BUILD_TYPE=Debug
```
```

````
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
```

运行：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
  books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
  1.0 2.0 -1.0 0.5 1000
```

kernel benchmark：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
```

输出 CSV 字段：

```text
target_arch,input_size,output_size,output_block,repeat,backend,available,average_us,max_abs_diff,checksum
```

当前 benchmark 按 backend 逐行输出：`scalar`、`packed_scalar`、`avx2`。x86-64 构建会编译 AVX2 路径；不支持 AVX2/FMA 的机器会把该 backend 标为 `available=0`，避免把源码存在误报成当前环境实测性能。

这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据，还必须继续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对照、sanitizer、fuzz 和 CI。

safetensors manifest gate：

`lcqi_tests` 会运行时生成一个最小 safetensors fixture，验证 header 长度、`__metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape 字节数不匹配的坏文件，确认加载器会拒绝。这个 gate 只覆盖模型资产目录表，不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行；后续真实模型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant descriptor 和 first token trace。

reference decoder trace：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
```

输出 CSV 字段：
````
````

```text
name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
```

这条 trace 固定 prompt fixture `tok1 tok2 tok3`，tokenizer 输出完整 `tokens=[0,1,2,3,4]`，其中 `0/4` 是 BOS/EOS；decoder trace 会剥离 BOS/EOS 后进入 tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同，避免把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`lcqi_tests` 会检查 tokenizer contract **hash**、关键 checkpoint 的 shape、stride、dtype、layout 和 logits golden，CTest 也会直接运行 `lcqi_trace`。

当前 tokenizer fixture 的关键输出：

```text
tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-chat-template-v1
```

GPT-2 自研 reference smoke：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
make cpu3-gpt2-smoke
make cpu3-gpt2-benchmark-compare
make cpu3-gpt2-hotspot-profile
python3 books/cpu-volume-3/tools/run_gpt2_optimization_ab.py --rounds 3
python3 books/cpu-volume-3/tools/run_gpt2_hotspot_profile.py --token-counts 4,16 --rounds 3
```

第一条命令不需要下载模型，直接运行仓库内 tiny GPT-2 fixture，输出固定 prompt ids、logits、predicted token 和 greedy generated ids。`lcqi_tests` 会覆盖 tiny GPT-2 forward/generation、byte-level BPE 编码解码、F32/F16/BF16 safetensors 读取、HuggingFace 风格 tiny GPT-2 checkpoint 加载和坏 shape 拒绝。

第二条命令显式依赖网络，只下载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors` 到 `~/.cache/lcqi-gpt2-smoke/openai-community--gpt2`，模型文件不进入 Git。脚本会校验四个文件的 bytes 和 SHA256，构建 `lcqi_gpt2`，运行：

```text
Hello, my name is
```

当前验证报告写到：

```text
books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
```

```
```

最近一次本机报告的关键行是：

```text
prompt_ids 15496 11 616 1438 318
generated_ids 15496 11 616 1438 318 1757
generated_text Hello, my name is John
validation=PASS
```

这说明 LCQI 自研 C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer、权重导入、forward 和 greedy generation。默认真实模型路径现在使用 `cached_kv`，也可以用 `--engine full` 复现旧的 full-prefix 重算路径：

```bash
books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_gpt2 \
  --benchmark --engine cached --threads 0 \
  ~/.cache/lcqi-gpt2-smoke/openai-community--gpt2 \
  "Hello, my name is" 4
```

`--threads 0` 表示自动选择 cached decoder worker 数；`--threads 1` 强制串行；`--threads N` 固定 worker 数。当前 auto 上限是 16，避免在短 decode 上把线程调度开销放大到超过收益。

`make cpu3-gpt2-benchmark-compare` 会生成：

```text
books/cpu-volume-3/results/lcqi-gpt2-benchmark-compare.txt
```

`run_gpt2_optimization_ab.py` 会生成：

```text
books/cpu-volume-3/results/lcqi-gpt2-optimization-ab.txt
```

这个 A/B 报告专门回答“当前 LCQI 代码相对指定 baseline 改快了吗”。脚本默认 baseline 是 `4febf3f`，会临时创建 baseline worktree，分别构建 baseline 和当前工作区的 Release `lcqi_gpt2`，再多轮运行同一条 prompt。上一轮工作区复用优化的 2 轮 smoke 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-optimization-ab-summary.md`；本轮线程优化的正式 raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md`。以 `73f40c7` 作为 baseline，在 `caw` 上 5 轮中位数显示：4 token cached-KV 生成阶段从 `395.397 ms` 降到 `76.4141 ms`，16 token 从 `989.060 ms` 降到 `185.479 ms`，生成吞吐分别提升到 `52.3464 token/s` 和 `86.2633 token/s`。端到端 GPT-2 数字受调度、温度和共享机器状态影响很大，因此正式结论必须看多轮中位数、min/max 和生成文本一致性，不能只挑一次最快结果。

`run_gpt2_hotspot_profile.py` 专门回答“现在热点到底在哪”。它通过 `--profile-

hotspots` 打开 cached decode 内部计时，字段包括 `hotspot_lm_head_pct`、`hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`hotspot_qkv_projection_pct` 和 `hotspot_attention_pct`。`caw` 上的 3 轮 Release profile 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-profile-summary.md`，原始报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-profile-caw.txt`。关键结论是：4 token 与 16 token 两个口径下，`lm_head` 中位数约 `30.2%`，MLP 两个线性投影合计约 `46%`，QKV 投影约 `17%`，cached attention 本体只有 `0.06%` 到 `0.11%`。所以当前短上下文 GPT-2 的慢点不是 KV attention，而是标量 F32 matvec 和 `50257 x 768` vocab 扫描。

当前优化点集中在 cached-KV generation：`Gpt2CachedGreedyDecoder` 把模型级 shape validation、KV cache、临时 workspace 和 worker pool 的生命周期提升到整段生成；`Gpt2ForwardWorkspace` 复用 hidden、normed、Q/K/V、packed QKV、attention、MLP、scores 和 logits 缓冲区；生成路径只需要 greedy token 时走 logits-free argmax，避免把完整 logits vector 作为 API 结果反复分配；QKV、attention output projection、MLP 两个 projection 和 tied `lm_head` 都在 cached session 内按输出行并行。KV cache 的 checked public API 仍保留，内部 cached attention 在一次边界校验后使用私有 unchecked span 扫描热路径。它不改变 GPT-2 数学语义，只改变会话生命周期、任务分片和热循环调度。

`make cpu3-gpt2-benchmark-compare` 仍用于和外部成熟引擎放在同一报告里看工程差距。同机 llama.cpp/SmolLM2 Q4_K_M 是成熟引擎参照，不是同模型质量排名：LCQI 跑的是 GPT-2 F32 reference path，llama.cpp 跑的是 GGUF 量化 small instruct 模型。它揭示的是工程差距：LCQI 已经消除了“每步重算前缀”的主要算法错误，并补上 cached F32 row parallelism，但还缺 packed/quantized weight、SIMD/SVE/NEON/AVX 高性能 matmul、mmap/lazy load、采样器、分页 KV cache、NUMA/亲和和成熟调度。

它仍然不是生产级推理引擎：当前 GPT-2 路径还没有把 logits 和 Transformers/PyTorch 逐数值对齐成外部 golden 报告，也没有 GGUF 自研导入、分页 KV cache、量化权重执行和高性能 SIMD/packed kernel。

7B ledger：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
  > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
```

输出 CSV 字段：

```text
section,item,value,unit,notes
```

这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`，报告 decode FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行，防止账本公式漂移。

serving SLO probe：

```bash
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe \
  > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
```

输出 CSV 字段：

```text
row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,metric_value
```

probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按 `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回 `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、`rejection_rate` 等服务化指标。

真实 small 开源模型 smoke：

```bash
make cpu3-smollm2-smoke
```

这条命令显式依赖网络和外部构建，因此不放进默认 CTest。脚本会把 llama.cpp 和模型文件缓存到 `~/.cache/lcqi-smollm2-small-smoke`，模型不进入 Git。固定模型是 `HuggingFaceTB/SmolLM2-135M-Instruct` 的 GGUF 量化版本，来源仓库为 `bartowski/SmolLM2-135M-Instruct-GGUF`，文件为 `SmolLM2-135M-Instruct-Q4_K_M.gguf`。脚本会校验文件大小 `105454432` 字节和 SHA256：

```text
2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```

脚本构建固定 commit 的 `llama-simple-chat`，用 `-ngl 0` 强制 CPU 路径，输入短 prompt，检查 assistant response 非空，并生成：

```text
books/cpu-volume-3/results/lcqi-smollm2-small-model-smoke.txt
```

这份报告只能证明真实 GGUF small 模型已经完成加载、tokenizer/chat template、prefill、decode 和文本输出。它不能证明 LCQI 自研 C++ 引擎已经支持完整 GGUF、不能证明回答质量正确，也不能替代 tiny fixture 的逐层 golden trace。tiny fixture 继续负责可手算语义和回归定位；SmolLM2 smoke 负责把“真实开源模型能在本机跑起来”变成可复查证据。

tiny MLP 模型文件是最短闭环的资产。它决定输入维度、hidden 维度、输出维度、权重量化值、scale 和 bias；测试中的 logits golden 依赖这份文件。

Listing 1.9 models/tiny_mlp_i8.txt

```
LCQI_MODEL_V1
input_size 4
hidden_size 3
output_size 2
hidden_scale 0.1
hidden_weights
5 -3 2 1
-4 6 1 -2
3 2 -5 4
hidden_bias
0.1 -0.2 0.0
output_scale 0.05
output_weights
4 -6 2
-3 5 8
output_bias
-0.5 0.4
```

tiny tokenizer fixture 固定 BOS、EOS、UNK、词表和模板 hash。tokenizer 合同如果漂移，后面的 decoder trace 和测试都会被污染。

Listing 1.10 models/tiny_tokenizer.txt

```
LCQI_TOKENIZER_V1
bos_id 0
eos_id 4
unk_id -1
chat_template_hash tiny-chat-template-v1
vocab_size 5
vocab
0 <bos>
1 tok1
2 tok2
3 tok3
4 <eos>
```

为什么要先讲调用链

实践与代码卷如果只把 `*.cpp` 和 `*.hpp` 全部贴出来，读者仍然不知道从哪里下手。LCQI 的代码同时包含 tiny MLP、int8 kernel、tokenizer、safetensors、reference decoder、GPT-2、benchmark、trace、ledger、serving probe 和外部模型脚本；这些文件不是平铺关系，而是几条调用链。先讲调用链，再给完整源码，读者才能把每个文件放到正确位置。

核心思想

> 读 LCQI 的顺序不是“哪个文件长先读哪个”，而是“入口、合同、资产、执行、证据”。入口说明目标怎样构建，合同说明类型怎样连接，资产说明模型从哪里来，执行说明 token 或张量怎样流动，证据说明测试和报告守住了哪些边界。

一、全工程入口：哪些文件决定能不能复现

文件	负责什么	读源码时要确认什么
第三册README	第三册 AI 计算工程路线、当前 LCQI 能力、GPT-2 smoke 和结果报告位置。	能力声明必须能在 labs/linux_cpu_inference、tools 和 results 中找到证据。
第三册CMake	第三册根 CMake，负责把 linux_cpu_inference 接入。	根 CMake 应保持薄层；真正 target 定义必须在 lab 内部，避免入口文件承担实现细节。
本卷CMake	本卷复用同一份 LCQI lab。	本卷不复制源码，只把同一工程接入自己的测试构建，防止两份代码漂移。
本卷Makefile	本卷的书籍构建入口。	它构建 PDF/EPUB，也提供 test 目标去构建第三册 LCQI；实践与代码卷不能只排版不验证。
LCQICMake	lcqi 库、CLI、benchmark、trace、GPT-2、ledger、serving probe、CTest 目标。	读这里能看到哪些文件进入库，哪些是可执行程序，哪些依赖可选外部库。
LCQIREADME	本地构建、tiny 推理、benchmark、trace、GPT-2、SmolLM2 smoke 的命令。	先按 README 跑命令，再读源码；否则不知道某个函数是否真的能从命令行触达。

上表里的短标签对应下面这些真实路径：

- books/cpu-volume-3/README.md
- books/cpu-volume-3/CMakeLists.txt
- books/cpu-volume-3-practice/CMakeLists.txt
- books/cpu-volume-3-practice/Makefile
- books/cpu-volume-3/labs/linux_cpu_inference/CMakeLists.txt
- books/cpu-volume-3/labs/linux_cpu_inference/README.md

这些入口文件回答“代码在哪里”和“怎样运行”。读者进入本卷代码走读章时，先不要直接跳到 gpt2_reference.cpp。先确认 lcqi 静态库包含 model.cpp、inference.cpp、int8_kernels.cpp、reference_decoder.cpp、tokenizer.cpp、safetensors.cpp 和 gpt2_reference.cpp；再确认 lcqi_cli、lcqi_bench、lcqi_trace、lcqi_gpt2、lcqi_ledger、lcqi_serving_probe 和 lcqi_tests 分别从哪条路径进入。

二、Tiny MLP 调用链：最短闭环怎样跑通

Tiny MLP 是最小推理闭环，用来证明“模型资产、shape 检查、int8 linear、激活函数、logits、argmax、CLI 输出”可以完整连起来。它不是大模型能力本身，但它是后面所有复杂路径的可手算底座。

文件	关键类型或函数	这段代码的意思
models/tiny_mlp_i8.txt	输入维度、hidden 维度、输出维度、int8 权重、scale、bias。	这是模型资产。测试中的 golden logits 来自这里，资产改动必须带着测试改动。
include/lcqi/model.hpp	QuantizedLinearLayer、TinyMlpModel、load_model。	定义 tiny MLP 如何在内存中表示；只负责模型结构和加载入口，不做推理。
src/model.cpp	read_i8_vector、validate_layer、load_model。	解析文本模型，拒绝坏 key、坏维度、权重数量不匹配和非法 scale，把错误挡在执行前。

include/lcqi/inference.hpp	InferenceResult、linear_i8、relu_inplace、run_inference。	定义推理输出要包含 hidden、logits 和 predicted class，方便测试定位错误层。
src/inference.cpp	输入 shape 检查、两层 int8 linear、ReLU、argmax。	最短执行路径：模型加输入变成 logits。这里的代码应保持清楚，不能混入 CLI 或文件解析。
src/main.cpp	解析命令行输入并打印结果。	CLI 是薄层；它只接模型路径和输入，不复制 run_inference 逻辑。

这条链路按下面顺序读：main() 取得模型路径和输入向量，调用 load_model() 把 tiny_mlp_i8.txt 变成 TinyMlpModel，再调用 run_inference()。在 run_inference() 内部，第一层 linear_i8() 把 float 输入和 int8 权重相乘，乘以 scale 后加 bias；relu_inplace() 把负 hidden 截断；第二层 linear_i8() 得到 logits；最后 argmax() 给出 predicted class。

读这个路径时要抓住边界条件。输入长度不等于 input_size 要立即拒绝；权重数量不是 rows*columns 要在加载时拒绝；logits 为空不能取 argmax。这些错误如果拖到 SIMD kernel 或 benchmark 里才暴露，定位成本会高很多。

三、Int8 Kernel 调用链：reference、packed、AVX2 和 benchmark 怎样对照

文件	关键代码	这段代码的意思
include/lcqi/int8_kernels.hpp	PackedLinearI8、KernelBenchmarkCase、benchmark_linear_i8_case。	定义 kernel 对标口径：shape、repeat、backend、平均耗时、checksum、max diff。
src/int8_kernels.cpp	pack_linear_i8、linear_i8_packed、linear_i8_packed_with_workspace、deterministic fixture。	scalar 和 packed 路径是正确性基线，workspace 路径减少重复分配，fixture 让 benchmark 和测试可复现。
src/int8_kernels_avx2.cpp	linear_i8_packed_avx2_available、linear_i8_packed_avx2。	平台优化路径。必须先判断机器能力，再执行 AVX2/FMA；不可用时报告 unavailable，而不是假装测过。
src/bench.cpp	shape 列表、repeat 参数、backend 输出。	benchmark 程序只负责测 kernel 入口。性能结论必须和 max_abs_diff、CPU、编译器、构建类型一起报告。

kernel 文件要按“可信基线到优化路径”的顺序读。先看 deterministic layer 和 input 怎么生成，保证每次运行输入相同；再看 linear_i8_packed() 怎样把 int8 权重、scale、bias 和 float 输入算成输出；再看 workspace 路径怎样复用临时缓冲；最后看 AVX2 路径怎样在支持平台上替换内层循环。测试用 packed 与 scalar 的 max_abs_diff 对照，benchmark 再比较耗时。

四、Tokenizer、Safetensors 与 Reference Decoder：真实模型前的三道门

文件	关键类型或函数	这段代码的意思
models/tiny_tokenizer.txt	BOS、EOS、UNK、词表、模板 hash。	固定 tokenizer 合同，避免 prompt 到 token id 的规则漂移。

include/lcqi/tokenizer.hpp 与 src/tokenizer.cpp	TokenizerModel、load_tokenizer、encode_prompt、tokenizer_contract_hash。	把 prompt 切成稳定 token id；未知 token 是否允许，必须由 UNK 合同决定。
include/lcqi/safetensors.hpp 与 src/safetensors.cpp	SafeTensorManifest、SafeTensorFile、manifest parser、dtype 转换。	解析模型字节之前先验证 header、dtype、shape、offset 和 payload 边界，防止坏资产进入数学层。
include/lcqi/reference_decoder.hpp 与 src/reference_decoder.cpp	embedding、RMSNorm、RoPE、KV cache、attention、SwiGLU、trace checkpoint。	Tiny decoder-only reference path。它用 checkpoint 解释每一步张量状态，不只返回最终 token。
src/trace.cpp	trace CSV 输出。	把 reference decoder 的 checkpoint 打印成可复查表格，定位错误发生在哪个阶段。

这三道门分别解决三个问题。Tokenizer 解决“人写的 prompt 如何变成 token id”；Safetensors 解决“磁盘里的模型字节如何变成带 shape 的张量”；Reference Decoder 解决“token 和权重如何经过 decoder-only 层生成 logits”。任何一层含糊，后面的 GPT-2 路径都会变成看似能跑、实则无法定位的黑盒。

五、GPT-2 调用链：从 HuggingFace 资产到生成 token

GPT-2 路径是本卷代码走读里最长的一条链，因为它要处理真实模型格式。读 gpt2_reference.cpp 时按模块读，不要从第一行一路滚到最后。

文件	关键代码	这段代码的意思
include/lcqi/gpt2_reference.hpp	Gpt2Config、Gpt2ReferenceModel、Gpt2KvCache、Gpt2Tokenizer、forward/generate API。	公开 GPT-2 reference path 的合同：配置、权重、KV cache、tokenizer、full-prefix 和 cached generation。
src/gpt2_reference.cpp	JSON parser、byte-level BPE、safetensors loader、LayerNorm、QKV、causal attention、GELU、lm head、KV cache。	正确性优先实现。它把 HuggingFace GPT-2 权重名、Conv1D 转置、tokenizer 和 forward 规则全部显式写出来。
src/gpt2_cli.cpp	tiny mode、真实模型目录模式、generation、benchmark 输出。	命令行入口。无参数跑 tiny fixture；传模型目录时加载真实 GPT-2 资产；benchmark 对比 full-prefix 和 cached-KV。
tools/run_gpt2_smoke.py	下载标准 GPT-2 资产，调用 lcqi_gpt2。	证明自研 safetensors、BPE、forward、KV cache 和 greedy generation 能跑真实开源模型。
tools/run_gpt2_benchmark_compare.py	运行 LCQI full-prefix、cached-KV，并可附带成熟引擎参考。	用同一模型口径比较算法差距和 kernel 差距，不把不同模型的速度混成一个结论。

GPT-2 代码按五段读。第一段是配置和权重加载：load_gpt2_config() 读 config.json，load_gpt2_reference_model() 从 safetensors 中取 embedding、每层 LayerNorm、QKV、MLP 和 final norm。HuggingFace GPT-2 的 Conv1D 权重形状和普通线性层方向不同，所以代码显式调用转置函数，不能跳过。

第二段是 tokenizer：`load_gpt2_tokenizer()` 读 `vocab.json` 和 `merges.txt`，`gpt2_encode()` 执行 byte-level BPE，`gpt2_decode()` 把 id 还原为文本。这里的重点是字节到 Unicode 映射、BPE merge rank 和 unknown token 边界。

第三段是 full-prefix forward：`run_gpt2_forward()` 每生成一个新 token 都用完整 token 前缀重新跑所有层。它慢，但容易验证。流程是 token embedding 加 position embedding，逐层执行 LayerNorm、packed QKV、causal attention、残差、MLP、残差，最后 final LayerNorm 和 tied lm head 得到 logits。

第四段是 cached-KV forward：`Gpt2KvCache` 保存每层每个 position 的 key/value，`run_gpt2_forward_cached()` 每次只处理最新 token，并让 attention 读取过去 cache。这里要特别看 `validate_written()`：读取未写入 cache slot 必须报错，否则生成错误会很难定位。

第五段是 greedy generation：`gpt2_generate_greedy()` 用 full-prefix 路径，`gpt2_generate_greedy_cached()` 用 KV cache 路径。两个函数都要检查 prompt 非空、`max_new_tokens` 非负、不会超过 `max_positions`，并在生成 EOS 时停止。

六、报告、服务探针、外部对标和测试

文件	关键代码	这段代码的意思
`src/ledger.cpp`	7B 参数量、KV cache、bandwidth-only 上限。	输出的是工程账本，不是实测速度；它帮助读者估算权重和 KV 内存压力。
`src/serving_probe.cpp`	短请求、RAG 请求、取消请求、超长请求，TTFT/TPOT 字段。	把服务层边界变成字段：哪些请求接受，哪些拒绝，取消后资源是否回收。
`src/onednn_bench.cpp`	oneDNN matmul benchmark。	可选外部库对标入口。没有 oneDNN 环境时不能声称已验证。
`tools/run_smollm2_small_smoke.py`	下载 SmolLM2-135M-Instruct GGUF，构建固定 commit 的 llama.cpp smoke。	这是成熟外部引擎上的真实 small 模型 smoke，用来建立外部参照，不代表 LCQI 已具备同等能力。
`tools/run_lcqi_benchmark.sh`	收集 LCQI 本地 benchmark。	在固定机器上重复运行同一组 shape，形成可复查 CSV 或文本报告。
`tests/lcqi_tests.cpp`	tiny MLP、tokenizer、safetensors、GPT-2 tiny、HF-style checkpoint、bad shape、decoder、trace、kernel。	这是本卷最重要的代码证据。新增功能如果没有进入这里，就不能写成已经守住的能力。

读测试文件时，不要只看最后是否返回 0。它里面的 fixture 本身就是源码讲解的一部分：测试会手写 safetensors header，构造 F16/BF16/F32 payload，生成 tiny GPT-2 权重，验证 bad shape 会被拒绝，比较 packed int8 kernel 和 scalar 路径。这些测试告诉读者“代码真正承诺了什么”。如果一个能力只出现在 README 或 CLI 输出里，却没有进入测试或 smoke 报告，就只能算实验入口，不能算稳定能力。

源码阅读任务

> 读完本章后，再去读相邻主题中的完整源码。每看到一个文件，都把它归到五类之一：入口、合同、资产解析、执行路径、证据。归不进去的文件，就说明工程边界需要重新解释或重构。

第 2 章实践：张量布局、地址流与第一个 MatVec

本章对应原书中的第三册“张量布局、地址流与第一个矩阵向量内核”。不要把它当作概念复述，本章要把 TensorView、dtype、shape、stride、row-major、地址公式、bias、ReLU、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `linear_i8`，核心命令或测试入口是 `lcqi_tests`。

Listing 2.1 第 2 章实践：张量布局、地址流与第一个 MatVec 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / linear_i8
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：手算一个 2 行 4 列权重矩阵的地址偏移。读者要先手写预期结果，再运行项目观察输出。动作是：把 `weight[out,k]` 展开成 `base+(out*K+k)*sizeof(i8)`，再对应到 `linear_i8` 的循环。如果输出里没有 `logit` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：张量不是裸数组；每个 kernel 入口都要能解释 dtype、shape、stride、scale 和输出缓冲这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一页地址流账本：输入向量、hidden 权重、输出 logits 各自的 shape、连续性、读写次数和错误边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 3 章实践：GEMM、packing 与 SIMD 证据

本章对应原书中的第三册“从矩阵向量乘走向 GEMM、packing 与 SIMD”。不要把它当作概念复述，本章要把 packed weight、scalar baseline、AVX2 backend、output block、shape sweep、反汇编和降级放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `output_block`，核心命令或测试入口是 `lcqi_bench`。

Listing 3.1 第 3 章实践：GEMM、packing 与 SIMD 证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / output_block
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：同一组输入同时跑 `scalar`、`packed_scalar` 和 `avx2`。读者要先手写预期结果，再运行项目观察输出。动作是：比较 CSV 中 `backend`、`available`、`average_us`、`max_abs_diff` 和 `checksum`。如果输出里没有 `max_abs_diff` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：SIMD 路径只有在结果和 baseline 对齐后才有资格谈速度；`available=0` 也是证据，不是失败这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 shape sweep 表，至少解释一个 SIMD 有收益、一个收益不明显、一个当前环境不可验证的场景。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 4 章实践：int8 量化、累加器与重新量化

本章对应原书中的第三册“量化系统总览”和“int8 量化、累加器与重新量化”。不要把它当作概念复述，本章要把 int8 weight、float activation、scale、accumulator、requantize、oracle tolerance 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `TinyMlpModel`，核心命令或测试入口是 `lcqi_tests`。

Listing 4.1 第 4 章实践：int8 量化、累加器与重新量化的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / TinyMlpModel
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：构造一行 int8 权重 `[2,-1,3,0]` 和输入 `[1,2,-1,0.5]`。读者要先手写预期结果，再运行项目观察输出。动作是：先手算 int32/float accumulator，再和 `linear_i8` 输出比较。如果输出里没有 `scale` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：量化不是把 float 强转成 int8；必须记录 scale 作用位置、累加宽度、误差阈值和 golden 输出这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交量化合同卡：每个 tensor 的 dtype、scale 粒度、accumulator 类型、误差门限和拒绝条件。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：公共合同、Tiny MLP 与 Int8 Kernel

为什么先读头文件

头文件是系统对外承诺的最小合同。实现可以重写，kernel 可以替换，benchmark 可以补充平台路径，但头文件里暴露的数据结构、函数入口和返回字段决定了测试、CLI、工具脚本和章节讲解怎样调用这套工程。

核心思想

> 读 LCQI 时不要从最长的 `gpt2_reference.cpp` 开始。先读公共头文件，明确“模型如何表示”“推理返回什么”“trace 记录什么”“KV cache 怎样寻址”“tokenizer 的坏输入怎样处理”，再进入实现。这样读到长循环和 shape 检查时，知道它们在维护哪份合同。

Tiny MLP 与推理入口

`model.hpp` 定义 tiny MLP 的模型对象、权重、bias、scale 和加载入口。它的责任是把文本模型资产解释成 C++ 对象，不负责执行推理。

Listing 4.2 include/lcqi/model.hpp

```cpp
#pragma once

#include <cstdint>
#include <filesystem>
#include <string>
#include <vector>

namespace lcqi {

struct QuantizedLinearLayer {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    float scale = 1.0F;
    std::vector<std::int8_t> weights;
    std::vector<float> bias;
};

struct TinyMlpModel {
    std::int32_t input_size = 0;
    std::int32_t hidden_size = 0;
    std::int32_t output_size = 0;
    QuantizedLinearLayer hidden;
    QuantizedLinearLayer output;
};

TinyMlpModel load_model(const std::filesystem::path& path);

}  // namespace lcqi
```

`inference.hpp` 定义推理结果和 `run_inference` 入口。返回值包含 hidden、logits 和 pre-

dicted class，测试可以用这些字段定位是加载错、kernel 错，还是 argmax 错。

Listing 4.3 include/lcqi/inference.hpp

```hpp
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {

struct InferenceResult {
    std::vector<float> logits;
    std::int32_t predicted_class = 0;
};

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
               std::span<float> output);

void relu_inplace(std::span<float> values);

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input);

}  // namespace lcqi
```

Int8 Kernel、Tokenizer 与 Safetensors

int8 kernel 头文件定义 packed linear、deterministic fixture、scalar/workspace/AVX2 路径和误差比较。注意 AVX2 是运行时能力，不是源码存在就等于当前机器已验证。

Listing 4.4 include/lcqi/int8_kernels.hpp

```hpp
#pragma once

#include <cstdint>
#include <span>
#include <vector>

#include <lcqi/model.hpp>

namespace lcqi {

struct PackedLinearI8 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
    float scale = 1.0F;
    std::vector<std::int8_t> packed_weights;
```

```
    std::vector<float> bias;
};

struct KernelBenchmarkCase {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::int32_t output_block_size = 0;
    std::int32_t repeat = 0;
};

struct KernelBackendBenchmark {
    const char* name = "";
    bool available = false;
    double average_us = 0.0;
    float max_abs_diff = 0.0F;
    float checksum = 0.0F;
};

struct KernelBenchmarkResult {
    KernelBenchmarkCase benchmark_case;
    std::vector<KernelBackendBenchmark> backends;
};

PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
                              std::int32_t output_block_size);

void linear_i8_packed(const PackedLinearI8& layer,
                      std::span<const float> input,
                      std::span<float> output);

void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
                                     std::span<const float> input,
                                     std::span<float> output,
                                     std::span<float> accumulator_workspace);

bool linear_i8_packed_avx2_available() noexcept;

void linear_i8_packed_avx2(const PackedLinearI8& layer,
                           std::span<const float> input,
                           std::span<float> output);

QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
                                                 std::int32_t output_size,
                                                 float scale);

std::vector<float> make_deterministic_input(std::int32_t input_size);

float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);

KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ↩
    ↪ benchmark_case);

```

```
68 }  // namespace lcqi
```

tokenizer 头文件固定 BOS/EOS/UNK、词表和 template hash。prompt 进入模型前，必须先变成稳定 token id；未知 token 行为也必须被写进合同。

Listing 4.5 include/lcqi/tokenizer.hpp

```
1  #pragma once
2
3  #include <cstdint>
4  #include <filesystem>
5  #include <string>
6  #include <string_view>
7  #include <vector>
8
9  namespace lcqi {
10
11 struct TokenEntry {
12     std::string text;
13     std::int32_t id = 0;
14 };
15
16 struct TokenizerModel {
17     std::int32_t bos_id = -1;
18     std::int32_t eos_id = -1;
19     std::int32_t unk_id = -1;
20     std::string chat_template_hash;
21     std::vector<TokenEntry> vocab;
22 };
23
24 [[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);
25
26 [[nodiscard]] std::vector<std::int32_t> encode_prompt(
27     const TokenizerModel& tokenizer,
28     std::string_view prompt);
29
30 [[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& ↩
      ↪ tokenizer);
31
32 }  // namespace lcqi
```

safetensors 头文件固定 dtype、shape、offset 和 payload 边界。它负责模型字节怎样被解释，不应该和数学推理逻辑混在一起。

Listing 4.6 include/lcqi/safetensors.hpp

```
1  #pragma once
2
3  #include <cstdint>
4  #include <filesystem>
5  #include <span>
6  #include <string>
7  #include <string_view>
```

```cpp
#include <utility>
#include <vector>

namespace lcqi {

struct SafeTensorEntry {
    std::string name;
    std::string dtype;
    std::vector<std::int64_t> shape;
    std::uint64_t data_begin = 0;
    std::uint64_t data_end = 0;

    [[nodiscard]] std::uint64_t byte_size() const;
};

struct SafeTensorManifest {
    std::uint64_t header_size = 0;
    std::uint64_t data_start_offset = 0;
    std::vector<std::pair<std::string, std::string>> metadata;
    std::vector<SafeTensorEntry> tensors;

    [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) const;
};

struct SafeTensorFile {
    std::filesystem::path path;
    SafeTensorManifest manifest;
    std::vector<std::uint8_t> bytes;
};

[[nodiscard]] SafeTensorManifest load_safetensors_manifest(
    const std::filesystem::path& path);

[[nodiscard]] SafeTensorFile load_safetensors_file(const std::filesystem::path& path);

[[nodiscard]] std::vector<float> read_safetensor_f32_tensor(
    const SafeTensorFile& file,
    std::string_view name);

[[nodiscard]] std::span<const std::uint8_t> read_safetensor_tensor_bytes(
    const SafeTensorFile& file,
    std::string_view name);

}  // namespace lcqi
```

Reference Decoder 与 GPT-2 合同

reference decoder 头文件定义张量、层配置、trace checkpoint、采样结果和 decode 函数。它是定位推理错误的主要合同，不只返回最终 token。

Listing 4.7 include/lcqi/reference_decoder.hpp

```hpp
#pragma once

#include <cstdint>
#include <span>
#include <string>
#include <vector>

namespace lcqi {

struct LinearWeightsF32 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::vector<float> weights;
    std::vector<float> bias;
};

struct DecoderConfig {
    std::int32_t hidden_size = 0;
    std::int32_t query_heads = 0;
    std::int32_t kv_heads = 0;
    std::int32_t head_dim = 0;
    std::int32_t intermediate_size = 0;
    std::int32_t vocab_size = 0;
    std::int32_t max_context = 0;
    float rms_epsilon = 1.0e-5F;
    float rope_base = 10000.0F;
};

struct DecoderLayerWeightsF32 {
    std::vector<float> rms_attention_weight;
    LinearWeightsF32 wq;
    LinearWeightsF32 wk;
    LinearWeightsF32 wv;
    LinearWeightsF32 wo;
    std::vector<float> rms_mlp_weight;
    LinearWeightsF32 w_gate;
    LinearWeightsF32 w_up;
    LinearWeightsF32 w_down;
};

struct ReferenceDecoderModel {
    DecoderConfig config;
    std::vector<float> token_embedding;
    std::vector<DecoderLayerWeightsF32> layers;
    std::vector<float> final_norm_weight;
    LinearWeightsF32 lm_head;
};

struct ReferenceDecodeResult {
    std::vector<float> logits;
    std::int32_t predicted_token = 0;
```

```cpp
};

struct ReferenceTensorCheckpoint {
    std::string name;
    std::int32_t token_position = 0;
    std::int32_t layer_id = 0;
    std::vector<std::int32_t> shape;
    std::vector<std::int32_t> stride;
    std::string dtype;
    std::string layout;
    float checksum = 0.0F;
    float max_abs = 0.0F;
    std::vector<float> values;
};

struct ReferenceDecodeTraceResult {
    ReferenceDecodeResult result;
    std::vector<ReferenceTensorCheckpoint> checkpoints;
};

class ReferenceKVCache {
public:
    ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count);

    void append(std::int32_t layer_id,
                std::int32_t model_position,
                std::span<const float> key,
                std::span<const float> value);

    [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t kv_head) const;

    [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t kv_head) const;

    [[nodiscard]] std::int32_t layer_count() const noexcept;
    [[nodiscard]] std::int32_t filled_tokens() const noexcept;

private:
    [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t kv_head) const;
    void validate_address(std::int32_t layer_id,
                          std::int32_t model_position,
                          std::int32_t kv_head) const;

    DecoderConfig config_;
    std::int32_t layer_count_ = 0;
    std::int32_t filled_tokens_ = 0;
    std::vector<float> keys_;
```

```
    std::vector<float> values_;
    std::vector<std::uint8_t> written_;
};

void rms_norm(std::span<const float> input,
              std::span<const float> weight,
              float epsilon,
              std::span<float> output);

void linear_f32(const LinearWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output);

void apply_rope(std::span<float> heads,
                std::int32_t head_count,
                std::int32_t head_dim,
                std::int32_t model_position,
                float rope_base);

void attention_decode(const DecoderConfig& config,
                      const ReferenceKVCache& cache,
                      std::int32_t layer_id,
                      std::int32_t model_position,
                      std::span<const float> query,
                      std::span<float> output);

ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
                                           std::span<const std::int32_t> token_ids);

ReferenceDecodeTraceResult run_reference_decode_with_trace(
    const ReferenceDecoderModel& model,
    std::span<const std::int32_t> token_ids);

ReferenceDecoderModel make_tiny_reference_decoder_model();

}  // namespace lcqi
```

GPT-2 reference 头文件定义 GPT-2 配置、byte-level BPE tokenizer、HuggingFace safetensors 加载、forward、cached forward 和 greedy generation。它单独存在，是因为 GPT-2 使用 LayerNorm、绝对位置嵌入、packed QKV、GELU 和 tied lm head，不能直接塞进 LLaMA-like reference decoder。

Listing 4.8 include/lcqi/gpt2_reference.hpp

```
#pragma once

#include <cstdint>
#include <filesystem>
#include <memory>
#include <span>
#include <string>
#include <string_view>
#include <unordered_map>
```

```cpp
#include <vector>

namespace lcqi {

struct Gpt2Config {
    std::int32_t hidden_size = 0;
    std::int32_t head_count = 0;
    std::int32_t layer_count = 0;
    std::int32_t max_positions = 0;
    std::int32_t vocab_size = 0;
    std::int32_t intermediate_size = 0;
    std::int32_t bos_token_id = -1;
    std::int32_t eos_token_id = -1;
    float layer_norm_epsilon = 1.0e-5F;
    std::string activation_function = "gelu_new";
};

struct Gpt2LinearF32 {
    std::int32_t input_size = 0;
    std::int32_t output_size = 0;
    std::vector<float> weights;
    std::vector<float> bias;
};

struct Gpt2LayerWeightsF32 {
    std::vector<float> ln_1_weight;
    std::vector<float> ln_1_bias;
    Gpt2LinearF32 c_attn;
    Gpt2LinearF32 c_proj;
    std::vector<float> ln_2_weight;
    std::vector<float> ln_2_bias;
    Gpt2LinearF32 c_fc;
    Gpt2LinearF32 mlp_c_proj;
};

struct Gpt2ReferenceModel {
    Gpt2Config config;
    std::vector<float> token_embedding;
    std::vector<float> position_embedding;
    std::vector<Gpt2LayerWeightsF32> layers;
    std::vector<float> final_ln_weight;
    std::vector<float> final_ln_bias;
    std::vector<float> lm_head_weight;
    bool tie_lm_head_to_embedding = true;
};

struct Gpt2ForwardResult {
    std::vector<float> logits;
    std::int32_t predicted_token = 0;
};

struct Gpt2HotspotProfile {
```

```
    std::int64_t decoder_steps = 0;
    std::int64_t layer_steps = 0;
    double total_step_ms = 0.0;
    double embedding_ms = 0.0;
    double layer_norm_ms = 0.0;
    double final_norm_ms = 0.0;
    double qkv_projection_ms = 0.0;
    double kv_append_ms = 0.0;
    double attention_ms = 0.0;
    double attention_projection_ms = 0.0;
    double residual_add_ms = 0.0;
    double mlp_fc_ms = 0.0;
    double gelu_ms = 0.0;
    double mlp_projection_ms = 0.0;
    double lm_head_ms = 0.0;
    double logits_result_ms = 0.0;
};

struct Gpt2ExecutionOptions {
    std::int32_t worker_count = 0;
    std::int32_t parallel_min_rows = 512;
};

class Gpt2KvCache;

namespace detail {
class Gpt2ParallelWorkerPool;

void attend_cached_position(const Gpt2KvCache& cache,
                            std::int32_t layer_id,
                            std::int32_t model_position,
                            std::span<const float> query,
                            std::span<float> scores,
                            std::span<float> output);
}  // namespace detail

class Gpt2ForwardWorkspace {
public:
    explicit Gpt2ForwardWorkspace(const Gpt2Config& config);

    void reset_for_config(const Gpt2Config& config);

    [[nodiscard]] std::span<float> hidden() noexcept;
    [[nodiscard]] std::span<float> normed() noexcept;
    [[nodiscard]] std::span<float> query() noexcept;
    [[nodiscard]] std::span<float> key() noexcept;
    [[nodiscard]] std::span<float> value() noexcept;
    [[nodiscard]] std::span<float> attention() noexcept;
    [[nodiscard]] std::span<float> projected() noexcept;
    [[nodiscard]] std::span<float> qkv_packed() noexcept;
    [[nodiscard]] std::span<float> mlp_fc() noexcept;
    [[nodiscard]] std::span<float> mlp_out() noexcept;
```

```
    [[nodiscard]] std::span<float> logits() noexcept;
    [[nodiscard]] std::span<float> scores_prefix(std::int32_t count);

private:
    Gpt2Config config_;
    std::vector<float> hidden_;
    std::vector<float> normed_;
    std::vector<float> query_;
    std::vector<float> key_;
    std::vector<float> value_;
    std::vector<float> attention_;
    std::vector<float> projected_;
    std::vector<float> qkv_packed_;
    std::vector<float> mlp_fc_;
    std::vector<float> mlp_out_;
    std::vector<float> logits_;
    std::vector<float> scores_;
};

class Gpt2KvCache {
public:
    explicit Gpt2KvCache(const Gpt2Config& config);

    void append(std::int32_t layer_id,
                std::int32_t model_position,
                std::span<const float> key,
                std::span<const float> value);

    [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t head) const;

    [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t head) const;

    [[nodiscard]] std::int32_t filled_tokens() const noexcept;
    [[nodiscard]] std::size_t byte_size() const noexcept;

private:
    friend void detail::attend_cached_position(const Gpt2KvCache& cache,
                                               std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::span<const float> query,
                                               std::span<float> scores,
                                               std::span<float> output);

    [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t head) const;
    [[nodiscard]] std::size_t base_offset_unchecked(std::int32_t layer_id,
                                                    std::int32_t model_position
```

```
    ↪ ,
                                                    std::int32_t head) const ↩
    ↪ noexcept;
    [[nodiscard]] std::span<const float> key_unchecked(std::int32_t layer_id,
                                                       std::int32_t ↩
    ↪ model_position,
                                                       std::int32_t head) const↩
    ↪  noexcept;
    [[nodiscard]] std::span<const float> value_unchecked(std::int32_t layer_id,
                                                         std::int32_t ↩
    ↪ model_position,
                                                         std::int32_t head) ↩
    ↪ const noexcept;
    void validate_address(std::int32_t layer_id,
                          std::int32_t model_position,
                          std::int32_t head) const;
    void validate_written(std::int32_t layer_id,
                          std::int32_t model_position) const;

    Gpt2Config config_;
    std::int32_t filled_tokens_ = 0;
    std::vector<float> keys_;
    std::vector<float> values_;
    std::vector<std::uint8_t> written_;
};

class Gpt2CachedGreedyDecoder {
public:
    explicit Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model);
    Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
                            Gpt2HotspotProfile* hotspot_profile);
    Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
                            Gpt2HotspotProfile* hotspot_profile,
                            const Gpt2ExecutionOptions& execution_options);
    ~Gpt2CachedGreedyDecoder();

    Gpt2CachedGreedyDecoder(const Gpt2CachedGreedyDecoder&) = delete;
    Gpt2CachedGreedyDecoder& operator=(const Gpt2CachedGreedyDecoder&) = delete↩
    ↪ ;
    Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) noexcept;
    Gpt2CachedGreedyDecoder& operator=(Gpt2CachedGreedyDecoder&&) noexcept;

    [[nodiscard]] std::int32_t step(std::int32_t token_id);
    [[nodiscard]] Gpt2ForwardResult step_with_logits(std::int32_t token_id);
    [[nodiscard]] const Gpt2KvCache& cache() const noexcept;
    [[nodiscard]] std::int32_t filled_tokens() const noexcept;
    [[nodiscard]] std::size_t kv_cache_bytes() const noexcept;
    [[nodiscard]] std::int32_t worker_count() const noexcept;

private:
    const Gpt2ReferenceModel* model_;
    Gpt2HotspotProfile* hotspot_profile_;
```

```
    Gpt2ExecutionOptions execution_options_;
    Gpt2KvCache cache_;
    Gpt2ForwardWorkspace workspace_;
    std::unique_ptr<detail::Gpt2ParallelWorkerPool> worker_pool_;
};

struct Gpt2Tokenizer {
    std::int32_t bos_token_id = -1;
    std::int32_t eos_token_id = -1;
    std::unordered_map<std::string, std::int32_t> vocab;
    std::vector<std::string> id_to_token;
    std::unordered_map<std::string, std::int32_t> bpe_ranks;
};

[[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer(
    const std::filesystem::path& vocab_json,
    const std::filesystem::path& merges_txt,
    std::int32_t bos_token_id,
    std::int32_t eos_token_id);

[[nodiscard]] std::vector<std::int32_t> gpt2_encode(
    const Gpt2Tokenizer& tokenizer,
    std::string_view text);

[[nodiscard]] std::string gpt2_decode(
    const Gpt2Tokenizer& tokenizer,
    std::span<const std::int32_t> token_ids);

[[nodiscard]] Gpt2Config load_gpt2_config(const std::filesystem::path& ↩
    ↪ config_json);

[[nodiscard]] Gpt2ReferenceModel load_gpt2_reference_model(
    const std::filesystem::path& config_json,
    const std::filesystem::path& safetensors_path);

[[nodiscard]] Gpt2ReferenceModel load_gpt2_from_directory(
    const std::filesystem::path& model_directory);

[[nodiscard]] Gpt2ForwardResult run_gpt2_forward(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> token_ids);

[[nodiscard]] Gpt2ForwardResult run_gpt2_forward_cached(
    const Gpt2ReferenceModel& model,
    Gpt2KvCache& cache,
    std::int32_t token_id);

[[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens);

```

```
[[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy_cached(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens);

[[nodiscard]] Gpt2ReferenceModel make_tiny_gpt2_reference_model();

}  // namespace lcqi
```

最短闭环：从模型文件到 logits

tiny MLP 路径不是为了证明 LCQI 已经是完整大模型引擎，而是为了建立最短的、可手算的、可测试的推理闭环。读者要把模型加载、输入检查、int8 linear、ReLU、第二层 logits、argmax 和 CLI 输出连起来看。

模型加载与基础推理

`model.cpp` 负责解析 tiny 文本模型、权重、scale、bias 和 shape。坏格式、坏维度和坏数值都应该在这里被拒绝，不能拖到 kernel 内部才崩溃。

Listing 4.9 src/model.cpp

```
#include <lcqi/model.hpp>

#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;

std::string read_key(std::istream& input) {
    std::string key;
    if (!(input >> key)) {
        throw std::runtime_error("unexpected end of model file");
    }
    return key;
}

void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
    if (key != expected) {
        throw std::runtime_error("expected key '" + expected + "', got '" + key
    ↪  + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
```

```
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid int32 value for key '" + key + "'");
    }
    return value;
}

float read_float(std::istream& input, const std::string& key) {
    expect_key(input, key);
    float value = 0.0F;
    if (!(input >> value)) {
        throw std::runtime_error("invalid float value for key '" + key + "'");
    }
    return value;
}

std::vector<std::int8_t> read_i8_vector(std::istream& input,
                                        const std::string& key,
                                        std::int32_t count) {
    expect_key(input, key);
    if (count < 0) {
        throw std::runtime_error("negative vector size for key '" + key + "'");
    }
    std::vector<std::int8_t> values(static_cast<std::size_t>(count));
    for (std::int32_t i = 0; i < count; ++i) {
        std::int32_t raw = 0;
        if (!(input >> raw)) {
            throw std::runtime_error("invalid int8 payload for key '" + key + "'");
        }
        if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
            throw std::runtime_error("int8 payload out of range for key '" + key + "'");
        }
        values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
    }
    return values;
}

std::vector<float> read_float_vector(std::istream& input,
                                     const std::string& key,
                                     std::int32_t count) {
    expect_key(input, key);
    if (count < 0) {
        throw std::runtime_error("negative vector size for key '" + key + "'");
    }
    std::vector<float> values(static_cast<std::size_t>(count));
    for (std::int32_t i = 0; i < count; ++i) {
        if (!(input >> values[static_cast<std::size_t>(i)])) {
            throw std::runtime_error("invalid float payload for key '" + key + "'");
        }
```

```cpp
    }
    return values;
}

void validate_layer(const QuantizedLinearLayer& layer, const std::string& name) {
    if (layer.input_size <= 0 || layer.output_size <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::size_t expected_weights =
        static_cast<std::size_t>(layer.input_size) *
        static_cast<std::size_t>(layer.output_size);
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error(name + " layer weight size mismatch");
    }
    if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error(name + " layer bias size mismatch");
    }
}

std::int32_t checked_element_count(std::int32_t rows,
                                   std::int32_t columns,
                                   const std::string& name) {
    if (rows <= 0 || columns <= 0) {
        throw std::runtime_error(name + " layer has invalid shape");
    }
    const std::int64_t count =
        static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
    if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::max())) {
        throw std::runtime_error(name + " layer element count is too large");
    }
    return static_cast<std::int32_t>(count);
}

}  // namespace

TinyMlpModel load_model(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open model file: " + path.string());
    }

    expect_key(input, "LCQI_MODEL_V1");

    TinyMlpModel model;
    model.input_size = read_i32(input, "input_size");
    model.hidden_size = read_i32(input, "hidden_size");
    model.output_size = read_i32(input, "output_size");

    const std::int32_t hidden_weight_count =
        checked_element_count(model.input_size, model.hidden_size, "hidden");
```

```
    const std::int32_t output_weight_count =
        checked_element_count(model.hidden_size, model.output_size, "output");

    model.hidden.input_size = model.input_size;
    model.hidden.output_size = model.hidden_size;
    model.hidden.scale = read_float(input, "hidden_scale");
    model.hidden.weights = read_i8_vector(
        input,
        "hidden_weights",
        hidden_weight_count);
    model.hidden.bias = read_float_vector(input, "hidden_bias", model.hidden_size);

    model.output.input_size = model.hidden_size;
    model.output.output_size = model.output_size;
    model.output.scale = read_float(input, "output_scale");
    model.output.weights = read_i8_vector(
        input,
        "output_weights",
        output_weight_count);
    model.output.bias = read_float_vector(input, "output_bias", model.output_size);

    validate_layer(model.hidden, "hidden");
    validate_layer(model.output, "output");
    return model;
}

}  // namespace lcqi
```

`inference.cpp` 执行 int8 linear、ReLU、第二层 logits 和 predicted class。它是系统中最短的“模型资产到输出”路径。

Listing 4.10 src/inference.cpp

```
#include <lcqi/inference.hpp>

#include <algorithm>
#include <cmath>
#include <stdexcept>

namespace lcqi {
namespace {

void validate_input_size(std::span<const float> input, std::int32_t expected) {
    if (input.size() != static_cast<std::size_t>(expected)) {
        throw std::runtime_error("input size does not match model shape");
    }
}

std::int32_t argmax(std::span<const float> values) {
    if (values.empty()) {
        throw std::runtime_error("cannot take argmax of empty logits");
```

```
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
    for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i)
    {
        const float candidate = values[static_cast<std::size_t>(i)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = i;
        }
    }
    return best_index;
}

}  // namespace

void linear_i8(const QuantizedLinearLayer& layer,
               std::span<const float> input,
               std::span<float> output) {
    if (input.size() != static_cast<std::size_t>(layer.input_size)) {
        throw std::runtime_error("linear_i8 input size mismatch");
    }
    if (output.size() != static_cast<std::size_t>(layer.output_size)) {
        throw std::runtime_error("linear_i8 output size mismatch");
    }

    for (std::int32_t out = 0; out < layer.output_size; ++out) {
        const std::size_t row_base =
            static_cast<std::size_t>(out) * static_cast<std::size_t>(layer.
    input_size);
        float acc = layer.bias[static_cast<std::size_t>(out)];
        for (std::int32_t in = 0; in < layer.input_size; ++in) {
            const std::int8_t weight =
                layer.weights[row_base + static_cast<std::size_t>(in)];
            acc += static_cast<float>(weight) * layer.scale *
                   input[static_cast<std::size_t>(in)];
        }
        output[static_cast<std::size_t>(out)] = acc;
    }
}

void relu_inplace(std::span<float> values) {
    for (float& value : values) {
        value = std::max(0.0F, value);
    }
}

InferenceResult run_inference(const TinyMlpModel& model,
                              std::span<const float> input) {
    validate_input_size(input, model.input_size);

    std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F
```

```
    ↪ );
    std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F↩
    ↪ );

    linear_i8(model.hidden, input, hidden);
    relu_inplace(hidden);
    linear_i8(model.output, hidden, logits);

    InferenceResult result;
    result.predicted_class = argmax(logits);
    result.logits = std::move(logits);
    return result;
}

}  // namespace lcqi
```

main.cpp 把模型路径和输入接到 run_inference，并打印结果。CLI 应保持薄层，不复制模型解析和推理逻辑。

Listing 4.11 src/main.cpp

```
#include <lcqi/inference.hpp>
#include <lcqi/model.hpp>

#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <filesystem>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;

std::vector<float> parse_input(int argc, char** argv, std::int32_t ↩
    ↪ expected_size) {
    if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
        throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...>↩
    ↪  [repeat]");
    }
    std::vector<float> input(static_cast<std::size_t>(expected_size));
    for (std::int32_t i = 0; i < expected_size; ++i) {
        input[static_cast<std::size_t>(i)] =
            std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
    }
    return input;
}
```

```cpp

std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
    const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;
    if (argc <= repeat_index) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[repeat_index]));
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

}  // namespace

int main(int argc, char** argv) {
    try {
        if (argc < LCQI_EXPECTED_ARGC_MIN) {
            throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats> ...> [repeat]");
        }

        const std::filesystem::path model_path = argv[1];
        const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
        const std::vector<float> input = parse_input(argc, argv, model.input_size);
        const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);

        const lcqi::InferenceResult result = lcqi::run_inference(model, input);
        std::cout << "predicted_class " << result.predicted_class << "\n";
        std::cout << "logits";
        for (const float value : result.logits) {
            std::cout << ' ' << value;
        }
        std::cout << "\n";

        const auto begin = std::chrono::steady_clock::now();
        std::int32_t checksum = 0;
        for (std::int32_t i = 0; i < repeat; ++i) {
            checksum += lcqi::run_inference(model, input).predicted_class;
        }
        const auto end = std::chrono::steady_clock::now();
        const std::chrono::duration<double> elapsed = end - begin;
        const double average_us =
            elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
        std::cout << "repeat " << repeat << "\n";
        std::cout << "average_us " << average_us << "\n";
        std::cout << "checksum " << checksum << "\n";
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
```

```
        std::cerr << "error: " << error.what() << "\n";
        return EXIT_FAILURE;
    }
}
```

Int8 Kernel 基础实现

`int8_kernels.cpp` 包含 scalar、packed、workspace、deterministic fixture 和误差比较。它是 AVX2 路径的 reference 对照，也是 benchmark 的正确性底座。

Listing 4.12 src/int8_kernels.cpp

```
#include <lcqi/int8_kernels.hpp>

#include <lcqi/inference.hpp>

#include <algorithm>
#include <chrono>
#include <cmath>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
constexpr const char* LCQI_BACKEND_AVX2 = "avx2";

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

std::size_t checked_size(std::int32_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

void validate_quantized_layer(const QuantizedLinearLayer& layer) {
    require_positive(layer.input_size, "input_size");
```

```
    require_positive(layer.output_size, "output_size");
    const std::size_t expected_weights =
        checked_size(layer.input_size, "input_size") *
        checked_size(layer.output_size, "output_size");
    if (layer.weights.size() != expected_weights) {
        throw std::runtime_error("quantized layer weight size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("quantized layer bias size mismatch");
    }
}

std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
    require_positive(value, "value");
    require_positive(block_size, "block_size");
    return ((value + block_size - 1) / block_size) * block_size;
}

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
    result.backends.push_back(KernelBackendBenchmark{
        .name = name,
        .available = false,
        .average_us = 0.0,
        .max_abs_diff = 0.0F,
        .checksum = 0.0F,
    });
}

template <typename Callable>
double measure_average_us(std::int32_t repeat, Callable&& callable) {
    require_positive(repeat, "repeat");
    const auto begin = std::chrono::steady_clock::now();
    for (std::int32_t index = 0; index < repeat; ++index) {
        callable();
    }
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed = end - begin;
    return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);
}

}  // namespace

PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
```

```
                                std::int32_t output_block_size) {
    validate_quantized_layer(layer);
    require_positive(output_block_size, "output_block_size");

    PackedLinearI8 packed;
    packed.input_size = layer.input_size;
    packed.output_size = layer.output_size;
    packed.output_block_size = output_block_size;
    packed.scale = layer.scale;
    packed.bias = layer.bias;

    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, output_block_size);
    packed.packed_weights.assign(
        checked_size(padded_outputs, "padded_outputs") *
            checked_size(layer.input_size, "input_size"),
        0);

    std::size_t packed_index = 0;
    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += output_block_size) {
        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
            for (std::int32_t offset = 0; offset < output_block_size; ++offset) {
                const std::int32_t output_index = output_block + offset;
                if (output_index < layer.output_size) {
                    const std::size_t source =
                        checked_size(output_index, "output_index") *
                            checked_size(layer.input_size, "input_size") +
                        checked_size(input_index, "input_index");
                    packed.packed_weights[packed_index] = layer.weights[source];
                }
                ++packed_index;
            }
        }
    }
    return packed;
}

void linear_i8_packed(const PackedLinearI8& layer,
                      std::span<const float> input,
                      std::span<float> output) {
    std::vector<float> accumulators(
        checked_size(layer.output_block_size, "output_block_size"),
        0.0F);
    linear_i8_packed_with_workspace(layer, input, output, accumulators);
}

void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
                                     std::span<const float> input,
```

```
                                    std::span<float> output,
                                    std::span<float> accumulator_workspace) {
    require_positive(layer.input_size, "packed input_size");
    require_positive(layer.output_size, "packed output_size");
    require_positive(layer.output_block_size, "packed output_block_size");
    if (input.size() != checked_size(layer.input_size, "input_size")) {
        throw std::runtime_error("linear_i8_packed input size mismatch");
    }
    if (output.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("linear_i8_packed output size mismatch");
    }
    if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
        throw std::runtime_error("linear_i8_packed bias size mismatch");
    }

    const std::int32_t padded_outputs =
        round_up_to_block(layer.output_size, layer.output_block_size);
    const std::size_t expected_packed_size =
        checked_size(padded_outputs, "padded_outputs") *
        checked_size(layer.input_size, "input_size");
    if (layer.packed_weights.size() != expected_packed_size) {
        throw std::runtime_error("linear_i8_packed packed weight size mismatch");
    }
    if (accumulator_workspace.size() != checked_size(layer.output_block_size, "output_block_size")) {
        throw std::runtime_error("linear_i8_packed accumulator workspace size mismatch");
    }

    std::size_t packed_index = 0;
    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += layer.output_block_size) {
        std::fill(accumulator_workspace.begin(), accumulator_workspace.end(), 0.0F);
        for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                accumulator_workspace[checked_size(offset, "offset")] =
                    layer.bias[checked_size(output_index, "output_index")];
            }
        }

        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
            const float input_value = input[checked_size(input_index, "input_index")];
            for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
                const std::int32_t output_index = output_block + offset;
                const std::int8_t weight = layer.packed_weights[packed_index
```

```
    ↪ ++];
                if (output_index < layer.output_size) {
                    accumulator_workspace[checked_size(offset, "offset")] +=
                        static_cast<float>(weight) * layer.scale * input_value;
                }
            }
        }

        for (std::int32_t offset = 0; offset < layer.output_block_size; ++↩
    ↪ offset) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                output[checked_size(output_index, "output_index")] =
                    accumulator_workspace[checked_size(offset, "offset")];
            }
        }
    }
}

QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
                                                 std::int32_t output_size,
                                                 float scale) {
    require_positive(input_size, "input_size");
    require_positive(output_size, "output_size");
    QuantizedLinearLayer layer;
    layer.input_size = input_size;
    layer.output_size = output_size;
    layer.scale = scale;
    const std::size_t weight_count =
        checked_size(input_size, "input_size") * checked_size(output_size, "↩
    ↪ output_size");
    layer.weights.resize(weight_count);
    layer.bias.resize(checked_size(output_size, "output_size"));
    for (std::int32_t output_index = 0; output_index < output_size; ++↩
    ↪ output_index) {
        layer.bias[checked_size(output_index, "output_index")] =
            static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
        for (std::int32_t input_index = 0; input_index < input_size; ++↩
    ↪ input_index) {
            const std::int32_t raw =
                ((output_index * input_size + input_index) % ↩
    ↪ LCQI_I8_PATTERN_MODULUS) -
                LCQI_I8_PATTERN_CENTER;
            layer.weights[checked_size(output_index, "output_index") *
                              checked_size(input_size, "input_size") +
                          checked_size(input_index, "input_index")] =
                static_cast<std::int8_t>(raw);
        }
    }
    return layer;
```

```
}

std::vector<float> make_deterministic_input(std::int32_t input_size) {
    require_positive(input_size, "input_size");
    std::vector<float> input(checked_size(input_size, "input_size"));
    for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {
        input[checked_size(input_index, "input_index")] =
            static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
                               LCQI_INPUT_PATTERN_CENTER) *
            LCQI_INPUT_PATTERN_SCALE;
    }
    return input;
}

float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
    if (lhs.size() != rhs.size()) {
        throw std::runtime_error("max_abs_diff size mismatch");
    }
    float diff = 0.0F;
    for (std::size_t index = 0; index < lhs.size(); ++index) {
        diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
    }
    return diff;
}

KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& benchmark_case) {
    require_positive(benchmark_case.input_size, "benchmark input_size");
    require_positive(benchmark_case.output_size, "benchmark output_size");
    require_positive(benchmark_case.output_block_size, "benchmark output_block_size");
    require_positive(benchmark_case.repeat, "benchmark repeat");

    const QuantizedLinearLayer layer =
        make_deterministic_i8_layer(benchmark_case.input_size,
                                    benchmark_case.output_size,
                                    0.015625F);
    const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.output_block_size);
    const PackedLinearI8 packed_simd = pack_linear_i8(layer, LCQI_SIMD_OUTPUT_BLOCK);
    const std::vector<float> input = make_deterministic_input(benchmark_case.input_size);
    std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "output_size"),
                                     0.0F);
    std::vector<float> packed_output(scalar_output.size(), 0.0F);
    std::vector<float> simd_output(scalar_output.size(), 0.0F);
    std::vector<float> packed_workspace(
        checked_size(benchmark_case.output_block_size, "output_block_size"),
        0.0F);
```

```
276
277     linear_i8(layer, input, scalar_output);
278     linear_i8_packed_with_workspace(packed, input, packed_output, ↩
        ↪ packed_workspace);
279
280     KernelBenchmarkResult result;
281     result.benchmark_case = benchmark_case;
282     result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
283     result.backends.push_back(KernelBackendBenchmark{
284         .name = LCQI_BACKEND_SCALAR,
285         .available = true,
286         .average_us = measure_average_us(benchmark_case.repeat,
287                                          [&]() { linear_i8(layer, input, ↩
        ↪ scalar_output); }),
288         .max_abs_diff = 0.0F,
289         .checksum = checksum(scalar_output),
290     });
291     result.backends.push_back(KernelBackendBenchmark{
292         .name = LCQI_BACKEND_PACKED_SCALAR,
293         .available = true,
294         .average_us = measure_average_us(
295             benchmark_case.repeat,
296             [&]() { linear_i8_packed_with_workspace(packed,
297                                                     input,
298                                                     packed_output,
299                                                     packed_workspace); }),
300         .max_abs_diff = max_abs_diff(scalar_output, packed_output),
301         .checksum = checksum(packed_output),
302     });
303     if (linear_i8_packed_avx2_available()) {
304         linear_i8_packed_avx2(packed_simd, input, simd_output);
305         result.backends.push_back(KernelBackendBenchmark{
306             .name = LCQI_BACKEND_AVX2,
307             .available = true,
308             .average_us = measure_average_us(
309                 benchmark_case.repeat,
310                 [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output);↩
        ↪  }),
311             .max_abs_diff = max_abs_diff(scalar_output, simd_output),
312             .checksum = checksum(simd_output),
313         });
314     } else {
315         add_unavailable_backend(result, LCQI_BACKEND_AVX2);
316     }
317     return result;
318 }
319
320 }  // namespace lcqi
321
322 #if !defined(LCQI_ENABLE_AVX2)
323 namespace lcqi {
324
```

```
325 bool linear_i8_packed_avx2_available() noexcept {
326     return false;
327 }
328
329 void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::↩
    ↪ span<float>) {
330     throw std::runtime_error("AVX2 packed kernel is not enabled in this build")↩
    ↪ ;
331 }
332
333 }  // namespace lcqi
334 #endif
```

int8_kernels_avx2.cpp 是平台优化路径。读者要先看可用性判断，再看向量化实现；如果机器不支持 AVX2/FMA，报告必须写 unavailable。

Listing 4.13 src/int8_kernels_avx2.cpp

```
1  #include <lcqi/int8_kernels.hpp>
2
3  #if defined(LCQI_ENABLE_AVX2)
4
5  #include <immintrin.h>
6
7  #include <algorithm>
8  #include <cstdint>
9  #include <cstring>
10 #include <stdexcept>
11 #include <string>
12
13 namespace lcqi {
14 namespace {
15
16 constexpr std::int32_t LCQI_AVX2_OUTPUT_BLOCK = 8;
17
18 void require_positive(std::int32_t value, const char* name) {
19     if (value <= 0) {
20         throw std::runtime_error(std::string(name) + " must be positive");
21     }
22 }
23
24 std::size_t checked_size(std::int32_t value, const char* name) {
25     if (value < 0) {
26         throw std::runtime_error(std::string(name) + " cannot be negative");
27     }
28     return static_cast<std::size_t>(value);
29 }
30
31 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
32     require_positive(value, "value");
33     require_positive(block_size, "block_size");
34     return ((value + block_size - 1) / block_size) * block_size;
35 }
```

```
36
37 void validate_avx2_contract(const PackedLinearI8& layer,
38                              std::span<const float> input,
39                              std::span<float> output) {
40     require_positive(layer.input_size, "packed input_size");
41     require_positive(layer.output_size, "packed output_size");
42     if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
43         throw std::runtime_error("AVX2 packed kernel requires output_block_size
       ↪  == 8");
44     }
45     if (input.size() != checked_size(layer.input_size, "input_size")) {
46         throw std::runtime_error("AVX2 packed kernel input size mismatch");
47     }
48     if (output.size() != checked_size(layer.output_size, "output_size")) {
49         throw std::runtime_error("AVX2 packed kernel output size mismatch");
50     }
51     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
52         throw std::runtime_error("AVX2 packed kernel bias size mismatch");
53     }
54     const std::int32_t padded_outputs =
55         round_up_to_block(layer.output_size, layer.output_block_size);
56     const std::size_t expected_packed_size =
57         checked_size(padded_outputs, "padded_outputs") *
58         checked_size(layer.input_size, "input_size");
59     if (layer.packed_weights.size() != expected_packed_size) {
60         throw std::runtime_error("AVX2 packed kernel packed weight size
       ↪ mismatch");
61     }
62 }
63
64 }  // namespace
65
66 bool linear_i8_packed_avx2_available() noexcept {
67 #if defined(__GNUC__) || defined(__clang__)
68     __builtin_cpu_init();
69     return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
70 #else
71     return true;
72 #endif
73 }
74
75 void linear_i8_packed_avx2(const PackedLinearI8& layer,
76                            std::span<const float> input,
77                            std::span<float> output) {
78     if (!linear_i8_packed_avx2_available()) {
79         throw std::runtime_error("AVX2/FMA is not available on this CPU");
80     }
81     validate_avx2_contract(layer, input, output);
82
83     const std::int32_t padded_outputs =
84         round_up_to_block(layer.output_size, layer.output_block_size);
85     std::size_t packed_index = 0;
```

```
    alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};

    for (std::int32_t output_block = 0; output_block < padded_outputs;
         output_block += LCQI_AVX2_OUTPUT_BLOCK) {
        __m256 accumulator = _mm256_setzero_ps();

        for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
            const std::int32_t output_index = output_block + offset;
            block_output[checked_size(offset, "offset")] =
                output_index < layer.output_size
                    ? layer.bias[checked_size(output_index, "output_index")]
                    : 0.0F;
        }
        accumulator = _mm256_load_ps(block_output);

        for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
            std::uint64_t packed_bytes = 0;
            std::memcpy(&packed_bytes,
                        layer.packed_weights.data() + packed_index,
                        sizeof(packed_bytes));
            const __m128i packed_i8 =
                _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
            packed_index += LCQI_AVX2_OUTPUT_BLOCK;
            const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
            const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
            const __m256 input_scaled =
                _mm256_set1_ps(input[checked_size(input_index, "input_index")] * layer.scale);
            accumulator = _mm256_fmadd_ps(weights_f32, input_scaled, accumulator);
        }

        _mm256_store_ps(block_output, accumulator);
        for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
            const std::int32_t output_index = output_block + offset;
            if (output_index < layer.output_size) {
                output[checked_size(output_index, "output_index")] =
                    block_output[checked_size(offset, "offset")];
            }
        }
    }
}

}  // namespace lcqi

#endif
```

第零部分

Transformer、KV Cache 与服务运行时

第 5 章实践：Transformer 切片与 KV Cache 状态

本章对应原书中的第三册“AI 推理原理、KV cache 与计算账本”和“KV cache 实现细节”。不要把它当作概念复述，本章要把 RMSNorm、Q/K/V、RoPE、GQA、KV slot、position、state edge 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `kv_cache`，核心命令或测试入口是 `lcqi_trace`。

Listing 5.1 第 5 章实践：Transformer 切片与 KV Cache 状态的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / kv_cache
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：固定 prompt `tok1tok2tok3`，观察第一个 decode step。读者要先手写预期结果，再运行项目观察输出。动作是：把 trace 中 tokenizer、q/k/v、`kv_cache_slot`、attention 和 logits 串成状态表。如果输出里没有 `token_position` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：KV cache 是请求状态，不是临时 activation；position、layer、head、layout 和生命周期必须进入 trace 这条规律要写进报告，不要只写 “运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为 “接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 KV cache 状态表：每层每个 token 写入哪里、何时读取、何时禁止复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 6 章实践：Reference Decoder、logits 与采样

本章对应原书中的第三册“Reference decoder 实现”和“推理算法：logits、softmax、采样”。不要把它当作概念复述，本章要把 reference decoder、logits golden、argmax sampler、tokenizer contract hash、BOS/EOS 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `argmax`，核心命令或测试入口是 `lcqi_trace`。

Listing 6.1 第 6 章实践：Reference Decoder、logits 与采样的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / argmax
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：让 trace 固定输出 tokenizer contract 和 logits golden。读者要先手写预期结果，再运行项目观察输出。动作是：比较 generated token、logits 数组和 tokenizer hash。如果输出里没有 `tokenizer_contract` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：采样前必须先证明 logits 可复查；sampler 的随机性、温度和 top-k 只能建立在 reference 通过之后这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交采样报告：argmax、top-k、temperature 三种策略各自需要哪些额外字段才能复现。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 7 章实践：服务调度、SLO 与资源预算

本章对应原书中的第三册“业务服务实战：请求、调度、队列、取消与资源预算”。不要把它当作概念复述，本章要把 admission、KV budget、queue wait、TTFT、TPOT、cancel、reject、p95 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `ttft_ms`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 7.1 第 7 章实践：服务调度、SLO 与资源预算的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / ttft_ms
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：运行 serving probe 中短请求、RAG 请求、取消请求和超长请求。读者要先手写预期结果，再运行项目观察输出。动作是：解释 `memory_budget_exceeded`、cancel reclaim、`queue_wait_p95_ms` 和 `rejection_rate`。如果输出里没有 `rejection_rate` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：推理服务不是只看 tokens/s；是否接收请求、如何预留 KV、取消后是否回收，决定系统是否可运营这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 SLO 表：每类请求的 prompt/new token、预算、是否接受、TTFT、TPOT、拒绝原因和未验证边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 8 章实践：RAG、Agent 与状态图边界

本章对应原书中的第三册“上层 AI 应用编排：RAG、Agent、工具和状态图”。不要把它当作概念复述，本章要把 retrieval、tool call、prompt assembly、state graph、budget、trace id 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `req_rag`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 8.1 第 8 章实践：RAG、Agent 与状态图边界的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / req_rag
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 RAG 请求拆成 retrieve、rerank、prompt build、prefill、decode 五段。读者要先手写预期结果，再运行项目观察输出。动作是：为每段写输入输出和失败后能否重试。如果输出里没有 `prompt_tokens` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：上层编排不能把外部工具结果和模型状态混成字符串；每个边界都要有身份、预算和可重放记录这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 RAG 状态图：节点、边、超时、重试、缓存键、prompt 版本和审计字段。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

模型导入、AI 编译器与内存计划

第 9 章实践：模型导入、manifest 与 Shape Verifier

本章对应原书中的第三册“模型导入、manifest 与 shape verifier”。不要把它当作概念复述，本章要把 safetensors header、metadata、tensor name、dtype、shape、`data_offsets`、payload boundary 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `load_safetensors_manifest`，核心命令或测试入口是 `lcqi_tests`。

Listing 9.1 第 9 章实践：模型导入、manifest 与 Shape Verifier 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / load_safetensors_manifest
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：加载测试临时 safetensors fixture。读者要先手写预期结果，再运行项目观察输出。动作是：故意构造 offset 越界和 dtype/shape 字节数不匹配。如果输出里没有 `data_offsets` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型导入的第一职责是早拒绝：名字、shape、dtype、offset 和 payload 都必须在 kernel 运行前被验证这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 manifest gate 报告：合法 fixture 和两个坏 fixture 的字段、错误原因和拒绝位置。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 10 章实践：真实模型加载闭环与首 Token Trace

本章对应原书中的第三册“真实模型加载闭环：Tokenizer、权重格式与首 token trace”。不要把它当作概念复述，本章要把 tokenizer、weight mapping、chat template、first token trace、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

本章新增一个明确边界：tiny fixture 负责手算和逐层 oracle；LCQI GPT-2 reference path 负责证明自研代码已经能加载标准 GPT-2 safetensors、执行 byte-level BPE、跑 GPT-2 forward 并 greedy 生成；SmolLM2 small smoke 负责证明外部 llama.cpp 路径能加载 GGUF small 模型。三者不能互相冒充。tiny MLP 算对，不等于 GPT-2 完成；GPT-2 reference 能出一个 token，不等于高性能 KV cache runtime 完成；SmolLM2 能输出文本，也不等于 LCQI 自研 GGUF 后端完成。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里有三组核心对象。第一组是 `tiny_tokenizer.txt`、reference decoder 和 `lcqi_trace`，用于手算 token 合同和逐 checkpoint 追踪。第二组是 `gpt2_reference.hpp/.cpp`、`lcqi_gpt2` 和 `tools/run_gpt2_smoke.py`，用于自研 GPT-2 safetensors 路径。第三组是 `tools/run_smollm2_small_smoke.py`，用于下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，再通过 llama.cpp 在 CPU 上实际生成 assistant 文本。

Listing 10.1 第 10 章实践：真实模型加载闭环与首 Token Trace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / tiny_tokenizer.txt
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
make cpu3-gpt2-smoke
make cpu3-smollm2-smoke
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：从 tokenizer fixture 开始，不跳到真实 7B。读者要先手写预期结果，再运行项目观察输出。动作是：解释 BOS/EOS、UNK、template hash 和 decoder 输入 tokens 的边界。如果输出里没有 `tokens=[0,1,2,3,4]` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：真实模型加载不是下载权重；必须先让 tokenizer 合同、权重 role、shape verifier 和首 token trace 形成闭环。这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、自研 GPT-2 reference smoke：先把标准 safetensors 跑通

GPT-2 不是 LLaMA。它使用 LayerNorm，不是 RMSNorm；使用 learned absolute position embedding，不是 RoPE；attention 里的 Q/K/V 在 HuggingFace 权重中是 packed Conv1D 权重；MLP 使用 GELU，不是 SwiGLU；`lm_head` 通常和 token embedding 共享权重。把 GPT-2 硬塞进原来的 LLaMA-like tiny decoder，会让概念混乱，所以 LCQI 新增了独立的 `gpt2_reference` 模块。

仓库内最小验收分两层。第一层不下载模型，运行 tiny GPT-2 fixture：

Listing 10.2 仓库内 tiny GPT-2 数学闭环

```
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
```

它固定 prompt ids 为 `123`，检查 logits、predicted token 和 greedy generated ids。对应测试还会临时写出一个 HuggingFace 风格 `model.safetensors`，再用 loader 读回来，覆盖 tensor name mapping、shape verifier、Conv1D 转置和 tied lm head。坏 shape 必须在 loader 阶段被拒绝。

第二层下载真实 GPT-2 资产：

Listing 10.3 LCQI 自研 GPT-2 smoke

```
make cpu3-gpt2-smoke
```

脚本下载 `openai-community/gpt2` 的 `config.json`、`vocab.json`、`merges.txt` 和 `model.safetensors` 到用户缓存目录，不提交模型文件。最近一次报告写到：

Listing 10.4 GPT-2 smoke 报告关键字段

```
books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
prompt_ids 15496 11 616 1438 318
generated_ids 15496 11 616 1438 318 1757
generated_text Hello, my name is John
validation=PASS
```

这说明自研 LCQI C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer、权重导入、forward 和 greedy generation。边界也要写清：当前实现为了可读和可验证，会重跑完整前缀，没有 KV cache、没有 packed weight、没有多线程，也没有把 logits 与 PyTorch/Transformers 的逐数值 golden 报告纳入仓库。因此它是正确性闭环，不是性能结论。

四、真实 small 模型 smoke：不用 tiny，先让开源模型实际跑起来

本章的真实模型 smoke 固定为 `SmolLM2-135M-Instruct`，规模是 135M 参数，不是 tiny 随机夹具。脚本使用 GGUF 文件 `SmolLM2-135M-Instruct-Q4_K_M.gguf`，来源仓库是 `bartowski/SmolLM2-135M-Instruct-GGUF`，原模型组织是 `HuggingFaceTB`，license 字段为 `apache-2.0`。脚本不会把模型文件提交到仓库，而是缓存到用户目录；验收时检查文件大小 `105454432` 字节和 SHA256：

Listing 10.5 SmolLM2 small GGUF 文件身份

```
model_source_id=HuggingFaceTB/SmolLM2-135M-Instruct
model_gguf_repo_id=bartowski/SmolLM2-135M-Instruct-GGUF
model_file=SmolLM2-135M-Instruct-Q4_K_M.gguf
model_expected_bytes=105454432
model_expected_sha256=2↩
    ↪ e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```

运行命令是：

Listing 10.6 真实 small 模型 CPU 冒烟

```
make cpu3-smollm2-smoke
```

这条命令会做四步。第一，下载或复用 GGUF 文件，并校验 SHA256，防止“文件名对了但内容不对”。第二，拉取固定 commit 的 `llama.cpp` 并构建 `llama-simple-chat`，避免依赖本机临时安装。第三，用 `-ngl0` 强制 CPU 路径，输入一个短 prompt，确认程序能走过 tokenizer、chat template、prefill、decode 和 sampler。第四，把报告写到：

Listing 10.7 真实模型 smoke 报告路径

```
books/cpu-volume-3/results/lcqi-smollm2-small-model-smoke.txt
```

报告里必须至少出现这些字段：`model_expected_sha256`、`llama_cpp_commit`、`command`、`exit_code=0`、`validation=PASS` 和 `assistant_response`。如果只有下载日志，没有 assistant response，不能算通过；如果 assistant response 内容答错了，也不能改写成“质量通过”。本章只验证真实模型资产可加载、可 tokenize、可 decode、可生成非空文本。

五、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：运行 GPT-2 smoke。报告不许只贴“生成了 John”，必须解释 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors` 各自对应哪段代码，为什么 GPT-2 需要 byte-level BPE，为什么 HuggingFace Conv1D 权重要转置，为什么 `lm_head` 可以 tied 到 embedding。

任务四：运行真实 small 模型 smoke。报告不许只贴“命令执行成功”，必须解释脚本为什么校验模型 hash、为什么固定 `llama.cpp` commit、为什么使用 `-ngl0`，以及为什么 assistant response 非空只是 smoke，不是质量评测。

任务五：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。GPT-2 反例可以是：tensor name 缺失、shape 和 config 不一致、vocab 中缺少 BPE merge 后 token、`max_new_tokens` 超过 context。真实模型 smoke 的反例可以是：模型 SHA256 不匹配、网络不可用、`cmake` 缺失、assistant response 为空、命令超时。每个反例都要写清期望处理方式。

六、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。新增 GPT-2 smoke 后，最低验收还包括：真实 GPT-2 文件身份有 hash，prompt ids 可解释，generated ids 可解释，文本反解码可解释，并明确声明这不是高性能推理结论。新增 SmolLM2 smoke 后，最低验收还包括：模型身份有 hash，模型规模不是 tiny，CPU 路径实际运行，报告里有非空 assistant response，并明确声明这不是自研 LCQI 完整 GGUF 支持。

递进大作业：提交 GPT-2 reference path 的下一阶段迁移清单：KV cache、prefill/decode 分离、packed weight、线程并行、Transformers golden logits、更多模型方言 name mapping、分片 safetensors 和性能报告。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：Reference Decoder、Tokenizer、Safetensors 与 GPT-2

从 tiny MLP 走向 decoder-only 模型

Reference decoder 和 GPT-2 reference path 是 LCQI 从教学闭环走向真实模型导入的关键层。这里的代码更长，是因为真实模型不只有矩阵乘法：它还需要 tokenizer、位置、归一化、QKV 拆分、attention mask、KV cache、激活函数、lm head、权重命名映射、dtype 转换和坏资产拒绝。

Tiny Reference Decoder

`reference_decoder.cpp` 实现 embedding、RMSNorm、Q/K/V、RoPE、KV cache、attention、SwiGLU、final norm、logits 和 sampler，并在关键位置生成 trace checkpoint。读者应按 checkpoint 顺序读，而不是从最终 logits 倒推。

Listing 10.8 src/reference_decoder.cpp

```cpp
#include <lcqi/reference_decoder.hpp>

#include <algorithm>
#include <array>
#include <cmath>
#include <limits>
#include <stdexcept>
#include <string>

namespace lcqi {
namespace {

constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::infinity();
constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

void validate_config(const DecoderConfig& config) {
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.query_heads, "query_heads");
    require_positive(config.kv_heads, "kv_heads");
    require_positive(config.head_dim, "head_dim");
    require_positive(config.intermediate_size, "intermediate_size");
    require_positive(config.vocab_size, "vocab_size");
```

```
33      require_positive(config.max_context, "max_context");
34      if (config.query_heads % config.kv_heads != 0) {
35          throw std::runtime_error("query_heads must be divisible by kv_heads");
36      }
37      if (config.hidden_size != config.query_heads * config.head_dim) {
38          throw std::runtime_error("hidden_size must equal query_heads * head_dim
        ↪ ");
39      }
40      if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
41          throw std::runtime_error("head_dim must be even for RoPE");
42      }
43      if (config.rope_base <= 1.0F) {
44          throw std::runtime_error("rope_base must be greater than 1");
45      }
46  }
47
48  std::size_t checked_size(std::int32_t value, const char* name) {
49      if (value < 0) {
50          throw std::runtime_error(std::string(name) + " cannot be negative");
51      }
52      return static_cast<std::size_t>(value);
53  }
54
55  std::int32_t kv_width(const DecoderConfig& config) {
56      return config.kv_heads * config.head_dim;
57  }
58
59  void validate_linear(const LinearWeightsF32& layer, const char* name) {
60      require_positive(layer.input_size, "linear input_size");
61      require_positive(layer.output_size, "linear output_size");
62      const std::size_t expected_weights =
63          checked_size(layer.input_size, name) * checked_size(layer.output_size,
        ↪ name);
64      if (layer.weights.size() != expected_weights) {
65          throw std::runtime_error(std::string(name) + " weight size mismatch");
66      }
67      if (!layer.bias.empty() &&
68          layer.bias.size() != checked_size(layer.output_size, name)) {
69          throw std::runtime_error(std::string(name) + " bias size mismatch");
70      }
71  }
72
73  void validate_span_size(std::size_t actual, std::int32_t expected, const char*
        ↪ name) {
74      if (actual != checked_size(expected, name)) {
75          throw std::runtime_error(std::string(name) + " size mismatch");
76      }
77  }
78
79  std::int32_t argmax(std::span<const float> values) {
80      if (values.empty()) {
81          throw std::runtime_error("cannot take argmax of empty values");
```

```
    }
    std::int32_t best_index = 0;
    float best_value = values[0];
    for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size↩
    ↪ ()); ++index) {
        const float candidate = values[static_cast<std::size_t>(index)];
        if (candidate > best_value) {
            best_value = candidate;
            best_index = index;
        }
    }
    return best_index;
}

float silu(float value) {
    return value / (1.0F + std::exp(-value));
}

std::span<const float> row_span(std::span<const float> values,
                                std::int32_t row,
                                std::int32_t row_width) {
    const std::size_t begin = checked_size(row, "row") * checked_size(row_width↩
    ↪ , "row_width");
    return values.subspan(begin, checked_size(row_width, "row_width"));
}

void add_inplace(std::span<float> target, std::span<const float> source) {
    if (target.size() != source.size()) {
        throw std::runtime_error("residual add size mismatch");
    }
    for (std::size_t index = 0; index < target.size(); ++index) {
        target[index] += source[index];
    }
}

void swiglu_mlp(const DecoderLayerWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output) {
    std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0↩
    ↪ F);
    std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
    std::vector<float> hidden(gate.size(), 0.0F);
    linear_f32(layer.w_gate, input, gate);
    linear_f32(layer.w_up, input, up);
    if (gate.size() != up.size()) {
        throw std::runtime_error("SwiGLU gate/up size mismatch");
    }
    for (std::size_t index = 0; index < hidden.size(); ++index) {
        hidden[index] = silu(gate[index]) * up[index];
    }
    linear_f32(layer.w_down, hidden, output);
}
```

```cpp

float checksum(std::span<const float> values) {
    float sum = 0.0F;
    for (const float value : values) {
        sum += value;
    }
    return sum;
}

float max_abs(std::span<const float> values) {
    float result = 0.0F;
    for (const float value : values) {
        result = std::max(result, std::fabs(value));
    }
    return result;
}

std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape) {
    std::vector<std::int32_t> stride(shape.size(), 1);
    std::int32_t running = 1;
    for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; index >= 0; --index) {
        stride[checked_size(index, "stride index")] = running;
        running *= shape[checked_size(index, "shape index")];
    }
    return stride;
}

void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
                    const char* name,
                    std::int32_t token_position,
                    std::int32_t layer_id,
                    std::span<const std::int32_t> shape,
                    const char* layout,
                    std::span<const float> values) {
    if (checkpoints == nullptr) {
        return;
    }
    ReferenceTensorCheckpoint checkpoint;
    checkpoint.name = name;
    checkpoint.token_position = token_position;
    checkpoint.layer_id = layer_id;
    checkpoint.shape.assign(shape.begin(), shape.end());
    checkpoint.stride = contiguous_stride(shape);
    checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
    checkpoint.layout = layout;
    checkpoint.checksum = checksum(values);
    checkpoint.max_abs = max_abs(values);
    checkpoint.values.assign(values.begin(), values.end());
    checkpoints->push_back(std::move(checkpoint));
}
```

```
181
182 void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
183                            std::span<const float> input,
184                            std::span<float> output,
185                            std::int32_t token_position,
186                            std::int32_t layer_id,
187                            std::vector<ReferenceTensorCheckpoint>* checkpoints)↩
    ↪ {
188     std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0↩
    ↪ F);
189     std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
190     std::vector<float> hidden(gate.size(), 0.0F);
191     linear_f32(layer.w_gate, input, gate);
192     linear_f32(layer.w_up, input, up);
193     if (gate.size() != up.size()) {
194         throw std::runtime_error("SwiGLU gate/up size mismatch");
195     }
196     const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.↩
    ↪ output_size};
197     add_checkpoint(checkpoints,
198                    "mlp_gate",
199                    token_position,
200                    layer_id,
201                    intermediate_shape,
202                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
203                    gate);
204     add_checkpoint(checkpoints,
205                    "mlp_up",
206                    token_position,
207                    layer_id,
208                    intermediate_shape,
209                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
210                    up);
211     for (std::size_t index = 0; index < hidden.size(); ++index) {
212         hidden[index] = silu(gate[index]) * up[index];
213     }
214     add_checkpoint(checkpoints,
215                    "mlp_hidden",
216                    token_position,
217                    layer_id,
218                    intermediate_shape,
219                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
220                    hidden);
221     linear_f32(layer.w_down, hidden, output);
222 }
223
224 void forward_layer(const DecoderConfig& config,
225                    const DecoderLayerWeightsF32& layer,
226                    std::int32_t layer_id,
227                    std::int32_t model_position,
228                    ReferenceKVCache& cache,
229                    std::span<float> hidden,
```

```
                    std::vector<ReferenceTensorCheckpoint>* checkpoints = nullptr) {
    validate_span_size(hidden.size(), config.hidden_size, "hidden");

    std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> query(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> key(checked_size(kv_width(config), "kv_width"), 0.0F);
    std::vector<float> value(checked_size(kv_width(config), "kv_width"), 0.0F);
    std::vector<float> attention(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> attention_projected(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> mlp(checked_size(config.hidden_size, "hidden_size"), 0.0F);

    const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
    const std::array<std::int32_t, 2> query_shape{config.query_heads, config.head_dim};
    const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim};
    const std::array<std::int32_t, 4> kv_slot_shape{
        1, 1, config.kv_heads, config.head_dim};

    rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
    add_checkpoint(checkpoints,
                   "attn_norm",
                   model_position,
                   layer_id,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   normed);
    linear_f32(layer.wq, normed, query);
    linear_f32(layer.wk, normed, key);
    linear_f32(layer.wv, normed, value);
    add_checkpoint(checkpoints,
                   "q_before_rope",
                   model_position,
                   layer_id,
                   query_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   query);
    add_checkpoint(checkpoints,
                   "k_before_rope",
                   model_position,
                   layer_id,
                   kv_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   key);
    add_checkpoint(checkpoints,
                   "v",
```

```
                    model_position,
                    layer_id,
                    kv_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    value);
    apply_rope(query, config.query_heads, config.head_dim, model_position, config.rope_base);
    apply_rope(key, config.kv_heads, config.head_dim, model_position, config.rope_base);
    add_checkpoint(checkpoints,
                    "q_after_rope",
                    model_position,
                    layer_id,
                    query_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    query);
    add_checkpoint(checkpoints,
                    "k_after_rope",
                    model_position,
                    layer_id,
                    kv_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    key);

    cache.append(layer_id, model_position, key, value);
    add_checkpoint(checkpoints,
                    "kv_cache_key_slot",
                    model_position,
                    layer_id,
                    kv_slot_shape,
                    LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
                    key);
    attention_decode(config, cache, layer_id, model_position, query, attention);
    add_checkpoint(checkpoints,
                    "attention_out",
                    model_position,
                    layer_id,
                    hidden_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    attention);
    linear_f32(layer.wo, attention, attention_projected);
    add_inplace(hidden, attention_projected);
    add_checkpoint(checkpoints,
                    "post_attention_residual",
                    model_position,
                    layer_id,
                    hidden_shape,
                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
                    hidden);

    rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);
```

```
323     add_checkpoint(checkpoints,
324                    "mlp_norm",
325                    model_position,
326                    layer_id,
327                    hidden_shape,
328                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
329                    normed);
330     if (checkpoints == nullptr) {
331         swiglu_mlp(layer, normed, mlp);
332     } else {
333         swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, ↩
    ↪ checkpoints);
334     }
335     add_inplace(hidden, mlp);
336     add_checkpoint(checkpoints,
337                    "layer_out",
338                    model_position,
339                    layer_id,
340                    hidden_shape,
341                    LCQI_TRACE_LAYOUT_CONTIGUOUS,
342                    hidden);
343 }
344
345 LinearWeightsF32 make_linear(std::int32_t input_size,
346                              std::int32_t output_size,
347                              std::vector<float> weights,
348                              std::vector<float> bias = {}) {
349     LinearWeightsF32 layer;
350     layer.input_size = input_size;
351     layer.output_size = output_size;
352     layer.weights = std::move(weights);
353     layer.bias = std::move(bias);
354     validate_linear(layer, "tiny linear");
355     return layer;
356 }
357
358 std::vector<float> zeros(std::int32_t count) {
359     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
360 }
361
362 }  // namespace
363
364 ReferenceDecodeTraceResult run_reference_decode_with_trace(
365     const ReferenceDecoderModel& model,
366     std::span<const std::int32_t> token_ids) {
367     validate_config(model.config);
368     if (token_ids.empty()) {
369         throw std::runtime_error("reference decode needs at least one token");
370     }
371     if (token_ids.size() > checked_size(model.config.max_context, "max_context"↩
    ↪ )) {
372         throw std::runtime_error("token_ids exceed max_context");
```

```
    }
    if (model.token_embedding.size() !=
        checked_size(model.config.vocab_size, "vocab_size") *
            checked_size(model.config.hidden_size, "hidden_size")) {
        throw std::runtime_error("token embedding size mismatch");
    }
    if (model.final_norm_weight.size() != checked_size(model.config.hidden_size, "hidden_size")) {
        throw std::runtime_error("final norm weight size mismatch");
    }
    validate_linear(model.lm_head, "lm_head");

    ReferenceDecodeTraceResult trace;
    ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers.size()));
    std::vector<float> hidden(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
    const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
    for (std::int32_t position = 0; position < static_cast<std::int32_t>(token_ids.size());
         ++position) {
        const std::int32_t token_id = token_ids[checked_size(position, "position")];
        if (token_id < 0 || token_id >= model.config.vocab_size) {
            throw std::runtime_error("token id out of vocabulary");
        }
        const std::span<const float> embedding =
            row_span(model.token_embedding, token_id, model.config.hidden_size);
        std::copy(embedding.begin(), embedding.end(), hidden.begin());
        add_checkpoint(&trace.checkpoints,
                       "embedding",
                       position,
                       -1,
                       hidden_shape,
                       LCQI_TRACE_LAYOUT_ROW_MAJOR,
                       hidden);
        for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(model.layers.size());
             ++layer_id) {
            forward_layer(model.config,
                          model.layers[checked_size(layer_id, "layer_id")],
                          layer_id,
                          position,
                          cache,
                          hidden,
                          &trace.checkpoints);
        }
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "hidden_size"), 0.0F);
```

```
    std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size"), 0.0F);
    const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
    const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size()) - 1;
    rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed);
    add_checkpoint(&trace.checkpoints,
                   "final_norm",
                   last_position,
                   -1,
                   hidden_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   normed);
    linear_f32(model.lm_head, normed, logits);
    add_checkpoint(&trace.checkpoints,
                   "logits",
                   last_position,
                   -1,
                   logits_shape,
                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
                   logits);

    trace.result.predicted_token = argmax(logits);
    trace.result.logits = std::move(logits);
    return trace;
}

ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count)
    : config_(config),
      layer_count_(layer_count) {
    validate_config(this->config_);
    require_positive(this->layer_count_, "layer_count");
    const std::size_t element_count =
        checked_size(this->layer_count_, "layer_count") *
        checked_size(this->config_.max_context, "max_context") *
        checked_size(this->config_.kv_heads, "kv_heads") *
        checked_size(this->config_.head_dim, "head_dim");
    this->keys_.assign(element_count, 0.0F);
    this->values_.assign(element_count, 0.0F);
    this->written_.assign(
        checked_size(this->layer_count_, "layer_count") *
            checked_size(this->config_.max_context, "max_context"),
        0);
}

void ReferenceKVCache::append(std::int32_t layer_id,
                              std::int32_t model_position,
                              std::span<const float> key,
                              std::span<const float> value) {
    validate_span_size(key.size(), kv_width(this->config_), "KV key");
```

```
    validate_span_size(value.size(), kv_width(this->config_), "KV value");
    this->validate_address(layer_id, model_position, 0);
    for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head) {
        const std::size_t target = this->base_offset(layer_id, model_position, kv_head);
        const std::size_t source =
            checked_size(kv_head, "kv_head") * checked_size(this->config_.head_dim, "head_dim");
        std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
                    this->config_.head_dim,
                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
        std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
                    this->config_.head_dim,
                    this->values_.begin() + static_cast<std::ptrdiff_t>(target));
    }
    const std::size_t written_index =
        checked_size(layer_id, "layer_id") * checked_size(this->config_.max_context, "max_context") +
        checked_size(model_position, "model_position");
    this->written_[written_index] = 1;
    this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
}

std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
                                             std::int32_t model_position,
                                             std::int32_t kv_head) const {
    this->validate_address(layer_id, model_position, kv_head);
    const std::size_t offset = this->base_offset(layer_id, model_position, kv_head);
    return std::span<const float>(this->keys_.data() + offset,
                                  checked_size(this->config_.head_dim, "head_dim"));
}

std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t kv_head) const {
    this->validate_address(layer_id, model_position, kv_head);
    const std::size_t offset = this->base_offset(layer_id, model_position, kv_head);
    return std::span<const float>(this->values_.data() + offset,
                                  checked_size(this->config_.head_dim, "head_dim"));
}

std::int32_t ReferenceKVCache::layer_count() const noexcept {
    return this->layer_count_;
}

std::int32_t ReferenceKVCache::filled_tokens() const noexcept {
```

```
    return this->filled_tokens_;
}

std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t kv_head) const {
    return (((checked_size(layer_id, "layer_id") *
              checked_size(this->config_.max_context, "max_context")) +
             checked_size(model_position, "model_position")) *
                checked_size(this->config_.kv_heads, "kv_heads") +
            checked_size(kv_head, "kv_head")) *
           checked_size(this->config_.head_dim, "head_dim");
}

void ReferenceKVCache::validate_address(std::int32_t layer_id,
                                        std::int32_t model_position,
                                        std::int32_t kv_head) const {
    if (layer_id < 0 || layer_id >= this->layer_count_) {
        throw std::runtime_error("KV layer_id out of range");
    }
    if (model_position < 0 || model_position >= this->config_.max_context) {
        throw std::runtime_error("KV model_position out of range");
    }
    if (kv_head < 0 || kv_head >= this->config_.kv_heads) {
        throw std::runtime_error("KV head out of range");
    }
}

void rms_norm(std::span<const float> input,
              std::span<const float> weight,
              float epsilon,
              std::span<float> output) {
    if (input.size() != weight.size() || input.size() != output.size()) {
        throw std::runtime_error("RMSNorm size mismatch");
    }
    if (input.empty()) {
        throw std::runtime_error("RMSNorm input is empty");
    }
    float sum_square = 0.0F;
    for (const float value : input) {
        sum_square += value * value;
    }
    const float mean_square = sum_square / static_cast<float>(input.size());
    const float scale = 1.0F / std::sqrt(mean_square + epsilon);
    for (std::size_t index = 0; index < input.size(); ++index) {
        output[index] = input[index] * scale * weight[index];
    }
}

void linear_f32(const LinearWeightsF32& layer,
                std::span<const float> input,
                std::span<float> output) {
```

```
560     validate_linear(layer, "linear_f32");
561     validate_span_size(input.size(), layer.input_size, "linear input");
562     validate_span_size(output.size(), layer.output_size, "linear output");
563     for (std::int32_t out = 0; out < layer.output_size; ++out) {
564         const std::size_t row_base =
565             checked_size(out, "out") * checked_size(layer.input_size, "input_size");
566         float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size(out, "out")];
567         for (std::int32_t in = 0; in < layer.input_size; ++in) {
568             accumulator += layer.weights[row_base + checked_size(in, "in")] *
569                            input[checked_size(in, "in")];
570         }
571         output[checked_size(out, "out")] = accumulator;
572     }
573 }
574
575 void apply_rope(std::span<float> heads,
576                 std::int32_t head_count,
577                 std::int32_t head_dim,
578                 std::int32_t model_position,
579                 float rope_base) {
580     require_positive(head_count, "head_count");
581     require_positive(head_dim, "head_dim");
582     validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
583     if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
584         throw std::runtime_error("RoPE head_dim must be even");
585     }
586     if (model_position < 0) {
587         throw std::runtime_error("RoPE model_position cannot be negative");
588     }
589     if (rope_base <= 1.0F) {
590         throw std::runtime_error("RoPE base must be greater than 1");
591     }
592     for (std::int32_t head = 0; head < head_count; ++head) {
593         for (std::int32_t offset = 0; offset < head_dim; offset += LCQI_ROPE_PAIR_WIDTH) {
594             const float exponent = static_cast<float>(offset) / static_cast<float>(head_dim);
595             const float theta = 1.0F / std::pow(rope_base, exponent);
596             const float angle = static_cast<float>(model_position) * theta;
597             const float cosine = std::cos(angle);
598             const float sine = std::sin(angle);
599             const std::size_t first =
600                 checked_size(head, "head") * checked_size(head_dim, "head_dim") +
601                 checked_size(offset, "offset");
602             const float x0 = heads[first];
603             const float x1 = heads[first + 1];
604             heads[first] = x0 * cosine - x1 * sine;
605             heads[first + 1] = x0 * sine + x1 * cosine;
606         }
```

```
    }
}

void attention_decode(const DecoderConfig& config,
                      const ReferenceKVCache& cache,
                      std::int32_t layer_id,
                      std::int32_t model_position,
                      std::span<const float> query,
                      std::span<float> output) {
    validate_config(config);
    validate_span_size(query.size(), config.hidden_size, "attention query");
    validate_span_size(output.size(), config.hidden_size, "attention output");
    if (model_position < 0 || model_position >= cache.filled_tokens()) {
        throw std::runtime_error("attention model_position has not been written");
    }
    std::fill(output.begin(), output.end(), 0.0F);

    const std::int32_t group_size = config.query_heads / config.kv_heads;
    const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.head_dim));
    std::vector<float> scores(checked_size(model_position + 1, "score count"),
                              LCQI_SOFTMAX_NEGATIVE_INFINITY);
    std::vector<float> probabilities(scores.size(), 0.0F);

    for (std::int32_t query_head = 0; query_head < config.query_heads; ++query_head) {
        const std::int32_t kv_head = query_head / group_size;
        const std::size_t query_base =
            checked_size(query_head, "query_head") * checked_size(config.head_dim, "head_dim");
        float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
        for (std::int32_t position = 0; position <= model_position; ++position) {
            const std::span<const float> key = cache.key(layer_id, position, kv_head);
            float dot = 0.0F;
            for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
                dot += query[query_base + checked_size(dim, "dim")] *
                       key[checked_size(dim, "dim")];
            }
            const float score = dot * score_scale;
            scores[checked_size(position, "position")] = score;
            max_score = std::max(max_score, score);
        }

        float probability_sum = 0.0F;
        for (std::int32_t position = 0; position <= model_position; ++position) {
            const std::size_t index = checked_size(position, "position");
            const float unnormalized = std::exp(scores[index] - max_score);
            probabilities[index] = unnormalized;
```

```
652             probability_sum += unnormalized;
653         }
654         if (probability_sum <= 0.0F) {
655             throw std::runtime_error("attention probability sum is invalid");
656         }
657         for (std::int32_t position = 0; position <= model_position; ++position)↩
    ↪  {
658             const float probability =
659                 probabilities[checked_size(position, "position")] / ↩
    ↪ probability_sum;
660             const std::span<const float> value = cache.value(layer_id, position↩
    ↪ , kv_head);
661             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
662                 output[query_base + checked_size(dim, "dim")] +=
663                     probability * value[checked_size(dim, "dim")];
664             }
665         }
666     }
667 }
668
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670                                            std::span<const std::int32_t> ↩
    ↪ token_ids) {
671     validate_config(model.config);
672     if (token_ids.empty()) {
673         throw std::runtime_error("reference decode needs at least one token");
674     }
675     if (token_ids.size() > checked_size(model.config.max_context, "max_context"↩
    ↪ )) {
676         throw std::runtime_error("token_ids exceed max_context");
677     }
678     if (model.token_embedding.size() !=
679         checked_size(model.config.vocab_size, "vocab_size") *
680             checked_size(model.config.hidden_size, "hidden_size")) {
681         throw std::runtime_error("token embedding size mismatch");
682     }
683     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size↩
    ↪ , "hidden_size")) {
684         throw std::runtime_error("final norm weight size mismatch");
685     }
686     validate_linear(model.lm_head, "lm_head");
687
688     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers↩
    ↪ .size()));
689     std::vector<float> hidden(checked_size(model.config.hidden_size, "↩
    ↪ hidden_size"), 0.0F);
690     for (std::int32_t position = 0; position < static_cast<std::int32_t>(↩
    ↪ token_ids.size());
691          ++position) {
692         const std::int32_t token_id = token_ids[checked_size(position, "↩
    ↪ position")];
693         if (token_id < 0 || token_id >= model.config.vocab_size) {
```

```
694             throw std::runtime_error("token id out of vocabulary");
695         }
696         const std::span<const float> embedding =
697             row_span(model.token_embedding, token_id, model.config.hidden_size)↩
    ↪ ;
698         std::copy(embedding.begin(), embedding.end(), hidden.begin());
699         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↩
    ↪ model.layers.size());
700              ++layer_id) {
701             forward_layer(model.config,
702                           model.layers[checked_size(layer_id, "layer_id")],
703                           layer_id,
704                           position,
705                           cache,
706                           hidden);
707         }
708     }
709 
710     std::vector<float> normed(checked_size(model.config.hidden_size, "↩
    ↪ hidden_size"), 0.0F);
711     std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size↩
    ↪ "), 0.0F);
712     rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)↩
    ↪ ;
713     linear_f32(model.lm_head, normed, logits);
714 
715     ReferenceDecodeResult result;
716     result.predicted_token = argmax(logits);
717     result.logits = std::move(logits);
718     return result;
719 }
720 
721 ReferenceDecoderModel make_tiny_reference_decoder_model() {
722     ReferenceDecoderModel model;
723     model.config.hidden_size = 4;
724     model.config.query_heads = 2;
725     model.config.kv_heads = 1;
726     model.config.head_dim = 2;
727     model.config.intermediate_size = 6;
728     model.config.vocab_size = 5;
729     model.config.max_context = 8;
730     model.config.rms_epsilon = 1.0e-5F;
731     model.config.rope_base = 10000.0F;
732 
733     model.token_embedding = {
734         0.20F, -0.10F, 0.05F, 0.30F,
735         -0.40F, 0.10F, 0.25F, -0.20F,
736         0.15F, 0.35F, -0.30F, 0.05F,
737         0.50F, -0.25F, 0.10F, -0.15F,
738         -0.05F, 0.20F, 0.40F, -0.35F,
739     };
740 
```

```
    DecoderLayerWeightsF32 layer;
    layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
    layer.wq = make_linear(4,
                           4,
                           {
                               0.20F, -0.10F, 0.05F, 0.30F,
                               -0.15F, 0.25F, 0.10F, -0.05F,
                               0.05F, 0.10F, -0.20F, 0.15F,
                               0.30F, -0.05F, 0.20F, 0.10F,
                           });
    layer.wk = make_linear(4,
                           2,
                           {
                               0.10F, 0.20F, -0.10F, 0.05F,
                               -0.05F, 0.15F, 0.25F, -0.20F,
                           });
    layer.wv = make_linear(4,
                           2,
                           {
                               0.25F, -0.05F, 0.10F, 0.15F,
                               -0.10F, 0.30F, 0.05F, 0.20F,
                           });
    layer.wo = make_linear(4,
                           4,
                           {
                               0.20F, 0.10F, -0.05F, 0.15F,
                               -0.10F, 0.25F, 0.20F, -0.05F,
                               0.05F, -0.20F, 0.30F, 0.10F,
                               0.15F, 0.05F, -0.10F, 0.25F,
                           });
    layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
    layer.w_gate = make_linear(4,
                               6,
                               {
                                   0.20F, -0.10F, 0.05F, 0.10F,
                                   -0.05F, 0.15F, 0.20F, -0.10F,
                                   0.10F, 0.05F, -0.15F, 0.25F,
                                   -0.20F, 0.10F, 0.15F, 0.05F,
                                   0.05F, -0.25F, 0.10F, 0.20F,
                                   0.15F, 0.05F, 0.05F, -0.15F,
                               });
    layer.w_up = make_linear(4,
                             6,
                             {
                                 0.10F, 0.20F, -0.05F, 0.05F,
                                 -0.15F, 0.05F, 0.25F, 0.10F,
                                 0.20F, -0.10F, 0.05F, 0.15F,
                                 0.05F, 0.10F, -0.20F, 0.25F,
                                 -0.05F, 0.15F, 0.10F, -0.10F,
                                 0.25F, -0.05F, 0.05F, 0.10F,
                             });
    layer.w_down = make_linear(6,
```

```
                                    4,
                                    {
                                        0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
                                        -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
                                        0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
                                        0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
                                    });
    model.layers.push_back(std::move(layer));
    model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
    model.lm_head = make_linear(4,
                                     5,
                                     {
                                         0.20F, -0.10F, 0.05F, 0.15F,
                                         -0.05F, 0.25F, 0.10F, -0.20F,
                                         0.15F, 0.05F, -0.25F, 0.10F,
                                         -0.10F, 0.20F, 0.15F, 0.05F,
                                         0.05F, -0.15F, 0.20F, 0.25F,
                                     },
                                     zeros(5));
    return model;
}

}  // namespace lcqi
```

Tokenizer 与 Safetensors 实现

tokenizer.cpp 把 prompt 变成 token id，并固定 unknown token 行为。读者应检查 BOS/EOS 是否稳定插入，UNK 是否按合同处理。

Listing 10.9 src/tokenizer.cpp

```
#include <lcqi/tokenizer.hpp>

#include <cstddef>
#include <fstream>
#include <stdexcept>
#include <string>
#include <string_view>

namespace lcqi {
namespace {

constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 14695981039346656037ULL;
constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;

std::string read_key(std::istream& input) {
    std::string key;
    if (!(input >> key)) {
        throw std::runtime_error("unexpected end of tokenizer file");
    }
    return key;
```

```cpp
}

void expect_key(std::istream& input, const std::string& expected) {
    const std::string key = read_key(input);
    if (key != expected) {
        throw std::runtime_error("expected tokenizer key '" + expected + "', got '" + key + "'");
    }
}

std::int32_t read_i32(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::int32_t value = 0;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer int32 value for key '" + key + "'");
    }
    return value;
}

std::string read_string(std::istream& input, const std::string& key) {
    expect_key(input, key);
    std::string value;
    if (!(input >> value)) {
        throw std::runtime_error("invalid tokenizer string value for key '" + key + "'");
    }
    return value;
}

std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view text) {
    for (const TokenEntry& entry : tokenizer.vocab) {
        if (entry.text == text) {
            return entry.id;
        }
    }
    return LCQI_INVALID_TOKEN_ID;
}

void validate_tokenizer(const TokenizerModel& tokenizer) {
    if (tokenizer.vocab.empty()) {
        throw std::runtime_error("tokenizer vocab is empty");
    }
    for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
        const TokenEntry& left = tokenizer.vocab[i];
        if (left.id < 0) {
            throw std::runtime_error("tokenizer token id must be non-negative");
        }
        for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
            const TokenEntry& right = tokenizer.vocab[j];
```

```
            if (left.id == right.id) {
                throw std::runtime_error("duplicate tokenizer token id");
            }
            if (left.text == right.text) {
                throw std::runtime_error("duplicate tokenizer token text");
            }
        }
    }
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
        throw std::runtime_error("tokenizer BOS id does not match vocab");
    }
    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
        lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
        throw std::runtime_error("tokenizer EOS id does not match vocab");
    }
}

std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
    hash ^= static_cast<std::uint64_t>(value);
    hash *= LCQI_FNV_PRIME;
    return hash;
}

std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
    for (const unsigned char value : text) {
        hash = hash_byte(hash, value);
    }
    return hash;
}

std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
    const std::uint32_t bits = static_cast<std::uint32_t>(value);
    for (std::int32_t shift = 0; shift < 32; shift += 8) {
        hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0xFFU));
    }
    return hash;
}

}  // namespace

TokenizerModel load_tokenizer(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open tokenizer file: " + path.string());
    }

    expect_key(input, "LCQI_TOKENIZER_V1");

    TokenizerModel tokenizer;
```

```
    tokenizer.bos_id = read_i32(input, "bos_id");
    tokenizer.eos_id = read_i32(input, "eos_id");
    tokenizer.unk_id = read_i32(input, "unk_id");
    tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
    const std::int32_t vocab_size = read_i32(input, "vocab_size");
    if (vocab_size <= 0) {
        throw std::runtime_error("tokenizer vocab_size must be positive");
    }

    tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
    expect_key(input, "vocab");
    for (std::int32_t i = 0; i < vocab_size; ++i) {
        TokenEntry entry;
        if (!(input >> entry.id >> entry.text)) {
            throw std::runtime_error("invalid tokenizer vocab entry");
        }
        tokenizer.vocab.push_back(entry);
    }

    validate_tokenizer(tokenizer);
    return tokenizer;
}

std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
                                        std::string_view prompt) {
    std::vector<std::int32_t> ids;
    if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.bos_id);
    }

    std::size_t begin = 0;
    while (begin < prompt.size()) {
        while (begin < prompt.size() && prompt[begin] == ' ') {
            ++begin;
        }
        if (begin >= prompt.size()) {
            break;
        }
        std::size_t end = begin;
        while (end < prompt.size() && prompt[end] != ' ') {
            ++end;
        }
        const std::string_view token = prompt.substr(begin, end - begin);
        const std::int32_t token_id = lookup_token_id(tokenizer, token);
        if (token_id == LCQI_INVALID_TOKEN_ID) {
            if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
                throw std::runtime_error("tokenizer encountered unknown token")
    ↪ ;
            }
            ids.push_back(tokenizer.unk_id);
        } else {
            ids.push_back(token_id);
```

```
        }
        begin = end;
    }

    if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
        ids.push_back(tokenizer.eos_id);
    }
    return ids;
}

std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
    std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
    hash = hash_string(hash, tokenizer.chat_template_hash);
    hash = hash_i32(hash, tokenizer.bos_id);
    hash = hash_i32(hash, tokenizer.eos_id);
    hash = hash_i32(hash, tokenizer.unk_id);
    for (const TokenEntry& entry : tokenizer.vocab) {
        hash = hash_string(hash, entry.text);
        hash = hash_i32(hash, entry.id);
    }
    return hash;
}

}  // namespace lcqi
```

`safetensors.cpp` 解析 header、metadata、dtype、shape 和 data offsets。坏 dtype、坏 shape、坏 offset 和 payload 太短都应该被拒绝。

Listing 10.10 src/safetensors.cpp

```
#include <lcqi/safetensors.hpp>

#include <algorithm>
#include <cctype>
#include <cstddef>
#include <cmath>
#include <cstring>
#include <fstream>
#include <limits>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace lcqi {
namespace {

constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U;
constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
constexpr std::uint64_t LCQI_BYTE_BITS = 8;
constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;
```

```
constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
constexpr std::uint32_t LCQI_F32_EXPONENT_MASK = 0x7F800000U;
constexpr std::uint32_t LCQI_F16_SIGN_MASK = 0x8000U;
constexpr std::uint32_t LCQI_F16_EXPONENT_MASK = 0x7C00U;
constexpr std::uint32_t LCQI_F16_MANTISSA_MASK = 0x03FFU;
constexpr std::uint32_t LCQI_F16_EXPONENT_BIAS = 15U;
constexpr std::uint32_t LCQI_F32_EXPONENT_BIAS = 127U;
constexpr std::uint32_t LCQI_F32_MANTISSA_BITS = 23U;
constexpr std::uint32_t LCQI_F16_MANTISSA_BITS = 10U;
constexpr std::uint32_t LCQI_F16_TO_F32_SIGN_SHIFT = 16U;

class HeaderCursor {
public:
    explicit HeaderCursor(std::string_view text) : text_(text) {}

    void expect(char expected) {
        this->skip_ws();
        if (this->eof() || this->text_[this->position_] != expected) {
            throw std::runtime_error("invalid safetensors header syntax");
        }
        ++this->position_;
    }

    [[nodiscard]] bool consume(char expected) {
        this->skip_ws();
        if (!this->eof() && this->text_[this->position_] == expected) {
            ++this->position_;
            return true;
        }
        return false;
    }

    [[nodiscard]] std::string parse_string() {
        this->skip_ws();
        this->expect('"');
        std::string result;
        while (!this->eof()) {
            const char ch = this->text_[this->position_++];
            if (ch == '"') {
                return result;
            }
            if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) ↩
    ↪ {
```

```
                throw std::runtime_error("control character in safetensors string");
            }
            if (ch == '\\') {
                result.push_back(this->parse_escape());
            } else {
                result.push_back(ch);
            }
        }
        throw std::runtime_error("unterminated safetensors string");
    }

    [[nodiscard]] std::uint64_t parse_u64() {
        this->skip_ws();
        if (this->eof() ||
            !std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            throw std::runtime_error("expected unsigned integer in safetensors header");
        }
        std::uint64_t value = 0;
        while (!this->eof() &&
               std::isdigit(static_cast<unsigned char>(this->text_[this->position_]))) {
            const std::uint64_t digit =
                static_cast<std::uint64_t>(this->text_[this->position_] - '0');
            if (value >
                (std::numeric_limits<std::uint64_t>::max() - digit) / LCQI_DECIMAL_BASE) {
                throw std::runtime_error("safetensors integer overflow");
            }
            value = value * LCQI_DECIMAL_BASE + digit;
            ++this->position_;
        }
        return value;
    }

    [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
        std::vector<std::int64_t> values;
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
        while (true) {
            const std::uint64_t raw = this->parse_u64();
            if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::int64_t>::max())) {
                throw std::runtime_error("safetensors shape value is too large");
            }
            values.push_back(static_cast<std::int64_t>(raw));
            if (this->consume(']')) {
```

```
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
        std::vector<std::uint64_t> values;
        this->expect('[');
        if (this->consume(']')) {
            return values;
        }
        while (true) {
            values.push_back(this->parse_u64());
            if (this->consume(']')) {
                return values;
            }
            this->expect(',');
        }
    }

    [[nodiscard]] bool eof_after_ws() {
        this->skip_ws();
        return this->eof();
    }

private:
    std::string_view text_;
    std::size_t position_ = 0;

    [[nodiscard]] bool eof() const {
        return this->position_ >= this->text_.size();
    }

    void skip_ws() {
        while (!this->eof() &&
               std::isspace(static_cast<unsigned char>(this->text_[this->position_]))) {
            ++this->position_;
        }
    }

    [[nodiscard]] char parse_escape() {
        if (this->eof()) {
            throw std::runtime_error("unterminated safetensors escape");
        }
        const char escaped = this->text_[this->position_++];
        switch (escaped) {
            case '"':
            case '\\':
            case '/':
                return escaped;
```

```
            case 'b':
                return '\b';
            case 'f':
                return '\f';
            case 'n':
                return '\n';
            case 'r':
                return '\r';
            case 't':
                return '\t';
            default:
                throw std::runtime_error("unsupported safetensors string escape");
        }
    }
};

std::uint64_t read_le_u64(std::span<const std::uint8_t> bytes) {
    if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
        throw std::runtime_error("safetensors file is too small");
    }
    std::uint64_t value = 0;
    for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
        value |= static_cast<std::uint64_t>(bytes[i]) << (LCQI_BYTE_BITS * i);
    }
    return value;
}

std::string read_file_bytes(const std::filesystem::path& path) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open safetensors file: " + path.string());
    }
    input.seekg(0, std::ios::end);
    const std::streamoff size = input.tellg();
    if (size < 0) {
        throw std::runtime_error("cannot determine safetensors file size");
    }
    input.seekg(0, std::ios::beg);
    std::string bytes(static_cast<std::size_t>(size), '\0');
    if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize>(bytes.size()))) {
        throw std::runtime_error("cannot read safetensors file");
    }
    return bytes;
}

std::vector<std::uint8_t> read_file_u8(const std::filesystem::path& path) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open safetensors file: " + path.string());
```

```
    ↪ ());
    }
    input.seekg(0, std::ios::end);
    const std::streamoff size = input.tellg();
    if (size < 0) {
        throw std::runtime_error("cannot determine safetensors file size");
    }
    input.seekg(0, std::ios::beg);
    std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size), 0);
    if (!bytes.empty() &&
        !input.read(reinterpret_cast<char*>(bytes.data()),
                    static_cast<std::streamsize>(bytes.size()))) {
        throw std::runtime_error("cannot read safetensors file");
    }
    return bytes;
}

void parse_metadata_value(HeaderCursor& cursor,
                          SafeTensorManifest& manifest,
                          const std::string& key) {
    if (key == "__metadata__") {
        cursor.expect('{');
        if (cursor.consume('}')) {
            return;
        }
        while (true) {
            const std::string metadata_key = cursor.parse_string();
            cursor.expect(':');
            const std::string metadata_value = cursor.parse_string();
            manifest.metadata.emplace_back(metadata_key, metadata_value);
            if (cursor.consume('}')) {
                return;
            }
            cursor.expect(',');
        }
    }
    throw std::runtime_error("unexpected safetensors metadata value");
}

SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& ↩
    ↪ name) {
    SafeTensorEntry entry;
    entry.name = name;
    bool saw_dtype = false;
    bool saw_shape = false;
    bool saw_offsets = false;

    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("empty safetensors tensor entry");
    }
    while (true) {
```

```
269         const std::string field = cursor.parse_string();
270         cursor.expect(':');
271         if (field == "dtype") {
272             entry.dtype = cursor.parse_string();
273             saw_dtype = true;
274         } else if (field == "shape") {
275             entry.shape = cursor.parse_i64_array();
276             saw_shape = true;
277         } else if (field == "data_offsets") {
278             const std::vector<std::uint64_t> offsets = cursor.parse_u64_array()
    ↪ ;
279             if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
280                 throw std::runtime_error("safetensors data_offsets must contain
    ↪  two values");
281             }
282             entry.data_begin = offsets[0];
283             entry.data_end = offsets[1];
284             saw_offsets = true;
285         } else {
286             throw std::runtime_error("unsupported safetensors tensor field: " +
    ↪  field);
287         }
288         if (cursor.consume('}')) {
289             break;
290         }
291         cursor.expect(',');
292     }
293
294     if (!saw_dtype || !saw_shape || !saw_offsets) {
295         throw std::runtime_error("safetensors tensor entry is missing required
    ↪ fields");
296     }
297     if (entry.data_end < entry.data_begin) {
298         throw std::runtime_error("safetensors tensor offset range is invalid");
299     }
300     return entry;
301 }
302
303 void parse_header(std::string_view header, SafeTensorManifest& manifest) {
304     HeaderCursor cursor(header);
305     cursor.expect('{');
306     if (cursor.consume('}')) {
307         throw std::runtime_error("safetensors header is empty");
308     }
309
310     while (true) {
311         const std::string key = cursor.parse_string();
312         cursor.expect(':');
313         if (key == "__metadata__") {
314             parse_metadata_value(cursor, manifest, key);
315         } else {
316             manifest.tensors.push_back(parse_tensor_entry(cursor, key));
```

```
317         }
318         if (cursor.consume('}')) {
319             break;
320         }
321         cursor.expect(',');
322     }
323 
324     if (!cursor.eof_after_ws()) {
325         throw std::runtime_error("trailing data after safetensors header");
326     }
327 }
328 
329 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);
330 
331 void validate_manifest(const SafeTensorManifest& manifest,
332                        std::uint64_t file_size) {
333     const std::uint64_t payload_size = file_size - manifest.data_start_offset;
334     for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
335         const SafeTensorEntry& left = manifest.tensors[i];
336         if (left.name.empty() || left.dtype.empty()) {
337             throw std::runtime_error("safetensors tensor has empty name or ↩
    ↪ dtype");
338         }
339         if (left.data_end > payload_size) {
340             throw std::runtime_error("safetensors tensor data_offsets exceed ↩
    ↪ payload size");
341         }
342         const std::uint64_t expected_bytes = expected_tensor_bytes(left);
343         if (left.byte_size() != expected_bytes) {
344             throw std::runtime_error("safetensors tensor byte size does not ↩
    ↪ match dtype and shape");
345         }
346         for (const std::int64_t dim : left.shape) {
347             if (dim < 0) {
348                 throw std::runtime_error("safetensors shape dimension is ↩
    ↪ negative");
349             }
350         }
351         for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
352             const SafeTensorEntry& right = manifest.tensors[j];
353             if (left.name == right.name) {
354                 throw std::runtime_error("duplicate safetensors tensor name");
355             }
356             const bool overlaps =
357                 left.data_begin < right.data_end && right.data_begin < left.↩
    ↪ data_end;
358             if (overlaps) {
359                 throw std::runtime_error("overlapping safetensors tensor data ↩
    ↪ range");
360             }
361         }
362     }
```

```
}

std::uint64_t dtype_byte_size(std::string_view dtype) {
    if (dtype == "F64") {
        return LCQI_DTYPE_BYTES_F64;
    }
    if (dtype == "F32") {
        return LCQI_DTYPE_BYTES_F32;
    }
    if (dtype == "F16" || dtype == "BF16") {
        return LCQI_DTYPE_BYTES_F16;
    }
    if (dtype == "I64" || dtype == "U64") {
        return LCQI_DTYPE_BYTES_I64;
    }
    if (dtype == "I32" || dtype == "U32") {
        return LCQI_DTYPE_BYTES_I32;
    }
    if (dtype == "I16" || dtype == "U16") {
        return LCQI_DTYPE_BYTES_I16;
    }
    if (dtype == "I8") {
        return LCQI_DTYPE_BYTES_I8;
    }
    if (dtype == "U8" || dtype == "BOOL") {
        return LCQI_DTYPE_BYTES_U8;
    }
    throw std::runtime_error("unsupported safetensors dtype: " + std::string(dtype));
}

std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
    std::uint64_t element_count = 1;
    for (const std::int64_t dim : entry.shape) {
        if (dim < 0) {
            throw std::runtime_error("safetensors shape dimension is negative");
        }
        const std::uint64_t extent = static_cast<std::uint64_t>(dim);
        if (extent != 0 &&
            element_count > std::numeric_limits<std::uint64_t>::max() / extent) {
            throw std::runtime_error("safetensors shape element count overflow");
        }
        element_count *= extent;
    }
    const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
    if (dtype_bytes != 0 &&
        element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes) {
        throw std::runtime_error("safetensors tensor byte size overflow");
```

```
410     }
411     return element_count * dtype_bytes;
412 }
413 
414 SafeTensorManifest parse_manifest_from_bytes(std::span<const std::uint8_t> ↩
        ↪ bytes) {
415     const std::uint64_t header_size = read_le_u64(bytes);
416     if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
417         throw std::runtime_error("safetensors header size is invalid");
418     }
419     const std::uint64_t data_start_offset =
420         static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size↩
        ↪ ;
421     if (data_start_offset > bytes.size()) {
422         throw std::runtime_error("safetensors header extends beyond file size")↩
        ↪ ;
423     }
424 
425     SafeTensorManifest manifest;
426     manifest.header_size = header_size;
427     manifest.data_start_offset = data_start_offset;
428     const char* header_begin =
429         reinterpret_cast<const char*>(bytes.data() + ↩
        ↪ LCQI_SAFETENSORS_PREFIX_BYTES);
430     const std::string_view header(header_begin, static_cast<std::size_t>(↩
        ↪ header_size));
431     parse_header(header, manifest);
432     validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
433     return manifest;
434 }
435 
436 std::uint16_t read_le_u16(std::span<const std::uint8_t> bytes, std::size_t ↩
        ↪ offset) {
437     return static_cast<std::uint16_t>(
438         static_cast<std::uint16_t>(bytes[offset]) |
439         (static_cast<std::uint16_t>(bytes[offset + 1]) << LCQI_BYTE_BITS));
440 }
441 
442 std::uint32_t read_le_u32(std::span<const std::uint8_t> bytes, std::size_t ↩
        ↪ offset) {
443     std::uint32_t value = 0;
444     for (std::size_t i = 0; i < LCQI_DTYPE_BYTES_F32; ++i) {
445         value |= static_cast<std::uint32_t>(bytes[offset + i]) << (↩
        ↪ LCQI_BYTE_BITS * i);
446     }
447     return value;
448 }
449 
450 float bits_to_float(std::uint32_t bits) {
451     float value = 0.0F;
452     std::memcpy(&value, &bits, sizeof(value));
453     return value;
```

```
454 }
455
456 float f16_to_f32(std::uint16_t bits) {
457     const std::uint32_t sign = (static_cast<std::uint32_t>(bits) & ↩
    ↪ LCQI_F16_SIGN_MASK)
458                                << LCQI_F16_TO_F32_SIGN_SHIFT;
459     const std::uint32_t exponent = static_cast<std::uint32_t>(bits) & ↩
    ↪ LCQI_F16_EXPONENT_MASK;
460     const std::uint32_t mantissa = static_cast<std::uint32_t>(bits) & ↩
    ↪ LCQI_F16_MANTISSA_MASK;
461
462     if (exponent == 0U) {
463         if (mantissa == 0U) {
464             return bits_to_float(sign);
465         }
466         float value = static_cast<float>(mantissa) /
467                       static_cast<float>(1U << LCQI_F16_MANTISSA_BITS);
468         value = std::ldexp(value, 1 - static_cast<int>(LCQI_F16_EXPONENT_BIAS))↩
    ↪ ;
469         return (sign == 0U) ? value : -value;
470     }
471
472     if (exponent == LCQI_F16_EXPONENT_MASK) {
473         const std::uint32_t f32_mantissa =
474             mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
475         return bits_to_float(sign | LCQI_F32_EXPONENT_MASK | f32_mantissa);
476     }
477
478     const std::uint32_t f32_exponent =
479         ((exponent >> LCQI_F16_MANTISSA_BITS) - LCQI_F16_EXPONENT_BIAS +
480          LCQI_F32_EXPONENT_BIAS)
481         << LCQI_F32_MANTISSA_BITS;
482     const std::uint32_t f32_mantissa =
483         mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
484     return bits_to_float(sign | f32_exponent | f32_mantissa);
485 }
486
487 float bf16_to_f32(std::uint16_t bits) {
488     return bits_to_float(static_cast<std::uint32_t>(bits) << 16U);
489 }
490
491 std::span<const std::uint8_t> tensor_payload_span(const SafeTensorFile& file,
492                                                   const SafeTensorEntry& entry)↩
    ↪  {
493     const std::uint64_t begin = file.manifest.data_start_offset + entry.↩
    ↪ data_begin;
494     const std::uint64_t end = file.manifest.data_start_offset + entry.data_end;
495     if (begin > end || end > file.bytes.size()) {
496         throw std::runtime_error("safetensors tensor byte range is invalid");
497     }
498     return std::span<const std::uint8_t>(
499         file.bytes.data() + static_cast<std::size_t>(begin),
```

```
        static_cast<std::size_t>(end - begin));
}

}  // namespace

std::uint64_t SafeTensorEntry::byte_size() const {
    return this->data_end - this->data_begin;
}

const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) const {
    const auto found = std::find_if(
        this->tensors.begin(),
        this->tensors.end(),
        [name](const SafeTensorEntry& entry) {
            return entry.name == name;
        });
    if (found == this->tensors.end()) {
        return nullptr;
    }
    return &(*found);
}

SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path) {
    const std::string bytes = read_file_bytes(path);
    return parse_manifest_from_bytes(
        std::span<const std::uint8_t>(
            reinterpret_cast<const std::uint8_t*>(bytes.data()),
            bytes.size()));
}

SafeTensorFile load_safetensors_file(const std::filesystem::path& path) {
    SafeTensorFile file;
    file.path = path;
    file.bytes = read_file_u8(path);
    file.manifest = parse_manifest_from_bytes(file.bytes);
    return file;
}

std::span<const std::uint8_t> read_safetensor_tensor_bytes(
    const SafeTensorFile& file,
    std::string_view name) {
    const SafeTensorEntry* entry = file.manifest.find_tensor(name);
    if (entry == nullptr) {
        throw std::runtime_error("missing safetensors tensor: " + std::string(name));
    }
    return tensor_payload_span(file, *entry);
}

std::vector<float> read_safetensor_f32_tensor(const SafeTensorFile& file,
```

```
549                                                 std::string_view name) {
550     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
551     if (entry == nullptr) {
552         throw std::runtime_error("missing safetensors tensor: " + std::string(↩
        ↪ name));
553     }
554     const std::span<const std::uint8_t> bytes = tensor_payload_span(file, *↩
        ↪ entry);
555     std::vector<float> values;
556     if (entry->dtype == "F32") {
557         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F32);
558         for (std::size_t index = 0; index < values.size(); ++index) {
559             values[index] = bits_to_float(
560                 read_le_u32(bytes, index * LCQI_DTYPE_BYTES_F32));
561         }
562         return values;
563     }
564     if (entry->dtype == "F16") {
565         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
566         for (std::size_t index = 0; index < values.size(); ++index) {
567             values[index] = f16_to_f32(
568                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
569         }
570         return values;
571     }
572     if (entry->dtype == "BF16") {
573         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
574         for (std::size_t index = 0; index < values.size(); ++index) {
575             values[index] = bf16_to_f32(
576                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
577         }
578         return values;
579     }
580     throw std::runtime_error("safetensors tensor is not float-compatible: " + ↩
        ↪ entry->name);
581 }
582
583 }  // namespace lcqi
```

GPT-2 Reference Path

`gpt2_reference.cpp` 包括 JSON 配置读取、byte-level BPE、safetensors F32/F16/BF16 张量读取、HuggingFace 权重名前缀解析、LayerNorm、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation 和 cached-KV generation。它仍然是正确性优先的 reference path，但现在已经开始承接前三册的优化方法：先建立 baseline，再改 hot path，再用测试和 A/B benchmark 决定改动是否能进入书。

一次可审查的 cached-KV 优化

优化不能从“感觉慢”开始。LCQI 先用 `makecpu3-gpt2-benchmark-compare` 证明 full-prefix 每生成一个 token 都重算前缀，而 cached-KV 已经把前缀复用下来；随后再看 cached path 内部。旧实现每处理一个 token、每经过一层 Transformer，都会重新分配 `normed`、`query`、`key`、

value、attention、projected、mlp_fc、mlp_out 和 attention scores。同时，生成路径只需要下一个 greedy token，却仍然每步构造完整 Gpt2ForwardResult.logits。这些问题不是算法语义错误，但会在真实 GPT-2 的 decode loop 里形成额外分配、重复 shape validation、重复边界转换和不必要的结果写回。

本轮优化对应三册里的三条原则。第一册要求从热循环和二进制可观察行为出发，所以先量 lm_head、MLP、QKV、attention projection 和 attention 本体各占多少，再决定改哪里。第二册要求减少重复分配和内存扰动，所以新增 Gpt2ForwardWorkspace，把 hidden、QKV、MLP、scores 和 logits 缓冲区的生命周期提升到整段 cached generation。第三册要求按 decoder runtime 的状态组织代码，所以新增 Gpt2CachedGreedyDecoder，让 KV cache、workspace、worker pool 和模型验证跟随一次生成会话，而不是散落在每个 token 的临时对象里。

代码上有四处关键边界。第一，旧的 run_gpt2_forward_cached 仍然保留完整 logits API，测试和源码阅读可以继续用它和 full-prefix logits 做逐项对照。第二，CLI 的 cached_kv 生成路径改用 Gpt2CachedGreedyDecoder::step，只返回 greedy token；需要完整 logits 的测试使用 step_with_logits。第三，KV cache 对外仍保留 checked key/value API，内部 attention 在一次地址和 workspace 校验后使用私有 unchecked span 扫描热路径，避免把“不检查”的能力暴露给普通调用者。第四，线程池只属于 cached decoder 会话；--threads0 自动选择 worker 数，--threads1 强制串行，--threadsN 固定线程数，避免把全局可变线程服务塞进 reference model。

正确性证据在 tests/lcqi_tests.cpp。tiny GPT-2 同一个 prompt 同时覆盖 full-prefix logits、旧 cached logits、优化 decoder logits、强制两 worker 的并行 decoder logits 和 logits-free greedy token。它要求 predicted token、logits size、逐项 logit 误差、KV cache filled token 数和 greedy generated ids 全部一致。这样优化后的代码不是“跑得快但不知道还对不对”，而是先通过 tiny golden，再进入真实 GPT-2 benchmark。

性能证据不只看一次运行。新增 tools/run_gpt2_optimization_ab.py 会从指定 baseline commit 建临时 worktree，baseline 和当前工作区都通过 books/cpu-volume-3-practice 的 Release CMake 入口构建，再用同一份 GPT-2 F32 safetensors 模型跑多轮 full-prefix 与 cached-KV。后来远端 caw 不能直接复用本机 SSH remote 别名时，本轮采用等价的源码包 A/B：本机用 gitarchive73f40c7 导出 baseline，用当前工作区导出 current，二者在 caw 上分别 Release 构建并运行同一模型。raw 报告保存在 books/cpu-volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt，汇总保存在 books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md。5 轮中位数显示，4 token cached-KV 生成阶段从 395.397ms 到 76.4141ms，16 token 从 989.060ms 到 185.479ms；生成吞吐分别到 52.3464token/s 和 86.2633token/s。

用户看到上一轮提升幅度后，正确反应不是继续猜，而是重新量热点。新增 --profile-hotspots 和 tools/run_gpt2_hotspot_profile.py 后，可以把 cached decode 的时间拆成 embedding、LayerNorm、QKV projection、KV append、attention、attention projection、MLP fc、GELU、MLP projection 和 lm head。远端 caw 使用 Intel Core Ultra 7 265KF、GCC 15.2.1、Release 构建、GPT-2 F32 safetensors，跑 Hello,mynameis 这个 prompt，优化前 3 轮中位数保存在 books/cpu-volume-3/reports/lcqi-gpt2-hotspot-profile-summary.md：生成 4 个 token 时，lm_head 约占 30.2423%，MLP 的 c_fc 和 c_proj 合计约 46.4228%，QKV projection 约 17.2127%，cached attention 本体只有 0.0558254%。这说明慢点不是 attention，而是大量互相独立的行点积。

这一轮优化据此把 lm_head greedy argmax 和 GPT-2 F32 linear 的输出行切给持久 worker pool。每一行仍然是同一个标量 dot product，所以浮点累加顺序在单行内部不变；不同线程只负责不同输出行。完整 logits 路径把 logits buffer 分片写入，logits-free greedy 路径每个分片先找本地 best token，再按分片顺序合并，用严格大于比较保留较小 token id 的 tie-breaking。这样并行改变调度，不改变数学定义。

还有一个细节说明为什么优化必须带着 profiler 走。第一次 auto 阈值只按“一百万次乘加”判断是否并行，结果 768\times768 的 attention output projection 仍走串行。中间 profile 立即显示 attention projection 从约 6% 反弹到约 25%，成为新瓶颈。最终阈值改成：输出行数至少 512，且输入列数至少 512 时也允许并行。最终 profile 保存在 books/cpu-volume-3/reports/lcqi-gpt2-threaded-hotspot-caw.txt：4 token 生成中位数 75.9807ms，16 token 184.

780ms；lm_head 约 20%，MLP 两个投影合计约 45%，QKV 约 19%，attention projection 约 12%，attention 本体仍低于 1%。

这个结论必须保守阅读。真实端到端 GPT-2 benchmark 受调度、频率、温度和共享机器状态影响明显，单次结果可能反向波动。因此报告里必须同时看 median/min/max、生成文本是否一致、baseline 和 current 是否同一构建入口。更大的差距仍然不在这轮 coarse row parallelism 里：LCQI 还没有 GGUF 量化导入、packed/quantized lm head、SIMD/SVE/NEON/AVX 高性能 matvec、分页 KV cache、NUMA/亲和和成熟调度。也就是说，本轮把 reference path 从“cached 会话复用工作区”推进到“cached 会话内多线程行点积”，但还没有把它变成 llama.cpp 级推理引擎。

Listing 10.11 src/gpt2_reference.cpp

```cpp
#include <lcqi/gpt2_reference.hpp>

#include <lcqi/safetensors.hpp>

#include <algorithm>
#include <array>
#include <chrono>
#include <condition_variable>
#include <cmath>
#include <cstddef>
#include <fstream>
#include <functional>
#include <limits>
#include <mutex>
#include <optional>
#include <sstream>
#include <stdexcept>
#include <string>
#include <string_view>
#include <thread>
#include <unordered_set>
#include <utility>

namespace lcqi {
namespace {

constexpr std::int32_t LCQI_GPT2_DEFAULT_BOS_EOS = 50256;
constexpr std::int32_t LCQI_GPT2_ATTENTION_PROJECTIONS = 3;
constexpr std::int32_t LCQI_GPT2_PAIR_TOKEN_COUNT = 2;
constexpr std::int32_t LCQI_GPT2_BYTE_COUNT = 256;
constexpr std::int32_t LCQI_GPT2_UNICODE_PRIVATE_BASE = 256;
constexpr std::int32_t LCQI_GPT2_ASCII_EXCLAMATION = 33;
constexpr std::int32_t LCQI_GPT2_ASCII_TILDE = 126;
constexpr std::int32_t LCQI_GPT2_LATIN_INVERTED_EXCLAMATION = 161;
constexpr std::int32_t LCQI_GPT2_LATIN_NOT_SIGN = 172;
constexpr std::int32_t LCQI_GPT2_LATIN_REGISTERED = 174;
constexpr std::int32_t LCQI_GPT2_LATIN_Y_DIAERESIS = 255;
constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_MASK = 0x3F;
constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_BITS = 0x80;
constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_HEAD = 0xC0;
constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_HEAD = 0xE0;
constexpr std::int32_t LCQI_GPT2_UTF8_FOUR_BYTE_HEAD = 0xF0;
```

```
constexpr std::int32_t LCQI_GPT2_UTF8_ONE_BYTE_LIMIT = 0x80;
constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_LIMIT = 0x800;
constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_LIMIT = 0x10000;
constexpr unsigned char LCQI_GPT2_ASCII_APOSTROPHE = 0x27U;
constexpr float LCQI_GPT2_GELU_SQRT_2_OVER_PI = 0.7978845608028654F;
constexpr float LCQI_GPT2_GELU_CUBIC_COEFF = 0.044715F;
constexpr float LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY =
    -std::numeric_limits<float>::infinity();
constexpr std::int32_t LCQI_GPT2_MAX_AUTO_WORKERS = 16;
constexpr std::int32_t LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS = 512;
constexpr std::int32_t LCQI_GPT2_PARALLEL_MIN_COLUMNS = 512;
constexpr std::int64_t LCQI_GPT2_PARALLEL_MIN_OPS = 1'000'000;
constexpr std::size_t LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER = 4;

using Gpt2ProfileClock = std::chrono::steady_clock;

}  // namespace

namespace detail {

class Gpt2ParallelWorkerPool {
public:
    explicit Gpt2ParallelWorkerPool(std::int32_t requested_workers)
        : worker_count_(Gpt2ParallelWorkerPool::normalize_worker_count(
    ↪ requested_workers)) {
        if (this->worker_count_ <= 1) {
            return;
        }
        const std::size_t background_count =
            static_cast<std::size_t>(this->worker_count_ - 1);
        this->threads_.reserve(background_count);
        for (std::size_t index = 0; index < background_count; ++index) {
            this->threads_.emplace_back([this]() { this->worker_loop(); });
        }
    }

    ~Gpt2ParallelWorkerPool() {
        {
            std::lock_guard<std::mutex> lock(this->mutex_);
            this->stopping_ = true;
            ++this->generation_;
        }
        this->condition_.notify_all();
        for (std::thread& thread : this->threads_) {
            if (thread.joinable()) {
                thread.join();
            }
        }
    }

    Gpt2ParallelWorkerPool(const Gpt2ParallelWorkerPool&) = delete;
    Gpt2ParallelWorkerPool& operator=(const Gpt2ParallelWorkerPool&) = delete;
```

```
94
95      [[nodiscard]] std::int32_t worker_count() const noexcept {
96          return this->worker_count_;
97      }
98
99      void parallel_for_rows(std::size_t begin,
100                             std::size_t end,
101                             std::size_t grain,
102                             const std::function<void(std::size_t, std::size_t)>&
        ↪  function) {
103         if (end <= begin) {
104             return;
105         }
106         const std::size_t row_count = end - begin;
107         if (this->worker_count_ <= 1 || row_count <= grain) {
108             function(begin, end);
109             return;
110         }
111
112         const std::size_t task_count =
113             std::min<std::size_t>(static_cast<std::size_t>(this->worker_count_)
        ↪ ,
114                                   (row_count + grain - 1) / grain);
115         if (task_count <= 1) {
116             function(begin, end);
117             return;
118         }
119
120         {
121             std::lock_guard<std::mutex> lock(this->mutex_);
122             this->task_begin_ = begin;
123             this->task_end_ = end;
124             this->task_count_ = task_count;
125             this->next_worker_index_ = 0;
126             this->active_workers_ = task_count - 1;
127             this->task_function_ = &function;
128             ++this->generation_;
129         }
130         this->condition_.notify_all();
131
132         const std::size_t main_worker = task_count - 1;
133         const auto [main_begin, main_end] = this->chunk_bounds(begin, end,
        ↪ task_count, main_worker);
134         function(main_begin, main_end);
135
136         std::unique_lock<std::mutex> lock(this->mutex_);
137         this->finished_.wait(lock, [this]() { return this->active_workers_ ==
        ↪ 0; });
138         this->task_function_ = nullptr;
139     }
140
141 private:
```

```
    std::int32_t worker_count_ = 1;
    std::vector<std::thread> threads_;
    std::mutex mutex_;
    std::condition_variable condition_;
    std::condition_variable finished_;
    bool stopping_ = false;
    std::uint64_t generation_ = 0;
    std::size_t task_begin_ = 0;
    std::size_t task_end_ = 0;
    std::size_t task_count_ = 0;
    std::size_t next_worker_index_ = 0;
    std::size_t active_workers_ = 0;
    const std::function<void(std::size_t, std::size_t)>* task_function_ = nullptr;

    [[nodiscard]] static std::int32_t normalize_worker_count(std::int32_t requested_workers) {
        if (requested_workers > 0) {
            return requested_workers;
        }
        const unsigned int hardware_workers = std::thread::hardware_concurrency();
        if (hardware_workers == 0) {
            return 1;
        }
        const std::int32_t capped =
            std::min<std::int32_t>(static_cast<std::int32_t>(hardware_workers),
                                   LCQI_GPT2_MAX_AUTO_WORKERS);
        return std::max(capped, 1);
    }

    [[nodiscard]] std::pair<std::size_t, std::size_t> chunk_bounds(
        std::size_t begin,
        std::size_t end,
        std::size_t task_count,
        std::size_t worker_index) const {
        const std::size_t row_count = end - begin;
        const std::size_t chunk_begin = begin + row_count * worker_index / task_count;
        const std::size_t chunk_end = begin + row_count * (worker_index + 1) / task_count;
        return {chunk_begin, chunk_end};
    }

    void worker_loop() {
        std::uint64_t observed_generation = 0;
        while (true) {
            std::size_t worker_index = 0;
            std::size_t begin = 0;
            std::size_t end = 0;
            const std::function<void(std::size_t, std::size_t)>* function = nullptr;
```

```
            {
                std::unique_lock<std::mutex> lock(this->mutex_);
                this->condition_.wait(lock, [this, observed_generation]() {
                    return this->stopping_ || this->generation_ != ↩
    ↪ observed_generation;
                });
                if (this->stopping_) {
                    return;
                }
                observed_generation = this->generation_;
                if (this->task_function_ == nullptr ||
                    this->next_worker_index_ >= this->task_count_ - 1) {
                    continue;
                }
                worker_index = this->next_worker_index_;
                ++this->next_worker_index_;
                const auto bounds =
                    this->chunk_bounds(this->task_begin_,
                                       this->task_end_,
                                       this->task_count_,
                                       worker_index);
                begin = bounds.first;
                end = bounds.second;
                function = this->task_function_;
            }

            (*function)(begin, end);

            {
                std::lock_guard<std::mutex> lock(this->mutex_);
                --this->active_workers_;
                if (this->active_workers_ == 0) {
                    this->finished_.notify_one();
                }
            }
        }
    }
};

}  // namespace detail

namespace {

class JsonCursor {
public:
    explicit JsonCursor(std::string_view text) : text_(text) {}

    void expect(char expected) {
        this->skip_ws();
        if (this->eof() || this->text_[this->position_] != expected) {
            throw std::runtime_error("invalid JSON syntax");
        }
```

```
        ++this->position_;
    }

    [[nodiscard]] bool consume(char expected) {
        this->skip_ws();
        if (!this->eof() && this->text_[this->position_] == expected) {
            ++this->position_;
            return true;
        }
        return false;
    }

    [[nodiscard]] std::string parse_string() {
        this->skip_ws();
        this->expect('"');
        std::string result;
        while (!this->eof()) {
            const char ch = this->text_[this->position_++];
            if (ch == '"') {
                return result;
            }
            if (ch == '\\') {
                result += this->parse_escape();
            } else {
                result.push_back(ch);
            }
        }
        throw std::runtime_error("unterminated JSON string");
    }

    [[nodiscard]] std::int64_t parse_i64() {
        this->skip_ws();
        bool negative = false;
        if (this->consume('-')) {
            negative = true;
        }
        if (this->eof() || this->text_[this->position_] < '0' ||
            this->text_[this->position_] > '9') {
            throw std::runtime_error("expected JSON integer");
        }
        std::int64_t value = 0;
        while (!this->eof() && this->text_[this->position_] >= '0' &&
               this->text_[this->position_] <= '9') {
            const std::int64_t digit = this->text_[this->position_] - '0';
            if (value >
                (std::numeric_limits<std::int64_t>::max() - digit) / 10) {
                throw std::runtime_error("JSON integer overflow");
            }
            value = value * 10 + digit;
            ++this->position_;
        }
        return negative ? -value : value;
```

```
    }

    [[nodiscard]] double parse_number() {
        this->skip_ws();
        const std::size_t begin = this->position_;
        if (!this->eof() && this->text_[this->position_] == '-') {
            ++this->position_;
        }
        while (!this->eof() && this->text_[this->position_] >= '0' &&
               this->text_[this->position_] <= '9') {
            ++this->position_;
        }
        if (!this->eof() && this->text_[this->position_] == '.') {
            ++this->position_;
            while (!this->eof() && this->text_[this->position_] >= '0' &&
                   this->text_[this->position_] <= '9') {
                ++this->position_;
            }
        }
        if (!this->eof() &&
            (this->text_[this->position_] == 'e' || this->text_[this->position_
    ↪ ] == 'E')) {
            ++this->position_;
            if (!this->eof() &&
                (this->text_[this->position_] == '+' || this->text_[this->
    ↪ position_] == '-')) {
                ++this->position_;
            }
            while (!this->eof() && this->text_[this->position_] >= '0' &&
                   this->text_[this->position_] <= '9') {
                ++this->position_;
            }
        }
        if (begin == this->position_) {
            throw std::runtime_error("expected JSON number");
        }
        return std::stod(std::string(this->text_.substr(begin, this->position_
    ↪ - begin)));
    }

    void skip_value() {
        this->skip_ws();
        if (this->eof()) {
            throw std::runtime_error("unexpected end of JSON");
        }
        const char ch = this->text_[this->position_];
        if (ch == '"') {
            static_cast<void>(this->parse_string());
            return;
        }
        if (ch == '{') {
            this->expect('{');
```

```
            if (this->consume('}')) {
                return;
            }
            while (true) {
                static_cast<void>(this->parse_string());
                this->expect(':');
                this->skip_value();
                if (this->consume('}')) {
                    return;
                }
                this->expect(',');
            }
        }
        if (ch == '[') {
            this->expect('[');
            if (this->consume(']')) {
                return;
            }
            while (true) {
                this->skip_value();
                if (this->consume(']')) {
                    return;
                }
                this->expect(',');
            }
        }
        if (ch == 't') {
            this->expect_literal("true");
            return;
        }
        if (ch == 'f') {
            this->expect_literal("false");
            return;
        }
        if (ch == 'n') {
            this->expect_literal("null");
            return;
        }
        static_cast<void>(this->parse_number());
    }

    [[nodiscard]] bool eof_after_ws() {
        this->skip_ws();
        return this->eof();
    }

private:
    std::string_view text_;
    std::size_t position_ = 0;

    [[nodiscard]] bool eof() const {
        return this->position_ >= this->text_.size();
```

```
392     }
393
394     void skip_ws() {
395         while (!this->eof() &&
396                (this->text_[this->position_] == ' ' ||
397                 this->text_[this->position_] == '\n' ||
398                 this->text_[this->position_] == '\r' ||
399                 this->text_[this->position_] == '\t')) {
400             ++this->position_;
401         }
402     }
403
404     void expect_literal(std::string_view literal) {
405         for (const char expected : literal) {
406             if (this->eof() || this->text_[this->position_] != expected) {
407                 throw std::runtime_error("invalid JSON literal");
408             }
409             ++this->position_;
410         }
411     }
412
413     [[nodiscard]] std::string parse_escape() {
414         if (this->eof()) {
415             throw std::runtime_error("unterminated JSON escape");
416         }
417         const char escaped = this->text_[this->position_++];
418         switch (escaped) {
419             case '"':
420             case '\\':
421             case '/':
422                 return std::string(1, escaped);
423             case 'b':
424                 return "\b";
425             case 'f':
426                 return "\f";
427             case 'n':
428                 return "\n";
429             case 'r':
430                 return "\r";
431             case 't':
432                 return "\t";
433             case 'u':
434                 return this->parse_unicode_escape();
435             default:
436                 throw std::runtime_error("unsupported JSON escape");
437         }
438     }
439
440     [[nodiscard]] std::string parse_unicode_escape() {
441         std::uint32_t value = 0;
442         for (std::int32_t i = 0; i < 4; ++i) {
443             if (this->eof()) {
```

```
                throw std::runtime_error("unterminated JSON unicode escape");
            }
            const char ch = this->text_[this->position_++];
            value <<= 4U;
            if (ch >= '0' && ch <= '9') {
                value += static_cast<std::uint32_t>(ch - '0');
            } else if (ch >= 'a' && ch <= 'f') {
                value += static_cast<std::uint32_t>(10 + ch - 'a');
            } else if (ch >= 'A' && ch <= 'F') {
                value += static_cast<std::uint32_t>(10 + ch - 'A');
            } else {
                throw std::runtime_error("invalid JSON unicode escape");
            }
        }
        std::string encoded;
        if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
            encoded.push_back(static_cast<char>(value));
        } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        } else {
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
            encoded.push_back(static_cast<char>(
                LCQI_GPT2_UTF8_CONTINUATION_BITS |
                (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        }
        return encoded;
    }
};

std::string read_text_file(const std::filesystem::path& path) {
    std::ifstream input(path);
    if (!input) {
        throw std::runtime_error("cannot open text file: " + path.string());
    }
    std::ostringstream buffer;
    buffer << input.rdbuf();
    return buffer.str();
}

std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
```

```cpp
}

std::string encode_codepoint(std::uint32_t value) {
    std::string encoded;
    if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(value));
    } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    } else if (value < LCQI_GPT2_UTF8_THREE_BYTE_LIMIT) {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    } else {
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_FOUR_BYTE_HEAD | (value >> 18U)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 12U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
        encoded.push_back(static_cast<char>(
            LCQI_GPT2_UTF8_CONTINUATION_BITS |
            (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
    }
    return encoded;
}

void require_positive(std::int32_t value, const char* name) {
    if (value <= 0) {
        throw std::runtime_error(std::string(name) + " must be positive");
    }
}

void validate_config(const Gpt2Config& config) {
    require_positive(config.hidden_size, "hidden_size");
    require_positive(config.head_count, "head_count");
    require_positive(config.layer_count, "layer_count");
    require_positive(config.max_positions, "max_positions");
    require_positive(config.vocab_size, "vocab_size");
    require_positive(config.intermediate_size, "intermediate_size");
    if (config.hidden_size % config.head_count != 0) {
        throw std::runtime_error("hidden_size must be divisible by head_count")↩
```

```
    ↪ ;
548     }
549     if (config.layer_norm_epsilon <= 0.0F) {
550         throw std::runtime_error("layer_norm_epsilon must be positive");
551     }
552 }
553
554 std::int32_t head_dim(const Gpt2Config& config) {
555     return config.hidden_size / config.head_count;
556 }
557
558 std::size_t matrix_size(std::int32_t rows, std::int32_t columns) {
559     return checked_size(rows, "rows") * checked_size(columns, "columns");
560 }
561
562 Gpt2ProfileClock::time_point profile_start(const Gpt2HotspotProfile* profile) {
563     if (profile == nullptr) {
564         return Gpt2ProfileClock::time_point{};
565     }
566     return Gpt2ProfileClock::now();
567 }
568
569 void add_profile_ms(Gpt2HotspotProfile* profile,
570                     double Gpt2HotspotProfile::*field,
571                     Gpt2ProfileClock::time_point start) {
572     if (profile == nullptr) {
573         return;
574     }
575     (*profile).*field +=
576         std::chrono::duration<double, std::milli>(Gpt2ProfileClock::now() - ↩
    ↪ start).count();
577 }
578
579 void validate_vector_size(std::size_t actual, std::int32_t expected, const char↩
    ↪ * name) {
580     if (actual != checked_size(expected, name)) {
581         throw std::runtime_error(std::string(name) + " size mismatch");
582     }
583 }
584
585 void validate_linear(const Gpt2LinearF32& linear, const char* name) {
586     require_positive(linear.input_size, "linear input_size");
587     require_positive(linear.output_size, "linear output_size");
588     if (linear.weights.size() != matrix_size(linear.output_size, linear.↩
    ↪ input_size)) {
589         throw std::runtime_error(std::string(name) + " weight size mismatch");
590     }
591     if (!linear.bias.empty() &&
592         linear.bias.size() != checked_size(linear.output_size, "linear ↩
    ↪ output_size")) {
593         throw std::runtime_error(std::string(name) + " bias size mismatch");
594     }
```

```
}

void validate_model(const Gpt2ReferenceModel& model) {
    validate_config(model.config);
    if (model.layers.size() != checked_size(model.config.layer_count, "layer_count")) {
        throw std::runtime_error("GPT-2 layer count mismatch");
    }
    if (model.token_embedding.size() !=
        matrix_size(model.config.vocab_size, model.config.hidden_size)) {
        throw std::runtime_error("GPT-2 token embedding size mismatch");
    }
    if (model.position_embedding.size() !=
        matrix_size(model.config.max_positions, model.config.hidden_size)) {
        throw std::runtime_error("GPT-2 position embedding size mismatch");
    }
    validate_vector_size(model.final_ln_weight.size(), model.config.hidden_size, "ln_f weight");
    validate_vector_size(model.final_ln_bias.size(), model.config.hidden_size, "ln_f bias");
    if (model.tie_lm_head_to_embedding) {
        if (!model.lm_head_weight.empty()) {
            throw std::runtime_error("tied GPT-2 lm_head should not carry separate weights");
        }
    } else if (model.lm_head_weight.size() !=
               matrix_size(model.config.vocab_size, model.config.hidden_size)) {
        throw std::runtime_error("GPT-2 lm_head size mismatch");
    }
    for (const Gpt2LayerWeightsF32& layer : model.layers) {
        validate_vector_size(layer.ln_1_weight.size(), model.config.hidden_size, "ln_1 weight");
        validate_vector_size(layer.ln_1_bias.size(), model.config.hidden_size, "ln_1 bias");
        validate_linear(layer.c_attn, "c_attn");
        validate_linear(layer.c_proj, "c_proj");
        validate_vector_size(layer.ln_2_weight.size(), model.config.hidden_size, "ln_2 weight");
        validate_vector_size(layer.ln_2_bias.size(), model.config.hidden_size, "ln_2 bias");
        validate_linear(layer.c_fc, "c_fc");
        validate_linear(layer.mlp_c_proj, "mlp_c_proj");
    }
}

std::span<const float> row_span(std::span<const float> values,
                                std::int32_t row,
                                std::int32_t row_width) {
    return values.subspan(checked_size(row, "row") * checked_size(row_width, "row_width"),
                          checked_size(row_width, "row_width"));
```

```
637 }
638
639 void add_inplace(std::span<float> target, std::span<const float> source) {
640     if (target.size() != source.size()) {
641         throw std::runtime_error("GPT-2 residual add size mismatch");
642     }
643     for (std::size_t index = 0; index < target.size(); ++index) {
644         target[index] += source[index];
645     }
646 }
647
648 void layer_norm(std::span<const float> input,
649                 std::span<const float> weight,
650                 std::span<const float> bias,
651                 float epsilon,
652                 std::span<float> output) {
653     if (input.empty() || input.size() != weight.size() || input.size() != bias.↩
    ↪ size() ||
654         input.size() != output.size()) {
655         throw std::runtime_error("GPT-2 LayerNorm size mismatch");
656     }
657     float mean = 0.0F;
658     for (const float value : input) {
659         mean += value;
660     }
661     mean /= static_cast<float>(input.size());
662
663     float variance = 0.0F;
664     for (const float value : input) {
665         const float centered = value - mean;
666         variance += centered * centered;
667     }
668     variance /= static_cast<float>(input.size());
669     const float scale = 1.0F / std::sqrt(variance + epsilon);
670
671     for (std::size_t index = 0; index < input.size(); ++index) {
672         output[index] = (input[index] - mean) * scale * weight[index] + bias[↩
    ↪ index];
673     }
674 }
675
676 void linear_f32(const Gpt2LinearF32& linear,
677                 std::span<const float> input,
678                 std::span<float> output) {
679     validate_linear(linear, "GPT-2 linear");
680     if (input.size() != checked_size(linear.input_size, "linear input") ||
681         output.size() != checked_size(linear.output_size, "linear output")) {
682         throw std::runtime_error("GPT-2 linear input/output size mismatch");
683     }
684     const std::size_t input_size = checked_size(linear.input_size, "linear ↩
    ↪ input");
685     const std::size_t output_size = checked_size(linear.output_size, "linear ↩
```

```
    ↪ output");
    for (std::size_t out = 0; out < output_size; ++out) {
        float sum = linear.bias.empty() ? 0.0F : linear.bias[out];
        const std::size_t row_base = out * input_size;
        for (std::size_t in = 0; in < input_size; ++in) {
            sum += linear.weights[row_base + in] * input[in];
        }
        output[out] = sum;
    }
}

void linear_f32_unchecked(const Gpt2LinearF32& linear,
                          std::span<const float> input,
                          std::span<float> output) {
    const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
    const std::size_t output_size = static_cast<std::size_t>(linear.output_size↩
    ↪ );
    for (std::size_t out = 0; out < output_size; ++out) {
        float sum = linear.bias.empty() ? 0.0F : linear.bias[out];
        const std::size_t row_base = out * input_size;
        for (std::size_t in = 0; in < input_size; ++in) {
            sum += linear.weights[row_base + in] * input[in];
        }
        output[out] = sum;
    }
}

[[nodiscard]] bool should_parallelize_rows(const Gpt2ExecutionOptions& options,
                                           std::size_t rows,
                                           std::size_t columns,
                                           const detail::Gpt2ParallelWorkerPool↩
    ↪ * worker_pool) {
    if (worker_pool == nullptr || worker_pool->worker_count() <= 1) {
        return false;
    }
    const std::int32_t min_rows =
        options.parallel_min_rows > 0 ? options.parallel_min_rows
                                      : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
    if (rows < checked_size(min_rows, "parallel_min_rows")) {
        return false;
    }
    if (options.worker_count > 1) {
        return true;
    }
    return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS, "↩
    ↪ parallel_min_columns") ||
           rows * columns >= static_cast<std::size_t>(↩
    ↪ LCQI_GPT2_PARALLEL_MIN_OPS);
}

[[nodiscard]] std::size_t parallel_row_grain(std::size_t rows,
                                             const detail::↩
```

```
    Gpt2ParallelWorkerPool& worker_pool) {
    const std::size_t workers = checked_size(worker_pool.worker_count(), "worker_count");
    const std::size_t target_tasks = workers * LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER;
    return std::max<std::size_t>(1, (rows + target_tasks - 1) / target_tasks);
}

[[nodiscard]] bool model_has_parallel_work(const Gpt2ReferenceModel& model,
                                           const Gpt2ExecutionOptions& options) {
    const std::size_t hidden_size = checked_size(model.config.hidden_size, "hidden_size");
    const std::size_t vocab_size = checked_size(model.config.vocab_size, "vocab_size");
    const std::size_t intermediate_size =
        checked_size(model.config.intermediate_size, "intermediate_size");
    const std::size_t qkv_rows =
        checked_size(model.config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS,
                     "qkv output rows");
    const std::int32_t min_rows =
        options.parallel_min_rows > 0 ? options.parallel_min_rows
                                      : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
    const auto large_enough = [min_rows](std::size_t rows, std::size_t columns) {
        if (rows < checked_size(min_rows, "parallel_min_rows")) {
            return false;
        }
        return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS,
                                       "parallel_min_columns") ||
               rows * columns >= static_cast<std::size_t>(LCQI_GPT2_PARALLEL_MIN_OPS);
    };
    if (options.worker_count > 1) {
        return vocab_size >= checked_size(min_rows, "parallel_min_rows") ||
               qkv_rows >= checked_size(min_rows, "parallel_min_rows") ||
               intermediate_size >= checked_size(min_rows, "parallel_min_rows") ||
               hidden_size >= checked_size(min_rows, "parallel_min_rows");
    }
    if (options.worker_count == 1) {
        return false;
    }
    return large_enough(vocab_size, hidden_size) ||
           large_enough(qkv_rows, hidden_size) ||
           large_enough(hidden_size, hidden_size) ||
           large_enough(intermediate_size, hidden_size) ||
           large_enough(hidden_size, intermediate_size);
}

[[nodiscard]] std::unique_ptr<detail::Gpt2ParallelWorkerPool> make_worker_pool(
```

```
    const Gpt2ReferenceModel& model,
    const Gpt2ExecutionOptions& options) {
    if (!model_has_parallel_work(model, options)) {
        return nullptr;
    }
    return std::make_unique<detail::Gpt2ParallelWorkerPool>(options.worker_count);
}

void linear_f32_unchecked_parallel(const Gpt2LinearF32& linear,
                                   std::span<const float> input,
                                   std::span<float> output,
                                   const Gpt2ExecutionOptions& options,
                                   detail::Gpt2ParallelWorkerPool* worker_pool) {
    const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
    const std::size_t output_size = static_cast<std::size_t>(linear.output_size);
    if (!should_parallelize_rows(options, output_size, input_size, worker_pool)) {
        linear_f32_unchecked(linear, input, output);
        return;
    }

    const bool has_bias = !linear.bias.empty();
    const float* weights = linear.weights.data();
    const float* bias = linear.bias.data();
    const float* input_data = input.data();
    float* output_data = output.data();
    const std::function<void(std::size_t, std::size_t)> rows_fn =
        [input_size, has_bias, weights, bias, input_data, output_data](
            std::size_t begin,
            std::size_t end) {
            for (std::size_t out = begin; out < end; ++out) {
                float sum = has_bias ? bias[out] : 0.0F;
                const float* row = weights + out * input_size;
                for (std::size_t in = 0; in < input_size; ++in) {
                    sum += row[in] * input_data[in];
                }
                output_data[out] = sum;
            }
        };
    worker_pool->parallel_for_rows(0, output_size, parallel_row_grain(output_size, *worker_pool), rows_fn);
}

float gelu_new(float value) {
    const float cubic = value * value * value;
    return 0.5F * value *
           (1.0F + std::tanh(LCQI_GPT2_GELU_SQRT_2_OVER_PI *
                             (value + LCQI_GPT2_GELU_CUBIC_COEFF * cubic)));
}
```

```
822
823 std::int32_t argmax(std::span<const float> values) {
824     if (values.empty()) {
825         throw std::runtime_error("cannot take argmax of empty logits");
826     }
827     std::int32_t best_index = 0;
828     float best_value = values.front();
829     for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size↩
      ↪ ()); ++index) {
830         const float value = values[checked_size(index, "argmax index")];
831         if (value > best_value) {
832             best_value = value;
833             best_index = index;
834         }
835     }
836     return best_index;
837 }
838
839 void project_qkv(const Gpt2Config& config,
840                  const Gpt2LayerWeightsF32& layer,
841                  std::span<const float> hidden,
842                  std::span<float> q,
843                  std::span<float> k,
844                  std::span<float> v) {
845     std::vector<float> packed(matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS,
846                                           config.hidden_size),
847                               0.0F);
848     linear_f32(layer.c_attn, hidden, packed);
849     const std::size_t width = checked_size(config.hidden_size, "hidden_size");
850     std::copy_n(packed.begin(), config.hidden_size, q.begin());
851     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
852                 config.hidden_size,
853                 k.begin());
854     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
855                 config.hidden_size,
856                 v.begin());
857 }
858
859 void project_qkv_reuse_workspace(const Gpt2Config& config,
860                                  const Gpt2LayerWeightsF32& layer,
861                                  std::span<const float> hidden,
862                                  std::span<float> packed,
863                                  std::span<float> q,
864                                  std::span<float> k,
865                                  std::span<float> v,
866                                  const Gpt2ExecutionOptions& options,
867                                  detail::Gpt2ParallelWorkerPool* worker_pool) {
868     linear_f32_unchecked_parallel(layer.c_attn, hidden, packed, options, ↩
      ↪ worker_pool);
869     const std::size_t width = checked_size(config.hidden_size, "hidden_size");
870     std::copy_n(packed.begin(), config.hidden_size, q.begin());
871     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
```

```
872                 config.hidden_size,
873                 k.begin());
874     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
875                 config.hidden_size,
876                 v.begin());
877 }
878
879 void attention_full_sequence(const Gpt2Config& config,
880                              std::span<const float> q_by_pos,
881                              std::span<const float> k_by_pos,
882                              std::span<const float> v_by_pos,
883                              std::int32_t sequence_length,
884                              std::span<float> output_by_pos) {
885     const std::int32_t local_head_dim = head_dim(config);
886     const float score_scale = 1.0F / std::sqrt(static_cast<float>(↩
    ↪ local_head_dim));
887     std::fill(output_by_pos.begin(), output_by_pos.end(), 0.0F);
888     std::vector<float> scores(checked_size(sequence_length, "sequence_length"),
889                               LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);
890
891     for (std::int32_t position = 0; position < sequence_length; ++position) {
892         for (std::int32_t head = 0; head < config.head_count; ++head) {
893             float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
894             for (std::int32_t past = 0; past <= position; ++past) {
895                 float dot = 0.0F;
896                 for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
897                     const std::size_t q_index =
898                         matrix_size(position, config.hidden_size) +
899                         matrix_size(head, local_head_dim) + checked_size(dim, "↩
    ↪ head dim");
900                     const std::size_t k_index =
901                         matrix_size(past, config.hidden_size) +
902                         matrix_size(head, local_head_dim) + checked_size(dim, "↩
    ↪ head dim");
903                     dot += q_by_pos[q_index] * k_by_pos[k_index];
904                 }
905                 const float score = dot * score_scale;
906                 scores[checked_size(past, "past")] = score;
907                 max_score = std::max(max_score, score);
908             }
909
910             float denominator = 0.0F;
911             for (std::int32_t past = 0; past <= position; ++past) {
912                 const std::size_t index = checked_size(past, "past");
913                 scores[index] = std::exp(scores[index] - max_score);
914                 denominator += scores[index];
915             }
916             if (denominator <= 0.0F) {
917                 throw std::runtime_error("GPT-2 attention denominator is ↩
    ↪ invalid");
918             }
919             for (std::int32_t past = 0; past <= position; ++past) {
```

```
                const float probability = scores[checked_size(past, "past")] / denominator;
                for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
                    const std::size_t output_index =
                        matrix_size(position, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    const std::size_t value_index =
                        matrix_size(past, config.hidden_size) +
                        matrix_size(head, local_head_dim) + checked_size(dim, "head dim");
                    output_by_pos[output_index] += probability * v_by_pos[value_index];
                }
            }
        }
    }
}

void forward_gpt2_layer(const Gpt2Config& config,
                        const Gpt2LayerWeightsF32& layer,
                        std::int32_t sequence_length,
                        std::vector<float>& hidden_by_pos) {
    std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> q_by_pos(matrix_size(sequence_length, config.hidden_size), 0.0F);
    std::vector<float> k_by_pos(q_by_pos.size(), 0.0F);
    std::vector<float> v_by_pos(q_by_pos.size(), 0.0F);
    std::vector<float> attention_by_pos(q_by_pos.size(), 0.0F);
    std::vector<float> projected(checked_size(config.hidden_size, "hidden_size"), 0.0F);
    std::vector<float> mlp_fc(checked_size(config.intermediate_size, "intermediate_size"), 0.0F);
    std::vector<float> mlp_out(checked_size(config.hidden_size, "hidden_size"), 0.0F);

    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        layer_norm(hidden,
                   layer.ln_1_weight,
                   layer.ln_1_bias,
                   config.layer_norm_epsilon,
                   normed);
        std::span<float> q = std::span<float>(
            q_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
        std::span<float> k = std::span<float>(
            k_by_pos.data() + matrix_size(position, config.hidden_size),
            checked_size(config.hidden_size, "hidden_size"));
```

```
963         std::span<float> v = std::span<float>(
964             v_by_pos.data() + matrix_size(position, config.hidden_size),
965             checked_size(config.hidden_size, "hidden_size"));
966         project_qkv(config, layer, normed, q, k, v);
967     }
968 
969     attention_full_sequence(config, q_by_pos, k_by_pos, v_by_pos, ↩
    ↪ sequence_length, attention_by_pos);
970 
971     for (std::int32_t position = 0; position < sequence_length; ++position) {
972         const std::span<float> hidden = std::span<float>(
973             hidden_by_pos.data() + matrix_size(position, config.hidden_size),
974             checked_size(config.hidden_size, "hidden_size"));
975         const std::span<const float> attention = std::span<const float>(
976             attention_by_pos.data() + matrix_size(position, config.hidden_size)↩
    ↪ ,
977             checked_size(config.hidden_size, "hidden_size"));
978         linear_f32(layer.c_proj, attention, projected);
979         add_inplace(hidden, projected);
980 
981         layer_norm(hidden,
982                    layer.ln_2_weight,
983                    layer.ln_2_bias,
984                    config.layer_norm_epsilon,
985                    normed);
986         linear_f32(layer.c_fc, normed, mlp_fc);
987         for (float& value : mlp_fc) {
988             value = gelu_new(value);
989         }
990         linear_f32(layer.mlp_c_proj, mlp_fc, mlp_out);
991         add_inplace(hidden, mlp_out);
992     }
993 }
994 
995 void forward_gpt2_layer_cached(const Gpt2Config& config,
996                                const Gpt2LayerWeightsF32& layer,
997                                std::int32_t layer_id,
998                                std::int32_t model_position,
999                                Gpt2KvCache& cache,
1000                               Gpt2ForwardWorkspace& workspace,
1001                               std::span<float> hidden,
1002                               Gpt2HotspotProfile* hotspot_profile,
1003                               const Gpt2ExecutionOptions& options,
1004                               detail::Gpt2ParallelWorkerPool* worker_pool) {
1005     if (hotspot_profile != nullptr) {
1006         ++hotspot_profile->layer_steps;
1007     }
1008     std::span<float> normed = workspace.normed();
1009     std::span<float> query = workspace.query();
1010     std::span<float> key = workspace.key();
1011     std::span<float> value = workspace.value();
1012     std::span<float> attention = workspace.attention();
```

```
    std::span<float> projected = workspace.projected();
    std::span<float> qkv_packed = workspace.qkv_packed();
    std::span<float> mlp_fc = workspace.mlp_fc();
    std::span<float> mlp_out = workspace.mlp_out();
    std::span<float> scores = workspace.scores_prefix(model_position + 1);

    Gpt2ProfileClock::time_point start =
        profile_start(hotspot_profile);
    layer_norm(hidden,
               layer.ln_1_weight,
               layer.ln_1_bias,
               config.layer_norm_epsilon,
               normed);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);

    start = profile_start(hotspot_profile);
    project_qkv_reuse_workspace(config,
                                layer,
                                normed,
                                qkv_packed,
                                query,
                                key,
                                value,
                                options,
                                worker_pool);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::qkv_projection_ms,
    ↪ start);

    start = profile_start(hotspot_profile);
    cache.append(layer_id, model_position, key, value);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::kv_append_ms, start);

    start = profile_start(hotspot_profile);
    detail::attend_cached_position(cache,
                                   layer_id,
                                   model_position,
                                   query,
                                   scores,
                                   attention);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::attention_ms, start);

    start = profile_start(hotspot_profile);
    linear_f32_unchecked_parallel(layer.c_proj, attention, projected, options,
    ↪ worker_pool);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::
    ↪ attention_projection_ms, start);

    start = profile_start(hotspot_profile);
    add_inplace(hidden, projected);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start
    ↪ );

```

```
    start = profile_start(hotspot_profile);
    layer_norm(hidden,
               layer.ln_2_weight,
               layer.ln_2_bias,
               config.layer_norm_epsilon,
               normed);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);

    start = profile_start(hotspot_profile);
    linear_f32_unchecked_parallel(layer.c_fc, normed, mlp_fc, options, worker_pool);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_fc_ms, start);

    start = profile_start(hotspot_profile);
    for (float& value_ref : mlp_fc) {
        value_ref = gelu_new(value_ref);
    }
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::gelu_ms, start);

    start = profile_start(hotspot_profile);
    linear_f32_unchecked_parallel(layer.mlp_c_proj, mlp_fc, mlp_out, options, worker_pool);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_projection_ms, start);

    start = profile_start(hotspot_profile);
    add_inplace(hidden, mlp_out);
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start);
}

void compute_logits(const Gpt2ReferenceModel& model,
                    std::span<const float> hidden,
                    std::span<float> logits,
                    const Gpt2ExecutionOptions& options,
                    detail::Gpt2ParallelWorkerPool* worker_pool) {
    if (logits.size() != checked_size(model.config.vocab_size, "vocab_size")) {
        throw std::runtime_error("GPT-2 logits size mismatch");
    }
    const std::span<const float> weight =
        model.tie_lm_head_to_embedding ? std::span<const float>(model.token_embedding)
                                       : std::span<const float>(model.lm_head_weight);
    const std::size_t hidden_size = checked_size(model.config.hidden_size, "hidden_size");
    const std::size_t vocab_size = checked_size(model.config.vocab_size, "vocab_size");
    const float* weight_data = weight.data();
    const float* hidden_data = hidden.data();
    const std::function<void(std::size_t, std::size_t)> rows_fn =
        [hidden_size, weight_data, hidden_data, logits_data = logits.data()](
```

```
            std::size_t begin,
            std::size_t end) {
            for (std::size_t token = begin; token < end; ++token) {
                float sum = 0.0F;
                const float* row = weight_data + token * hidden_size;
                for (std::size_t dim = 0; dim < hidden_size; ++dim) {
                    sum += row[dim] * hidden_data[dim];
                }
                logits_data[token] = sum;
            }
        };
    if (should_parallelize_rows(options, vocab_size, hidden_size, worker_pool))↩
    ↪  {
        worker_pool->parallel_for_rows(0,
                                       vocab_size,
                                       parallel_row_grain(vocab_size, *↩
    ↪ worker_pool),
                                       rows_fn);
    } else {
        rows_fn(0, vocab_size);
    }
}

void compute_logits(const Gpt2ReferenceModel& model,
                    std::span<const float> hidden,
                    std::span<float> logits) {
    Gpt2ExecutionOptions options;
    options.worker_count = 1;
    compute_logits(model, hidden, logits, options, nullptr);
}

struct Gpt2TokenScore {
    std::int32_t token = 0;
    float value = -std::numeric_limits<float>::infinity();
};

[[nodiscard]] Gpt2TokenScore compute_predicted_token_range(const float* ↩
    ↪ weight_data,
                                                           const float* ↩
    ↪ hidden_data,
                                                           std::size_t ↩
    ↪ hidden_size,
                                                           std::size_t begin,
                                                           std::size_t end) {
    Gpt2TokenScore best;
    best.token = static_cast<std::int32_t>(begin);
    for (std::size_t token = begin; token < end; ++token) {
        float sum = 0.0F;
        const float* row = weight_data + token * hidden_size;
        for (std::size_t dim = 0; dim < hidden_size; ++dim) {
            sum += row[dim] * hidden_data[dim];
        }
```

```
        if (token == begin || sum > best.value) {
            best.value = sum;
            best.token = static_cast<std::int32_t>(token);
        }
    }
    return best;
}

std::int32_t compute_predicted_token(const Gpt2ReferenceModel& model,
                                     std::span<const float> hidden,
                                     const Gpt2ExecutionOptions& options,
                                     detail::Gpt2ParallelWorkerPool* worker_pool) {
    const std::span<const float> weight =
        model.tie_lm_head_to_embedding ? std::span<const float>(model.token_embedding)
                                       : std::span<const float>(model.lm_head_weight);
    const std::size_t hidden_size = checked_size(model.config.hidden_size, "hidden_size");
    const std::size_t vocab_size = checked_size(model.config.vocab_size, "vocab_size");
    const float* weight_data = weight.data();
    const float* hidden_data = hidden.data();
    if (!should_parallelize_rows(options, vocab_size, hidden_size, worker_pool) ) {
        return compute_predicted_token_range(weight_data, hidden_data, hidden_size, 0, vocab_size)
            .token;
    }

    const std::size_t worker_count = checked_size(worker_pool->worker_count(), "worker_count");
    const std::size_t chunk_count = std::min(worker_count, vocab_size);
    std::vector<Gpt2TokenScore> partials(chunk_count);
    const std::function<void(std::size_t, std::size_t)> rows_fn =
        [chunk_count, hidden_size, vocab_size, weight_data, hidden_data, &partials](
            std::size_t begin,
            std::size_t end) {
            for (std::size_t chunk = begin; chunk < end; ++chunk) {
                const std::size_t token_begin = vocab_size * chunk / chunk_count;
                const std::size_t token_end = vocab_size * (chunk + 1) / chunk_count;
                partials[chunk] = compute_predicted_token_range(
                    weight_data,
                    hidden_data,
                    hidden_size,
                    token_begin,
                    token_end);
            }
```

```
        };
    worker_pool->parallel_for_rows(0, chunk_count, 1, rows_fn);

    Gpt2TokenScore best = partials.front();
    for (std::size_t index = 1; index < partials.size(); ++index) {
        if (partials[index].value > best.value) {
            best = partials[index];
        }
    }
    return best.token;
}

Gpt2ForwardResult run_gpt2_cached_step_unchecked(const Gpt2ReferenceModel& model,
                                                 Gpt2KvCache& cache,
                                                 Gpt2ForwardWorkspace& workspace,
                                                 std::int32_t token_id,
                                                 bool need_logits,
                                                 Gpt2HotspotProfile* hotspot_profile,
                                                 const Gpt2ExecutionOptions& options,
                                                 detail::Gpt2ParallelWorkerPool* worker_pool) {
    const Gpt2ProfileClock::time_point step_start = profile_start(hotspot_profile);
    if (hotspot_profile != nullptr) {
        ++hotspot_profile->decoder_steps;
    }
    if (token_id < 0 || token_id >= model.config.vocab_size) {
        throw std::runtime_error("GPT-2 token id out of range");
    }
    const std::int32_t model_position = cache.filled_tokens();
    if (model_position >= model.config.max_positions) {
        throw std::runtime_error("GPT-2 cached forward would exceed max_positions");
    }

    std::span<float> hidden = workspace.hidden();
    const std::span<const float> token =
        row_span(model.token_embedding, token_id, model.config.hidden_size);
    const std::span<const float> position_embedding =
        row_span(model.position_embedding, model_position, model.config.hidden_size);
    Gpt2ProfileClock::time_point start = profile_start(hotspot_profile);
    for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
        hidden[checked_size(dim, "dim")] =
            token[checked_size(dim, "dim")] + position_embedding[checked_size(dim, "dim")];
    }
    add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::embedding_ms, start);
```

```
1236
1237     for (std::int32_t layer_id = 0;
1238          layer_id < static_cast<std::int32_t>(model.layers.size());
1239          ++layer_id) {
1240         forward_gpt2_layer_cached(model.config,
1241                                   model.layers[checked_size(layer_id, "layer_id
     ↪ ")],
1242                                   layer_id,
1243                                   model_position,
1244                                   cache,
1245                                   workspace,
1246                                   hidden,
1247                                   hotspot_profile,
1248                                   options,
1249                                   worker_pool);
1250     }
1251
1252     std::span<float> normed = workspace.normed();
1253     start = profile_start(hotspot_profile);
1254     layer_norm(hidden,
1255                model.final_ln_weight,
1256                model.final_ln_bias,
1257                model.config.layer_norm_epsilon,
1258                normed);
1259     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::final_norm_ms, start);
1260
1261     Gpt2ForwardResult result;
1262     if (need_logits) {
1263         std::span<float> logits = workspace.logits();
1264         start = profile_start(hotspot_profile);
1265         compute_logits(model, normed, logits, options, worker_pool);
1266         add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start)
     ↪ ;
1267         start = profile_start(hotspot_profile);
1268         result.logits.assign(logits.begin(), logits.end());
1269         result.predicted_token = argmax(result.logits);
1270         add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::logits_result_ms,
     ↪ start);
1271     } else {
1272         start = profile_start(hotspot_profile);
1273         result.predicted_token = compute_predicted_token(model, normed, options
     ↪ , worker_pool);
1274         add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start)
     ↪ ;
1275     }
1276     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms,
     ↪ step_start);
1277     return result;
1278 }
1279
1280 std::vector<std::string> gpt2_bytes_to_unicode() {
1281     std::vector<std::int32_t> bytes;
```

```
    for (std::int32_t value = LCQI_GPT2_ASCII_EXCLAMATION;
         value <= LCQI_GPT2_ASCII_TILDE;
         ++value) {
        bytes.push_back(value);
    }
    for (std::int32_t value = LCQI_GPT2_LATIN_INVERTED_EXCLAMATION;
         value <= LCQI_GPT2_LATIN_NOT_SIGN;
         ++value) {
        bytes.push_back(value);
    }
    for (std::int32_t value = LCQI_GPT2_LATIN_REGISTERED;
         value <= LCQI_GPT2_LATIN_Y_DIAERESIS;
         ++value) {
        bytes.push_back(value);
    }

    std::unordered_set<std::int32_t> byte_set(bytes.begin(), bytes.end());
    std::vector<std::int32_t> chars = bytes;
    std::int32_t private_offset = 0;
    for (std::int32_t byte = 0; byte < LCQI_GPT2_BYTE_COUNT; ++byte) {
        if (!byte_set.contains(byte)) {
            bytes.push_back(byte);
            chars.push_back(LCQI_GPT2_UNICODE_PRIVATE_BASE + private_offset);
            ++private_offset;
        }
    }

    std::vector<std::string> mapping(LCQI_GPT2_BYTE_COUNT);
    for (std::size_t index = 0; index < bytes.size(); ++index) {
        mapping[checked_size(bytes[index], "byte unicode index")] =
            encode_codepoint(static_cast<std::uint32_t>(chars[index]));
    }
    return mapping;
}

std::unordered_map<std::string, unsigned char> gpt2_unicode_to_bytes(
    const std::vector<std::string>& byte_encoder) {
    std::unordered_map<std::string, unsigned char> result;
    for (std::size_t byte = 0; byte < byte_encoder.size(); ++byte) {
        result.emplace(byte_encoder[byte], static_cast<unsigned char>(byte));
    }
    return result;
}

std::string bytes_to_bpe_alphabet(std::string_view text) {
    static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode↩
    ↪ ();
    std::string result;
    for (const unsigned char byte : text) {
        result += BYTE_ENCODER[byte];
    }
    return result;
```

```
}

std::vector<std::string> utf8_symbols(std::string_view text) {
    std::vector<std::string> symbols;
    std::size_t offset = 0;
    while (offset < text.size()) {
        const unsigned char lead = static_cast<unsigned char>(text[offset]);
        std::size_t width = 1;
        if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
            width = 2;
        } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
            width = 3;
        } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
            width = 4;
        }
        if (offset + width > text.size()) {
            throw std::runtime_error("invalid UTF-8/BPE symbol boundary");
        }
        symbols.emplace_back(text.substr(offset, width));
        offset += width;
    }
    return symbols;
}

std::string pair_key(std::string_view left, std::string_view right) {
    std::string key(left);
    key.push_back('\n');
    key += right;
    return key;
}

std::vector<std::string> bpe_apply(const Gpt2Tokenizer& tokenizer, std::string_view token) {
    std::vector<std::string> word = utf8_symbols(token);
    if (word.size() <= 1) {
        return word;
    }

    while (true) {
        std::optional<std::int32_t> best_rank;
        std::string best_left;
        std::string best_right;
        for (std::size_t index = 0; index + 1 < word.size(); ++index) {
            const auto rank = tokenizer.bpe_ranks.find(pair_key(word[index], word[index + 1]));
            if (rank != tokenizer.bpe_ranks.end() &&
                (!best_rank.has_value() || rank->second < *best_rank)) {
                best_rank = rank->second;
                best_left = word[index];
                best_right = word[index + 1];
            }
        }
```

```
        if (!best_rank.has_value()) {
            return word;
        }

        std::vector<std::string> merged;
        std::size_t index = 0;
        while (index < word.size()) {
            if (index + 1 < word.size() && word[index] == best_left &&
                word[index + 1] == best_right) {
                merged.push_back(best_left + best_right);
                index += LCQI_GPT2_PAIR_TOKEN_COUNT;
            } else {
                merged.push_back(word[index]);
                ++index;
            }
        }
        word = std::move(merged);
        if (word.size() == 1) {
            return word;
        }
    }
}

bool is_ascii_letter(unsigned char value) {
    return (value >= 'A' && value <= 'Z') || (value >= 'a' && value <= 'z');
}

bool is_ascii_digit(unsigned char value) {
    return value >= '0' && value <= '9';
}

bool is_ascii_space(unsigned char value) {
    return value == ' ' || value == '\t' || value == '\n' ||
           value == '\r' || value == '\f' || value == '\v';
}

bool is_gpt2_punctuation(unsigned char value) {
    return !is_ascii_letter(value) && !is_ascii_digit(value) &&
           !is_ascii_space(value);
}

std::size_t consume_utf8_codepoint(std::string_view text, std::size_t offset) {
    const unsigned char lead = static_cast<unsigned char>(text[offset]);
    std::size_t width = 1;
    if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
        width = 2;
    } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
        width = 3;
    } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
        width = 4;
    }
    if (offset + width > text.size()) {
```

```
1435         throw std::runtime_error("invalid UTF-8 text for GPT-2 tokenizer");
1436     }
1437     return offset + width;
1438 }
1439
1440 bool starts_with_contraction(std::string_view text,
1441                              std::size_t offset,
1442                              std::string_view contraction) {
1443     if (offset + contraction.size() > text.size()) {
1444         return false;
1445     }
1446     for (std::size_t index = 0; index < contraction.size(); ++index) {
1447         char left = text[offset + index];
1448         const char right = contraction[index];
1449         if (left >= 'A' && left <= 'Z') {
1450             left = static_cast<char>(left - 'A' + 'a');
1451         }
1452         if (left != right) {
1453             return false;
1454         }
1455     }
1456     return true;
1457 }
1458
1459 std::vector<std::string> gpt2_pretokenize(std::string_view text) {
1460     std::vector<std::string> pieces;
1461     std::size_t offset = 0;
1462     const std::array<std::string_view, 7> contractions{
1463         "'s", "'t", "'re", "'ve", "'m", "'ll", "'d",
1464     };
1465     while (offset < text.size()) {
1466         bool emitted_contraction = false;
1467         for (const std::string_view contraction : contractions) {
1468             if (starts_with_contraction(text, offset, contraction)) {
1469                 pieces.emplace_back(text.substr(offset, contraction.size()));
1470                 offset += contraction.size();
1471                 emitted_contraction = true;
1472                 break;
1473             }
1474         }
1475         if (emitted_contraction) {
1476             continue;
1477         }
1478
1479         const std::size_t begin = offset;
1480         bool has_prefix_space = false;
1481         if (text[offset] == ' ' && offset + 1 < text.size() &&
1482             !is_ascii_space(static_cast<unsigned char>(text[offset + 1]))) {
1483             has_prefix_space = true;
1484             ++offset;
1485         }
1486
```

```
        if (offset >= text.size()) {
            pieces.emplace_back(text.substr(begin, offset - begin));
            continue;
        }

        const unsigned char ch = static_cast<unsigned char>(text[offset]);
        if (is_ascii_letter(ch)) {
            while (offset < text.size() &&
                   is_ascii_letter(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else if (is_ascii_digit(ch)) {
            while (offset < text.size() &&
                   is_ascii_digit(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else if (is_gpt2_punctuation(ch) && ch != LCQI_GPT2_ASCII_APOSTROPHE) {
            while (offset < text.size() &&
                   is_gpt2_punctuation(static_cast<unsigned char>(text[offset])) &&
                   static_cast<unsigned char>(text[offset]) != LCQI_GPT2_ASCII_APOSTROPHE) {
                offset = consume_utf8_codepoint(text, offset);
            }
        } else if (is_ascii_space(ch)) {
            while (offset < text.size() &&
                   is_ascii_space(static_cast<unsigned char>(text[offset]))) {
                ++offset;
            }
        } else {
            offset = consume_utf8_codepoint(text, offset);
        }

        if (has_prefix_space || offset > begin) {
            pieces.emplace_back(text.substr(begin, offset - begin));
        } else {
            ++offset;
        }
    }
    return pieces;
}

std::int32_t parse_i32_field(JsonCursor& cursor) {
    const std::int64_t value = cursor.parse_i64();
    if (value < std::numeric_limits<std::int32_t>::min() ||
        value > std::numeric_limits<std::int32_t>::max()) {
        throw std::runtime_error("JSON int32 field out of range");
    }
    return static_cast<std::int32_t>(value);
}

```

```
std::unordered_map<std::string, std::int32_t> parse_vocab_json(std::string_view text) {
    JsonCursor cursor(text);
    std::unordered_map<std::string, std::int32_t> vocab;
    cursor.expect('{');
    if (cursor.consume('}')) {
        return vocab;
    }
    while (true) {
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        const std::int32_t id = parse_i32_field(cursor);
        vocab.emplace(key, id);
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }
    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after vocab JSON");
    }
    return vocab;
}

std::vector<std::string> build_id_to_token(
    const std::unordered_map<std::string, std::int32_t>& vocab) {
    std::int32_t max_id = -1;
    for (const auto& [token, id] : vocab) {
        if (id < 0) {
            throw std::runtime_error("GPT-2 vocab id cannot be negative");
        }
        max_id = std::max(max_id, id);
    }
    std::vector<std::string> result(checked_size(max_id + 1, "max vocab id"));
    for (const auto& [token, id] : vocab) {
        std::string& slot = result[checked_size(id, "vocab id")];
        if (!slot.empty()) {
            throw std::runtime_error("duplicate GPT-2 vocab id");
        }
        slot = token;
    }
    return result;
}

std::unordered_map<std::string, std::int32_t> parse_merges(std::string_view text) {
    std::unordered_map<std::string, std::int32_t> ranks;
    std::istringstream input{std::string(text)};
    std::string line;
    std::int32_t rank = 0;
    while (std::getline(input, line)) {
        if (line.empty() || line[0] == '#') {
```

```
            continue;
        }
        std::istringstream line_stream(line);
        std::string left;
        std::string right;
        if (!(line_stream >> left >> right)) {
            throw std::runtime_error("invalid GPT-2 merges line");
        }
        ranks.emplace(pair_key(left, right), rank);
        ++rank;
    }
    return ranks;
}

void transpose_hf_conv1d(std::vector<float>& values,
                         std::int32_t input_size,
                         std::int32_t output_size) {
    if (values.size() != matrix_size(input_size, output_size)) {
        throw std::runtime_error("GPT-2 Conv1D tensor size mismatch");
    }
    std::vector<float> transposed(matrix_size(output_size, input_size), 0.0F);
    for (std::int32_t input = 0; input < input_size; ++input) {
        for (std::int32_t output = 0; output < output_size; ++output) {
            transposed[matrix_size(output, input_size) + checked_size(input, "input")] =
                values[matrix_size(input, output_size) + checked_size(output, "output")];
        }
    }
    values = std::move(transposed);
}

std::vector<float> require_tensor(const SafeTensorFile& file,
                                  std::string_view name,
                                  std::span<const std::int64_t> expected_shape) {
    const SafeTensorEntry* entry = file.manifest.find_tensor(name);
    if (entry == nullptr) {
        throw std::runtime_error("missing GPT-2 tensor: " + std::string(name));
    }
    if (entry->shape.size() != expected_shape.size()) {
        throw std::runtime_error("GPT-2 tensor rank mismatch: " + std::string(name));
    }
    for (std::size_t index = 0; index < expected_shape.size(); ++index) {
        if (entry->shape[index] != expected_shape[index]) {
            throw std::runtime_error("GPT-2 tensor shape mismatch: " + std::string(name));
        }
    }
    return read_safetensor_f32_tensor(file, name);
}
```

```
1633
1634 std::vector<float> require_first_tensor(const SafeTensorFile& file,
1635                                          std::span<const std::string_view> names
     ↪ ,
1636                                          std::span<const std::int64_t> ↩
     ↪ expected_shape) {
1637     for (const std::string_view name : names) {
1638         if (file.manifest.find_tensor(name) != nullptr) {
1639             return require_tensor(file, name, expected_shape);
1640         }
1641     }
1642     throw std::runtime_error("missing GPT-2 tensor: " + std::string(names.front
     ↪ ()));
1643 }
1644
1645 Gpt2LinearF32 make_linear(std::int32_t input_size,
1646                           std::int32_t output_size,
1647                           std::vector<float> weights,
1648                           std::vector<float> bias = {}) {
1649     Gpt2LinearF32 linear;
1650     linear.input_size = input_size;
1651     linear.output_size = output_size;
1652     linear.weights = std::move(weights);
1653     linear.bias = std::move(bias);
1654     validate_linear(linear, "make GPT-2 linear");
1655     return linear;
1656 }
1657
1658 std::vector<float> zeros(std::int32_t count) {
1659     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
1660 }
1661
1662 std::vector<float> ones(std::int32_t count) {
1663     return std::vector<float>(checked_size(count, "one count"), 1.0F);
1664 }
1665
1666 std::string tensor_name(std::int32_t layer, std::string_view suffix) {
1667     return "transformer.h." + std::to_string(layer) + "." + std::string(suffix)
     ↪ ;
1668 }
1669
1670 std::string bare_tensor_name(std::int32_t layer, std::string_view suffix) {
1671     return "h." + std::to_string(layer) + "." + std::string(suffix);
1672 }
1673
1674 std::array<std::string_view, 2> embedding_names(std::string_view bare) {
1675     if (bare == "wte.weight") {
1676         return {"transformer.wte.weight", "wte.weight"};
1677     }
1678     if (bare == "wpe.weight") {
1679         return {"transformer.wpe.weight", "wpe.weight"};
1680     }
```

```
    if (bare == "ln_f.weight") {
        return {"transformer.ln_f.weight", "ln_f.weight"};
    }
    if (bare == "ln_f.bias") {
        return {"transformer.ln_f.bias", "ln_f.bias"};
    }
    throw std::runtime_error("unsupported GPT-2 embedding tensor alias");
}

std::array<std::string, 2> layer_names(std::int32_t layer, std::string_view suffix) {
    return {tensor_name(layer, suffix), bare_tensor_name(layer, suffix)};
}

std::vector<float> require_layer_tensor(const SafeTensorFile& file,
                                        std::int32_t layer,
                                        std::string_view suffix,
                                        std::span<const std::int64_t> expected_shape) {
    const std::array<std::string, 2> names = layer_names(layer, suffix);
    const std::array<std::string_view, 2> views{names[0], names[1]};
    return require_first_tensor(file, views, expected_shape);
}

std::filesystem::path find_gpt2_weight_file(const std::filesystem::path& model_directory) {
    const std::array<const char*, 2> candidates{
        "model.safetensors",
        "pytorch_model.safetensors",
    };
    for (const char* candidate : candidates) {
        const std::filesystem::path path = model_directory / candidate;
        if (std::filesystem::exists(path)) {
            return path;
        }
    }
    throw std::runtime_error("cannot find model.safetensors in GPT-2 directory");
}

}  // namespace

Gpt2Tokenizer load_gpt2_tokenizer(const std::filesystem::path& vocab_json,
                                  const std::filesystem::path& merges_txt,
                                  std::int32_t bos_token_id,
                                  std::int32_t eos_token_id) {
    Gpt2Tokenizer tokenizer;
    tokenizer.bos_token_id = bos_token_id;
    tokenizer.eos_token_id = eos_token_id;
    tokenizer.vocab = parse_vocab_json(read_text_file(vocab_json));
    tokenizer.id_to_token = build_id_to_token(tokenizer.vocab);
    tokenizer.bpe_ranks = parse_merges(read_text_file(merges_txt));
```

```
    if (tokenizer.vocab.empty() || tokenizer.bpe_ranks.empty()) {
        throw std::runtime_error("GPT-2 tokenizer assets are empty");
    }
    return tokenizer;
}

std::vector<std::int32_t> gpt2_encode(const Gpt2Tokenizer& tokenizer,
                                      std::string_view text) {
    std::vector<std::int32_t> ids;
    for (const std::string& piece : gpt2_pretokenize(text)) {
        const std::string byte_piece = bytes_to_bpe_alphabet(piece);
        const std::vector<std::string> bpe_tokens = bpe_apply(tokenizer, byte_piece);
        for (const std::string& token : bpe_tokens) {
            const auto found = tokenizer.vocab.find(token);
            if (found == tokenizer.vocab.end()) {
                throw std::runtime_error("GPT-2 BPE token is missing from vocab");
            }
            ids.push_back(found->second);
        }
    }
    return ids;
}

std::string gpt2_decode(const Gpt2Tokenizer& tokenizer,
                        std::span<const std::int32_t> token_ids) {
    static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode();
    static const std::unordered_map<std::string, unsigned char> BYTE_DECODER =
        gpt2_unicode_to_bytes(BYTE_ENCODER);

    std::string token_text;
    for (const std::int32_t token_id : token_ids) {
        if (token_id < 0 || token_id >= static_cast<std::int32_t>(tokenizer.id_to_token.size())) {
            throw std::runtime_error("GPT-2 token id out of range");
        }
        token_text += tokenizer.id_to_token[checked_size(token_id, "token id")];
    }

    std::string result;
    const std::vector<std::string> symbols = utf8_symbols(token_text);
    for (const std::string& symbol : symbols) {
        const auto found = BYTE_DECODER.find(symbol);
        if (found == BYTE_DECODER.end()) {
            throw std::runtime_error("GPT-2 token cannot be decoded to byte");
        }
        result.push_back(static_cast<char>(found->second));
    }
    return result;
```

```
}

Gpt2Config load_gpt2_config(const std::filesystem::path& config_json) {
    const std::string config_text = read_text_file(config_json);
    JsonCursor cursor(config_text);
    Gpt2Config config;
    config.bos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
    config.eos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;

    cursor.expect('{');
    if (cursor.consume('}')) {
        throw std::runtime_error("GPT-2 config is empty");
    }
    while (true) {
        const std::string key = cursor.parse_string();
        cursor.expect(':');
        if (key == "n_embd") {
            config.hidden_size = parse_i32_field(cursor);
        } else if (key == "n_head") {
            config.head_count = parse_i32_field(cursor);
        } else if (key == "n_layer") {
            config.layer_count = parse_i32_field(cursor);
        } else if (key == "n_positions" || key == "n_ctx") {
            const std::int32_t value = parse_i32_field(cursor);
            config.max_positions = std::max(config.max_positions, value);
        } else if (key == "vocab_size") {
            config.vocab_size = parse_i32_field(cursor);
        } else if (key == "n_inner") {
            config.intermediate_size = parse_i32_field(cursor);
        } else if (key == "layer_norm_epsilon") {
            config.layer_norm_epsilon = static_cast<float>(cursor.parse_number↩
    ↪ ());
        } else if (key == "activation_function") {
            config.activation_function = cursor.parse_string();
        } else if (key == "bos_token_id") {
            config.bos_token_id = parse_i32_field(cursor);
        } else if (key == "eos_token_id") {
            config.eos_token_id = parse_i32_field(cursor);
        } else {
            cursor.skip_value();
        }
        if (cursor.consume('}')) {
            break;
        }
        cursor.expect(',');
    }
    if (!cursor.eof_after_ws()) {
        throw std::runtime_error("trailing data after GPT-2 config JSON");
    }
    if (config.intermediate_size == 0 && config.hidden_size > 0) {
        config.intermediate_size = config.hidden_size * 4;
    }
```

```
    validate_config(config);
    return config;
}

Gpt2ReferenceModel load_gpt2_reference_model(const std::filesystem::path& config_json,
                                             const std::filesystem::path& safetensors_path) {
    Gpt2ReferenceModel model;
    model.config = load_gpt2_config(config_json);
    const SafeTensorFile file = load_safetensors_file(safetensors_path);
    const Gpt2Config& config = model.config;
    const std::array<std::int64_t, 2> wte_shape{config.vocab_size, config.hidden_size};
    const std::array<std::int64_t, 2> wpe_shape{config.max_positions, config.hidden_size};
    const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};

    const std::array<std::string_view, 2> wte_names = embedding_names("wte.weight");
    const std::array<std::string_view, 2> wpe_names = embedding_names("wpe.weight");
    model.token_embedding = require_first_tensor(file, wte_names, wte_shape);
    model.position_embedding = require_first_tensor(file, wpe_names, wpe_shape);
    model.layers.resize(checked_size(config.layer_count, "layer_count"));

    for (std::int32_t layer_id = 0; layer_id < config.layer_count; ++layer_id) {
        Gpt2LayerWeightsF32& layer = model.layers[checked_size(layer_id, "layer_id")];
        layer.ln_1_weight = require_layer_tensor(file, layer_id, "ln_1.weight", hidden_shape);
        layer.ln_1_bias = require_layer_tensor(file, layer_id, "ln_1.bias", hidden_shape);
        layer.ln_2_weight = require_layer_tensor(file, layer_id, "ln_2.weight", hidden_shape);
        layer.ln_2_bias = require_layer_tensor(file, layer_id, "ln_2.bias", hidden_shape);

        const std::array<std::int64_t, 2> c_attn_shape{
            config.hidden_size,
            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
        std::vector<float> c_attn_weight =
            require_layer_tensor(file, layer_id, "attn.c_attn.weight", c_attn_shape);
        transpose_hf_conv1d(c_attn_weight,
                            config.hidden_size,
                            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS);
        const std::array<std::int64_t, 1> c_attn_bias_shape{
            config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
```

```
        layer.c_attn = make_linear(config.hidden_size,
                                   config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS,
                                   std::move(c_attn_weight),
                                   require_layer_tensor(file,
                                                        layer_id,
                                                        "attn.c_attn.bias",
                                                        c_attn_bias_shape));

        const std::array<std::int64_t, 2> c_proj_shape{config.hidden_size, config.hidden_size};
        std::vector<float> c_proj_weight =
            require_layer_tensor(file, layer_id, "attn.c_proj.weight", c_proj_shape);
        transpose_hf_conv1d(c_proj_weight, config.hidden_size, config.hidden_size);
        layer.c_proj = make_linear(config.hidden_size,
                                   config.hidden_size,
                                   std::move(c_proj_weight),
                                   require_layer_tensor(file,
                                                        layer_id,
                                                        "attn.c_proj.bias",
                                                        hidden_shape));

        const std::array<std::int64_t, 2> c_fc_shape{
            config.hidden_size,
            config.intermediate_size};
        std::vector<float> c_fc_weight =
            require_layer_tensor(file, layer_id, "mlp.c_fc.weight", c_fc_shape);
        transpose_hf_conv1d(c_fc_weight, config.hidden_size, config.intermediate_size);
        const std::array<std::int64_t, 1> intermediate_shape{config.intermediate_size};
        layer.c_fc = make_linear(config.hidden_size,
                                 config.intermediate_size,
                                 std::move(c_fc_weight),
                                 require_layer_tensor(file,
                                                      layer_id,
                                                      "mlp.c_fc.bias",
                                                      intermediate_shape));

        const std::array<std::int64_t, 2> mlp_proj_shape{
            config.intermediate_size,
            config.hidden_size};
        std::vector<float> mlp_proj_weight =
            require_layer_tensor(file, layer_id, "mlp.c_proj.weight", mlp_proj_shape);
        transpose_hf_conv1d(mlp_proj_weight, config.intermediate_size, config.hidden_size);
        layer.mlp_c_proj = make_linear(config.intermediate_size,
                                       config.hidden_size,
```

```
                                        std::move(mlp_proj_weight),
                                        require_layer_tensor(file,
                                                             layer_id,
                                                             "mlp.c_proj.bias",
                                                             hidden_shape));
    }

    const std::array<std::string_view, 2> ln_weight_names = embedding_names("ln_f.weight");
    const std::array<std::string_view, 2> ln_bias_names = embedding_names("ln_f.bias");
    model.final_ln_weight = require_first_tensor(file, ln_weight_names, hidden_shape);
    model.final_ln_bias = require_first_tensor(file, ln_bias_names, hidden_shape);
    if (file.manifest.find_tensor("lm_head.weight") != nullptr) {
        model.tie_lm_head_to_embedding = false;
        model.lm_head_weight = require_tensor(file, "lm_head.weight", wte_shape);
    } else {
        model.tie_lm_head_to_embedding = true;
    }
    validate_model(model);
    return model;
}

Gpt2ReferenceModel load_gpt2_from_directory(const std::filesystem::path& model_directory) {
    return load_gpt2_reference_model(model_directory / "config.json",
                                     find_gpt2_weight_file(model_directory));
}

Gpt2ForwardWorkspace::Gpt2ForwardWorkspace(const Gpt2Config& config) : config_(config) {
    this->reset_for_config(config);
}

void Gpt2ForwardWorkspace::reset_for_config(const Gpt2Config& config) {
    validate_config(config);
    this->config_ = config;
    const std::size_t hidden_size = checked_size(this->config_.hidden_size, "hidden_size");
    const std::size_t intermediate_size =
        checked_size(this->config_.intermediate_size, "intermediate_size");
    this->hidden_.assign(hidden_size, 0.0F);
    this->normed_.assign(hidden_size, 0.0F);
    this->query_.assign(hidden_size, 0.0F);
    this->key_.assign(hidden_size, 0.0F);
    this->value_.assign(hidden_size, 0.0F);
    this->attention_.assign(hidden_size, 0.0F);
    this->projected_.assign(hidden_size, 0.0F);
    this->qkv_packed_.assign(
```

```
        matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS, this->config_.hidden_size),
        0.0F);
    this->mlp_fc_.assign(intermediate_size, 0.0F);
    this->mlp_out_.assign(hidden_size, 0.0F);
    this->logits_.assign(checked_size(this->config_.vocab_size, "vocab_size"), 0.0F);
    this->scores_.assign(checked_size(this->config_.max_positions, "max_positions"), 0.0F);
}

std::span<float> Gpt2ForwardWorkspace::hidden() noexcept {
    return this->hidden_;
}

std::span<float> Gpt2ForwardWorkspace::normed() noexcept {
    return this->normed_;
}

std::span<float> Gpt2ForwardWorkspace::query() noexcept {
    return this->query_;
}

std::span<float> Gpt2ForwardWorkspace::key() noexcept {
    return this->key_;
}

std::span<float> Gpt2ForwardWorkspace::value() noexcept {
    return this->value_;
}

std::span<float> Gpt2ForwardWorkspace::attention() noexcept {
    return this->attention_;
}

std::span<float> Gpt2ForwardWorkspace::projected() noexcept {
    return this->projected_;
}

std::span<float> Gpt2ForwardWorkspace::qkv_packed() noexcept {
    return this->qkv_packed_;
}

std::span<float> Gpt2ForwardWorkspace::mlp_fc() noexcept {
    return this->mlp_fc_;
}

std::span<float> Gpt2ForwardWorkspace::mlp_out() noexcept {
    return this->mlp_out_;
}

std::span<float> Gpt2ForwardWorkspace::logits() noexcept {
```

```
    return this->logits_;
}

std::span<float> Gpt2ForwardWorkspace::scores_prefix(std::int32_t count) {
    if (count < 0 || count > this->config_.max_positions) {
        throw std::runtime_error("GPT-2 attention scores count out of range");
    }
    return std::span<float>(this->scores_.data(), checked_size(count, "scores count"));
}

Gpt2KvCache::Gpt2KvCache(const Gpt2Config& config) : config_(config) {
    validate_config(this->config_);
    const std::size_t element_count =
        checked_size(this->config_.layer_count, "layer_count") *
        checked_size(this->config_.max_positions, "max_positions") *
        checked_size(this->config_.head_count, "head_count") *
        checked_size(head_dim(this->config_), "head_dim");
    this->keys_.assign(element_count, 0.0F);
    this->values_.assign(element_count, 0.0F);
    this->written_.assign(
        checked_size(this->config_.layer_count, "layer_count") *
            checked_size(this->config_.max_positions, "max_positions"),
        0);
}

void Gpt2KvCache::append(std::int32_t layer_id,
                         std::int32_t model_position,
                         std::span<const float> key,
                         std::span<const float> value) {
    const std::size_t hidden_size = checked_size(this->config_.hidden_size, "hidden_size");
    if (key.size() != hidden_size || value.size() != hidden_size) {
        throw std::runtime_error("GPT-2 KV key/value size mismatch");
    }
    this->validate_address(layer_id, model_position, 0);
    const std::int32_t local_head_dim = head_dim(this->config_);
    for (std::int32_t head = 0; head < this->config_.head_count; ++head) {
        const std::size_t source =
            checked_size(head, "head") * checked_size(local_head_dim, "head_dim");
        const std::size_t target = this->base_offset(layer_id, model_position, head);
        std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
                    local_head_dim,
                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
        std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
                    local_head_dim,
                    this->values_.begin() + static_cast<std::ptrdiff_t>(target));
    }
    const std::size_t written_index =
```

```
        checked_size(layer_id, "layer_id") *
            checked_size(this->config_.max_positions, "max_positions") +
        checked_size(model_position, "model_position");
    this->written_[written_index] = 1;
    this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
}

std::span<const float> Gpt2KvCache::key(std::int32_t layer_id,
                                        std::int32_t model_position,
                                        std::int32_t head) const {
    this->validate_address(layer_id, model_position, head);
    this->validate_written(layer_id, model_position);
    return this->key_unchecked(layer_id, model_position, head);
}

std::span<const float> Gpt2KvCache::value(std::int32_t layer_id,
                                          std::int32_t model_position,
                                          std::int32_t head) const {
    this->validate_address(layer_id, model_position, head);
    this->validate_written(layer_id, model_position);
    return this->value_unchecked(layer_id, model_position, head);
}

void detail::attend_cached_position(const Gpt2KvCache& cache,
                                    std::int32_t layer_id,
                                    std::int32_t model_position,
                                    std::span<const float> query,
                                    std::span<float> scores,
                                    std::span<float> output) {
    cache.validate_address(layer_id, model_position, 0);
    cache.validate_written(layer_id, model_position);
    const std::int32_t local_head_dim = head_dim(cache.config_);
    const std::size_t hidden_size = checked_size(cache.config_.hidden_size, "hidden_size");
    const std::size_t local_head_dim_size = checked_size(local_head_dim, "head_dim");
    const std::size_t model_position_size =
        checked_size(model_position, "model_position");
    if (query.size() != hidden_size || output.size() != hidden_size ||
        scores.size() < model_position_size + 1) {
        throw std::runtime_error("GPT-2 cached attention workspace size mismatch");
    }

    const float score_scale = 1.0F / std::sqrt(static_cast<float>(local_head_dim));
    std::fill(output.begin(), output.end(), 0.0F);
    for (std::int32_t head = 0; head < cache.config_.head_count; ++head) {
        const std::size_t head_base =
            checked_size(head, "head") * local_head_dim_size;
        float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
        for (std::size_t past = 0; past <= model_position_size; ++past) {
```

```
            const std::int32_t past_i32 = static_cast<std::int32_t>(past);
            const std::span<const float> key =
                cache.key_unchecked(layer_id, past_i32, head);
            float dot = 0.0F;
            for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
                dot += query[head_base + dim] * key[dim];
            }
            const float score = dot * score_scale;
            scores[past] = score;
            max_score = std::max(max_score, score);
        }

        float denominator = 0.0F;
        for (std::size_t past = 0; past <= model_position_size; ++past) {
            scores[past] = std::exp(scores[past] - max_score);
            denominator += scores[past];
        }
        if (denominator <= 0.0F) {
            throw std::runtime_error("GPT-2 cached attention denominator is invalid");
        }

        for (std::size_t past = 0; past <= model_position_size; ++past) {
            const std::int32_t past_i32 = static_cast<std::int32_t>(past);
            const float probability = scores[past] / denominator;
            const std::span<const float> value =
                cache.value_unchecked(layer_id, past_i32, head);
            for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
                output[head_base + dim] += probability * value[dim];
            }
        }
    }
}

std::span<const float> Gpt2KvCache::key_unchecked(std::int32_t layer_id,
                                                  std::int32_t model_position,
                                                  std::int32_t head) const noexcept {
    const std::size_t offset = this->base_offset_unchecked(layer_id, model_position, head);
    return std::span<const float>(
        this->keys_.data() + offset,
        static_cast<std::size_t>(head_dim(this->config_)));
}

std::span<const float> Gpt2KvCache::value_unchecked(std::int32_t layer_id,
                                                    std::int32_t model_position,
                                                    std::int32_t head) const noexcept {
    const std::size_t offset = this->base_offset_unchecked(layer_id, model_position, head);
```

```
    return std::span<const float>(
        this->values_.data() + offset,
        static_cast<std::size_t>(head_dim(this->config_)));
}

std::int32_t Gpt2KvCache::filled_tokens() const noexcept {
    return this->filled_tokens_;
}

std::size_t Gpt2KvCache::byte_size() const noexcept {
    return (this->keys_.size() + this->values_.size()) * sizeof(float) +
           this->written_.size() * sizeof(std::uint8_t);
}

std::size_t Gpt2KvCache::base_offset(std::int32_t layer_id,
                                     std::int32_t model_position,
                                     std::int32_t head) const {
    return (((checked_size(layer_id, "layer_id") *
              checked_size(this->config_.max_positions, "max_positions")) +
             checked_size(model_position, "model_position")) *
                checked_size(this->config_.head_count, "head_count") +
            checked_size(head, "head")) *
           checked_size(head_dim(this->config_), "head_dim");
}

std::size_t Gpt2KvCache::base_offset_unchecked(std::int32_t layer_id,
                                               std::int32_t model_position,
                                               std::int32_t head) const ↩
    ↪ noexcept {
    return (((static_cast<std::size_t>(layer_id) *
              static_cast<std::size_t>(this->config_.max_positions)) +
             static_cast<std::size_t>(model_position)) *
                static_cast<std::size_t>(this->config_.head_count) +
            static_cast<std::size_t>(head)) *
           static_cast<std::size_t>(head_dim(this->config_));
}

void Gpt2KvCache::validate_address(std::int32_t layer_id,
                                   std::int32_t model_position,
                                   std::int32_t head) const {
    if (layer_id < 0 || layer_id >= this->config_.layer_count) {
        throw std::runtime_error("GPT-2 KV layer_id out of range");
    }
    if (model_position < 0 || model_position >= this->config_.max_positions) {
        throw std::runtime_error("GPT-2 KV model_position out of range");
    }
    if (head < 0 || head >= this->config_.head_count) {
        throw std::runtime_error("GPT-2 KV head out of range");
    }
}

void Gpt2KvCache::validate_written(std::int32_t layer_id,
```

```
                                       std::int32_t model_position) const {
    const std::size_t written_index =
        checked_size(layer_id, "layer_id") *
            checked_size(this->config_.max_positions, "max_positions") +
        checked_size(model_position, "model_position");
    if (written_index >= this->written_.size() || this->written_[written_index]
    ↪  == 0) {
        throw std::runtime_error("GPT-2 KV slot has not been written");
    }
}

Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& 
    ↪ model)
    : Gpt2CachedGreedyDecoder(model, nullptr) {}

Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
    const Gpt2ReferenceModel& model,
    Gpt2HotspotProfile* hotspot_profile)
    : Gpt2CachedGreedyDecoder(model, hotspot_profile, Gpt2ExecutionOptions{}) 
    ↪ {}

Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(
    const Gpt2ReferenceModel& model,
    Gpt2HotspotProfile* hotspot_profile,
    const Gpt2ExecutionOptions& execution_options)
    : model_(&model),
      hotspot_profile_(hotspot_profile),
      execution_options_(execution_options),
      cache_(model.config),
      workspace_(model.config),
      worker_pool_(make_worker_pool(model, execution_options)) {
    validate_model(*this->model_);
}

Gpt2CachedGreedyDecoder::~Gpt2CachedGreedyDecoder() = default;

Gpt2CachedGreedyDecoder::Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) 
    ↪ noexcept =
    default;

Gpt2CachedGreedyDecoder& Gpt2CachedGreedyDecoder::operator=(
    Gpt2CachedGreedyDecoder&&) noexcept = default;

std::int32_t Gpt2CachedGreedyDecoder::step(std::int32_t token_id) {
    return run_gpt2_cached_step_unchecked(*this->model_,
                                          this->cache_,
                                          this->workspace_,
                                          token_id,
                                          false,
                                          this->hotspot_profile_,
                                          this->execution_options_,
                                          this->worker_pool_.get())
```

```
        .predicted_token;
}

Gpt2ForwardResult Gpt2CachedGreedyDecoder::step_with_logits(std::int32_t token_id) {
    return run_gpt2_cached_step_unchecked(*this->model_,
                                          this->cache_,
                                          this->workspace_,
                                          token_id,
                                          true,
                                          this->hotspot_profile_,
                                          this->execution_options_,
                                          this->worker_pool_.get());
}

const Gpt2KvCache& Gpt2CachedGreedyDecoder::cache() const noexcept {
    return this->cache_;
}

std::int32_t Gpt2CachedGreedyDecoder::filled_tokens() const noexcept {
    return this->cache_.filled_tokens();
}

std::size_t Gpt2CachedGreedyDecoder::kv_cache_bytes() const noexcept {
    return this->cache_.byte_size();
}

std::int32_t Gpt2CachedGreedyDecoder::worker_count() const noexcept {
    return this->worker_pool_ == nullptr ? 1 : this->worker_pool_->worker_count();
}

Gpt2ForwardResult run_gpt2_forward(const Gpt2ReferenceModel& model,
                                   std::span<const std::int32_t> token_ids) {
    validate_model(model);
    if (token_ids.empty()) {
        throw std::runtime_error("GPT-2 forward needs at least one token");
    }
    if (token_ids.size() > checked_size(model.config.max_positions, "max_positions")) {
        throw std::runtime_error("GPT-2 token_ids exceed max_positions");
    }

    const std::int32_t sequence_length = static_cast<std::int32_t>(token_ids.size());
    std::vector<float> hidden_by_pos(matrix_size(sequence_length, model.config.hidden_size), 0.0F);
    for (std::int32_t position = 0; position < sequence_length; ++position) {
        const std::int32_t token_id = token_ids[checked_size(position, "position")];
        if (token_id < 0 || token_id >= model.config.vocab_size) {
            throw std::runtime_error("GPT-2 token id out of range");
```

```
        }
        const std::span<const float> token =
            row_span(model.token_embedding, token_id, model.config.hidden_size)↩
    ↪ ;
        const std::span<const float> position_embedding =
            row_span(model.position_embedding, position, model.config.↩
    ↪ hidden_size);
        std::span<float> hidden = std::span<float>(
            hidden_by_pos.data() + matrix_size(position, model.config.↩
    ↪ hidden_size),
            checked_size(model.config.hidden_size, "hidden_size"));
        for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
            hidden[checked_size(dim, "dim")] =
                token[checked_size(dim, "dim")] + position_embedding[↩
    ↪ checked_size(dim, "dim")];
        }
    }

    for (const Gpt2LayerWeightsF32& layer : model.layers) {
        forward_gpt2_layer(model.config, layer, sequence_length, hidden_by_pos)↩
    ↪ ;
    }

    std::vector<float> normed(checked_size(model.config.hidden_size, "↩
    ↪ hidden_size"), 0.0F);
    const std::span<const float> final_hidden = std::span<const float>(
        hidden_by_pos.data() + matrix_size(sequence_length - 1, model.config.↩
    ↪ hidden_size),
        checked_size(model.config.hidden_size, "hidden_size"));
    layer_norm(final_hidden,
               model.final_ln_weight,
               model.final_ln_bias,
               model.config.layer_norm_epsilon,
               normed);

    Gpt2ForwardResult result;
    result.logits.assign(checked_size(model.config.vocab_size, "vocab_size"), ↩
    ↪ 0.0F);
    compute_logits(model, normed, result.logits);
    result.predicted_token = argmax(result.logits);
    return result;
}

Gpt2ForwardResult run_gpt2_forward_cached(const Gpt2ReferenceModel& model,
                                          Gpt2KvCache& cache,
                                          std::int32_t token_id) {
    validate_model(model);
    Gpt2ForwardWorkspace workspace(model.config);
    Gpt2ExecutionOptions options;
    options.worker_count = 1;
    return run_gpt2_cached_step_unchecked(
        model,
```

```
        cache,
        workspace,
        token_id,
        true,
        nullptr,
        options,
        nullptr);
}

std::vector<std::int32_t> gpt2_generate_greedy(const Gpt2ReferenceModel& model,
                                               std::span<const std::int32_t> prompt_token_ids,
                                               std::int32_t max_new_tokens) {
    if (max_new_tokens < 0) {
        throw std::runtime_error("max_new_tokens cannot be negative");
    }
    std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids.end());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        const Gpt2ForwardResult result = run_gpt2_forward(model, tokens);
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
        }
    }
    return tokens;
}

std::vector<std::int32_t> gpt2_generate_greedy_cached(
    const Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_token_ids,
    std::int32_t max_new_tokens) {
    if (prompt_token_ids.empty()) {
        throw std::runtime_error("GPT-2 generation needs at least one prompt token");
    }
    if (max_new_tokens < 0) {
        throw std::runtime_error("max_new_tokens cannot be negative");
    }
    if (prompt_token_ids.size() > checked_size(model.config.max_positions, "max_positions")) {
        throw std::runtime_error("GPT-2 prompt exceeds max_positions");
    }

    Gpt2CachedGreedyDecoder decoder(model);
    std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids
```

```
        ↪ .end());
    if (max_new_tokens == 0) {
        return tokens;
    }
    std::int32_t predicted_token = 0;
    for (const std::int32_t token_id : prompt_token_ids) {
        predicted_token = decoder.step(token_id);
    }
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "↩
        ↪ max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed ↩
        ↪ max_positions");
        }
        tokens.push_back(predicted_token);
        if (model.config.eos_token_id >= 0 && predicted_token == model.config.↩
        ↪ eos_token_id) {
            break;
        }
        if (step + 1 < max_new_tokens) {
            predicted_token = decoder.step(predicted_token);
        }
    }
    return tokens;
}

Gpt2ReferenceModel make_tiny_gpt2_reference_model() {
    Gpt2ReferenceModel model;
    model.config.hidden_size = 4;
    model.config.head_count = 2;
    model.config.layer_count = 1;
    model.config.max_positions = 8;
    model.config.vocab_size = 6;
    model.config.intermediate_size = 8;
    model.config.bos_token_id = 0;
    model.config.eos_token_id = 5;
    model.config.layer_norm_epsilon = 1.0e-5F;
    model.config.activation_function = "gelu_new";

    model.token_embedding = {
        0.20F, -0.10F, 0.00F, 0.10F,
        -0.10F, 0.30F, 0.20F, -0.20F,
        0.05F, 0.10F, -0.30F, 0.25F,
        0.40F, -0.20F, 0.15F, 0.05F,
        -0.30F, 0.05F, 0.35F, -0.15F,
        0.10F, 0.20F, -0.10F, 0.30F,
    };
    model.position_embedding = {
        0.01F, 0.02F, 0.03F, 0.04F,
        -0.02F, 0.01F, 0.00F, 0.03F,
        0.03F, -0.01F, 0.02F, 0.00F,
        0.00F, 0.02F, -0.02F, 0.01F,
```

```
        0.02F, 0.00F, 0.01F, -0.01F,
        -0.01F, 0.03F, 0.00F, 0.02F,
        0.01F, -0.02F, 0.03F, 0.00F,
        0.00F, 0.01F, -0.01F, 0.02F,
    };

    Gpt2LayerWeightsF32 layer;
    layer.ln_1_weight = ones(4);
    layer.ln_1_bias = zeros(4);
    layer.ln_2_weight = {0.9F, 1.1F, 1.0F, 0.95F};
    layer.ln_2_bias = {0.01F, -0.02F, 0.00F, 0.03F};
    layer.c_attn = make_linear(
        4,
        12,
        {
            0.10F, -0.20F, 0.05F, 0.15F,
            -0.05F, 0.12F, 0.20F, -0.10F,
            0.18F, 0.02F, -0.08F, 0.11F,
            -0.12F, 0.06F, 0.14F, 0.04F,
            0.07F, 0.16F, -0.09F, 0.03F,
            -0.15F, 0.05F, 0.13F, 0.10F,
            0.04F, -0.11F, 0.19F, -0.02F,
            0.09F, 0.08F, -0.06F, 0.17F,
            0.20F, -0.04F, 0.01F, 0.06F,
            -0.08F, 0.15F, 0.07F, -0.03F,
            0.05F, 0.10F, -0.14F, 0.12F,
            0.11F, -0.07F, 0.16F, 0.02F,
        },
        {
            0.01F, -0.02F, 0.00F, 0.03F,
            -0.01F, 0.02F, 0.01F, -0.02F,
            0.00F, 0.01F, -0.01F, 0.02F,
        });
    layer.c_proj = make_linear(
        4,
        4,
        {
            0.10F, 0.05F, -0.08F, 0.12F,
            -0.04F, 0.11F, 0.09F, -0.02F,
            0.07F, -0.10F, 0.13F, 0.05F,
            0.02F, 0.14F, -0.06F, 0.08F,
        },
        {0.01F, 0.00F, -0.01F, 0.02F});
    layer.c_fc = make_linear(
        4,
        8,
        {
            0.20F, -0.10F, 0.05F, 0.03F,
            -0.05F, 0.14F, 0.12F, -0.08F,
            0.09F, 0.07F, -0.11F, 0.16F,
            -0.12F, 0.05F, 0.18F, 0.02F,
            0.04F, -0.09F, 0.13F, 0.10F,
```

```
            0.11F, 0.03F, -0.07F, 0.15F,
            -0.02F, 0.17F, 0.06F, -0.04F,
            0.08F, -0.06F, 0.10F, 0.12F,
        },
        {0.01F, -0.02F, 0.00F, 0.03F, -0.01F, 0.02F, 0.01F, 0.00F});
    layer.mlp_c_proj = make_linear(
        8,
        4,
        {
            0.10F, -0.08F, 0.06F, 0.12F, -0.04F, 0.07F, 0.03F, 0.11F,
            -0.05F, 0.13F, 0.02F, -0.09F, 0.15F, 0.04F, -0.03F, 0.08F,
            0.07F, 0.01F, -0.12F, 0.10F, 0.06F, -0.05F, 0.14F, 0.02F,
            0.03F, 0.09F, 0.11F, -0.06F, 0.05F, 0.12F, -0.08F, 0.04F,
        },
        {0.00F, 0.01F, -0.01F, 0.02F});
    model.layers.push_back(std::move(layer));
    model.final_ln_weight = {1.0F, 0.95F, 1.05F, 1.0F};
    model.final_ln_bias = {0.0F, 0.01F, -0.01F, 0.02F};
    model.tie_lm_head_to_embedding = true;
    validate_model(model);
    return model;
}

}  // namespace lcqi
```

`gpt2_cli.cpp` 负责把模型目录、prompt、`max_new_tokens` 和 benchmark 选项接到 GPT-2 reference path。无参数时运行 tiny GPT-2 fixture；传入 HuggingFace 模型目录时加载真实资产。

Listing 10.12 src/gpt2_cli.cpp

```
#include <lcqi/gpt2_reference.hpp>

#include <chrono>
#include <cstdlib>
#include <exception>
#include <filesystem>
#include <iostream>
#include <span>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

namespace {

constexpr int LCQI_GPT2_TINY_ARGC = 1;
constexpr int LCQI_GPT2_MIN_REAL_ARGC = 3;
constexpr std::int32_t LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS = 8;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_0 = 1;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_1 = 2;
constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_2 = 3;

enum class Gpt2EngineMode {
```

```
    full_prefix,
    cached_kv,
};

struct Gpt2CliOptions {
    Gpt2EngineMode engine = Gpt2EngineMode::cached_kv;
    bool benchmark = false;
    bool profile_hotspots = false;
    std::filesystem::path model_dir;
    std::string prompt;
    std::int32_t max_new_tokens = LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS;
    std::int32_t worker_count = 0;
};

struct Gpt2BenchmarkTimings {
    double load_ms = 0.0;
    double tokenize_ms = 0.0;
    double prefill_ms = 0.0;
    double decode_ms = 0.0;
    double generate_ms = 0.0;
    double total_ms = 0.0;
    std::size_t prefill_steps = 0;
    std::size_t decode_steps = 0;
    std::size_t kv_cache_bytes = 0;
    std::int32_t worker_count = 1;
};

using Clock = std::chrono::steady_clock;

void print_ids(std::span<const std::int32_t> ids) {
    for (std::size_t index = 0; index < ids.size(); ++index) {
        if (index != 0) {
            std::cout << ' ';
        }
        std::cout << ids[index];
    }
}

[[nodiscard]] double elapsed_ms(Clock::time_point start, Clock::time_point end)↩
    ↪ {
    return std::chrono::duration<double, std::milli>(end - start).count();
}

[[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return static_cast<std::size_t>(value);
}

[[nodiscard]] const char* engine_name(Gpt2EngineMode engine) {
    switch (engine) {
```

```
        case Gpt2EngineMode::full_prefix:
            return "full_prefix";
        case Gpt2EngineMode::cached_kv:
            return "cached_kv";
    }
    return "unknown";
}

[[nodiscard]] Gpt2EngineMode parse_engine(std::string_view value) {
    if (value == "full" || value == "full_prefix") {
        return Gpt2EngineMode::full_prefix;
    }
    if (value == "cached" || value == "cached_kv") {
        return Gpt2EngineMode::cached_kv;
    }
    throw std::runtime_error("unknown --engine value, expected cached or full");
}

[[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
                                                  const char* name) {
    const std::int32_t value = static_cast<std::int32_t>(std::stoi(text));
    if (value < 0) {
        throw std::runtime_error(std::string(name) + " cannot be negative");
    }
    return value;
}

[[nodiscard]] Gpt2CliOptions parse_options(int argc, char** argv) {
    Gpt2CliOptions options;
    std::vector<std::string> positional;
    for (int index = 1; index < argc; ++index) {
        const std::string arg = argv[index];
        if (arg == "--benchmark") {
            options.benchmark = true;
        } else if (arg == "--profile-hotspots") {
            options.profile_hotspots = true;
            options.benchmark = true;
        } else if (arg == "--engine") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--engine requires a value");
            }
            ++index;
            options.engine = parse_engine(argv[index]);
        } else if (arg.rfind("--engine=", 0) == 0) {
            options.engine = parse_engine(std::string_view(arg).substr(std::
string("--engine=").size()));
        } else if (arg == "--threads") {
            if (index + 1 >= argc) {
                throw std::runtime_error("--threads requires a value");
            }
            ++index;
```

```
            options.worker_count = parse_non_negative_i32(argv[index], "--threads");
        } else if (arg.rfind("--threads=", 0) == 0) {
            options.worker_count =
                parse_non_negative_i32(arg.substr(std::string("--threads=").size()), "--threads");
        } else if (arg == "--full") {
            options.engine = Gpt2EngineMode::full_prefix;
        } else if (arg == "--cached") {
            options.engine = Gpt2EngineMode::cached_kv;
        } else if (!arg.empty() && arg.front() == '-') {
            throw std::runtime_error("unknown option: " + arg);
        } else {
            positional.push_back(arg);
        }
    }

    if (positional.size() < 2 || positional.size() > 3) {
        throw std::runtime_error(
            "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark] "
            "model_dir prompt [max_new_tokens]");
    }
    options.model_dir = positional[0];
    options.prompt = positional[1];
    if (positional.size() == 3) {
        options.max_new_tokens = parse_non_negative_i32(positional[2], "max_new_tokens");
    }
    return options;
}

[[nodiscard]] std::vector<std::int32_t> generate_full_prefix(
    const lcqi::Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_ids,
    std::int32_t max_new_tokens,
    Gpt2BenchmarkTimings& timings) {
    const Clock::time_point generate_start = Clock::now();
    std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens);
        ++timings.decode_steps;
        tokens.push_back(result.predicted_token);
        if (model.config.eos_token_id >= 0 && result.predicted_token == model.config.eos_token_id) {
            break;
```

```
        }
    }
    const Clock::time_point generate_end = Clock::now();
    timings.generate_ms = elapsed_ms(generate_start, generate_end);
    timings.decode_ms = timings.generate_ms;
    return tokens;
}

[[nodiscard]] std::vector<std::int32_t> generate_cached(
    const lcqi::Gpt2ReferenceModel& model,
    std::span<const std::int32_t> prompt_ids,
    std::int32_t max_new_tokens,
    Gpt2BenchmarkTimings& timings,
    lcqi::Gpt2HotspotProfile* hotspot_profile,
    std::int32_t worker_count) {
    if (prompt_ids.empty()) {
        throw std::runtime_error("GPT-2 generation needs at least one prompt token");
    }
    lcqi::Gpt2ExecutionOptions execution_options;
    execution_options.worker_count = worker_count;
    lcqi::Gpt2CachedGreedyDecoder decoder(model, hotspot_profile, execution_options);
    timings.kv_cache_bytes = decoder.kv_cache_bytes();
    timings.worker_count = decoder.worker_count();
    std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
    if (max_new_tokens == 0) {
        return tokens;
    }

    const Clock::time_point generate_start = Clock::now();
    std::int32_t predicted_token = 0;
    const Clock::time_point prefill_start = Clock::now();
    for (const std::int32_t token_id : prompt_ids) {
        predicted_token = decoder.step(token_id);
        ++timings.prefill_steps;
    }
    const Clock::time_point prefill_end = Clock::now();
    timings.prefill_ms = elapsed_ms(prefill_start, prefill_end);

    const Clock::time_point decode_start = Clock::now();
    for (std::int32_t step = 0; step < max_new_tokens; ++step) {
        if (tokens.size() >= checked_size(model.config.max_positions, "max_positions")) {
            throw std::runtime_error("GPT-2 generation would exceed max_positions");
        }
        tokens.push_back(predicted_token);
        if (model.config.eos_token_id >= 0 && predicted_token == model.config.eos_token_id) {
            break;
        }
```

```
        if (step + 1 < max_new_tokens) {
            predicted_token = decoder.step(predicted_token);
            ++timings.decode_steps;
        }
    }
    const Clock::time_point decode_end = Clock::now();
    const Clock::time_point generate_end = Clock::now();
    timings.decode_ms = elapsed_ms(decode_start, decode_end);
    timings.generate_ms = elapsed_ms(generate_start, generate_end);
    return tokens;
}

void print_benchmark(const Gpt2BenchmarkTimings& timings,
                     std::size_t prompt_tokens,
                     std::size_t generated_tokens) {
    const double new_tokens = static_cast<double>(generated_tokens);
    const double generated_tokens_per_second =
        timings.generate_ms > 0.0 ? new_tokens * 1000.0 / timings.generate_ms : 0.0;
    const double decode_tokens_per_second =
        timings.decode_ms > 0.0
            ? static_cast<double>(timings.decode_steps) * 1000.0 / timings.decode_ms
            : 0.0;

    std::cout << "benchmark_load_ms " << timings.load_ms << "\n";
    std::cout << "benchmark_tokenize_ms " << timings.tokenize_ms << "\n";
    std::cout << "benchmark_prefill_ms " << timings.prefill_ms << "\n";
    std::cout << "benchmark_decode_ms " << timings.decode_ms << "\n";
    std::cout << "benchmark_generate_ms " << timings.generate_ms << "\n";
    std::cout << "benchmark_total_ms " << timings.total_ms << "\n";
    std::cout << "benchmark_prompt_tokens " << prompt_tokens << "\n";
    std::cout << "benchmark_generated_tokens " << generated_tokens << "\n";
    std::cout << "benchmark_prefill_steps " << timings.prefill_steps << "\n";
    std::cout << "benchmark_decode_steps " << timings.decode_steps << "\n";
    std::cout << "benchmark_worker_count " << timings.worker_count << "\n";
    std::cout << "benchmark_generate_tokens_per_second "
              << generated_tokens_per_second << "\n";
    std::cout << "benchmark_decode_tokens_per_second "
              << decode_tokens_per_second << "\n";
    std::cout << "benchmark_kv_cache_bytes " << timings.kv_cache_bytes << "\n";
}

void print_hotspot_profile(const lcqi::Gpt2HotspotProfile& profile) {
    const double total = profile.total_step_ms > 0.0 ? profile.total_step_ms : 1.0;
    const auto print_ms = [total](std::string_view name, double value) {
        std::cout << "hotspot_" << name << "_ms " << value << "\n";
        std::cout << "hotspot_" << name << "_pct " << (value * 100.0 / total)
    << "\n";
    };
    std::cout << "hotspot_decoder_steps " << profile.decoder_steps << "\n";
```

```
264     std::cout << "hotspot_layer_steps " << profile.layer_steps << "\n";
265     print_ms("total_step", profile.total_step_ms);
266     print_ms("embedding", profile.embedding_ms);
267     print_ms("layer_norm", profile.layer_norm_ms);
268     print_ms("final_norm", profile.final_norm_ms);
269     print_ms("qkv_projection", profile.qkv_projection_ms);
270     print_ms("kv_append", profile.kv_append_ms);
271     print_ms("attention", profile.attention_ms);
272     print_ms("attention_projection", profile.attention_projection_ms);
273     print_ms("residual_add", profile.residual_add_ms);
274     print_ms("mlp_fc", profile.mlp_fc_ms);
275     print_ms("gelu", profile.gelu_ms);
276     print_ms("mlp_projection", profile.mlp_projection_ms);
277     print_ms("lm_head", profile.lm_head_ms);
278     print_ms("logits_result", profile.logits_result_ms);
279 }
280
281 int run_tiny_mode() {
282     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model↩
    ↪ ();
283     const std::vector<std::int32_t> prompt{
284         LCQI_GPT2_TINY_PROMPT_0,
285         LCQI_GPT2_TINY_PROMPT_1,
286         LCQI_GPT2_TINY_PROMPT_2,
287     };
288     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, prompt↩
    ↪ );
289     const std::vector<std::int32_t> generated =
290         lcqi::gpt2_generate_greedy(model, prompt, 2);
291     std::cout << "mode tiny\n";
292     std::cout << "prompt_ids ";
293     print_ids(prompt);
294     std::cout << "\n";
295     std::cout << "predicted_token " << result.predicted_token << "\n";
296     std::cout << "logits";
297     for (const float value : result.logits) {
298         std::cout << ' ' << value;
299     }
300     std::cout << "\n";
301     std::cout << "generated_ids ";
302     print_ids(generated);
303     std::cout << "\n";
304     return EXIT_SUCCESS;
305 }
306
307 int run_real_model_mode(int argc, char** argv) {
308     if (argc < LCQI_GPT2_MIN_REAL_ARGC) {
309         throw std::runtime_error(
310             "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark↩
    ↪ ] "
311             "model_dir prompt [max_new_tokens]");
312     }
```

```
313     const Clock::time_point total_start = Clock::now();
314     const Gpt2CliOptions options = parse_options(argc, argv);
315 
316     Gpt2BenchmarkTimings timings;
317     const Clock::time_point load_start = Clock::now();
318     const lcqi::Gpt2ReferenceModel model = lcqi::load_gpt2_from_directory(↩
    ↪ options.model_dir);
319     const lcqi::Gpt2Tokenizer tokenizer = lcqi::load_gpt2_tokenizer(
320         options.model_dir / "vocab.json",
321         options.model_dir / "merges.txt",
322         model.config.bos_token_id,
323         model.config.eos_token_id);
324     timings.load_ms = elapsed_ms(load_start, Clock::now());
325 
326     const Clock::time_point tokenize_start = Clock::now();
327     const std::vector<std::int32_t> prompt_ids = lcqi::gpt2_encode(tokenizer, ↩
    ↪ options.prompt);
328     timings.tokenize_ms = elapsed_ms(tokenize_start, Clock::now());
329 
330     std::vector<std::int32_t> generated;
331     lcqi::Gpt2HotspotProfile hotspot_profile;
332     if (options.engine == Gpt2EngineMode::full_prefix) {
333         if (options.profile_hotspots) {
334             throw std::runtime_error("--profile-hotspots currently supports ↩
    ↪ cached engine only");
335         }
336         generated = generate_full_prefix(model, prompt_ids, options.↩
    ↪ max_new_tokens, timings);
337     } else {
338         generated = generate_cached(model,
339                                     prompt_ids,
340                                     options.max_new_tokens,
341                                     timings,
342                                     options.profile_hotspots ? &hotspot_profile↩
    ↪  : nullptr,
343                                     options.worker_count);
344     }
345     timings.total_ms = elapsed_ms(total_start, Clock::now());
346 
347     std::cout << "mode gpt2_directory\n";
348     std::cout << "engine " << engine_name(options.engine) << "\n";
349     std::cout << "model_dir " << options.model_dir.string() << "\n";
350     std::cout << "prompt_ids ";
351     print_ids(prompt_ids);
352     std::cout << "\n";
353     std::cout << "generated_ids ";
354     print_ids(generated);
355     std::cout << "\n";
356     std::cout << "generated_text " << lcqi::gpt2_decode(tokenizer, generated) ↩
    ↪ << "\n";
357     if (options.benchmark) {
358         const std::size_t generated_tokens = generated.size() - prompt_ids.size↩
```

```
    ↪ ();
359         print_benchmark(timings, prompt_ids.size(), generated_tokens);
360     }
361     if (options.profile_hotspots) {
362         print_hotspot_profile(hotspot_profile);
363     }
364     return EXIT_SUCCESS;
365 }
366
367 }  // namespace
368
369 int main(int argc, char** argv) {
370     try {
371         if (argc == LCQI_GPT2_TINY_ARGC) {
372             return run_tiny_mode();
373         }
374         return run_real_model_mode(argc, argv);
375     } catch (const std::exception& error) {
376         std::cerr << "error: " << error.what() << "\n";
377         return EXIT_FAILURE;
378     }
379 }
```

第 11 章实践：运行时内存计划、liveness 与 Workspace

本章对应原书中的第三册“推理运行时与内存规划”。不要把它当作概念复述，本章要把 activation arena、workspace、liveness、state edge、debug dump、buffer reuse 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `layout`，核心命令或测试入口是 `lcqi_trace`。

Listing 11.1 第 11 章实践：运行时内存计划、liveness 与 Workspace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / layout
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：比较普通 activation 和 KV cache 的生命周期。读者要先手写预期结果，再运行项目观察输出。动作是：说明为什么 KV cache 不能被 activation planner 当作临时缓冲复用。如果输出里没有 `checksum` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：内存复用只在生命周期不重叠时成立；debug/oracle/materialization 会延长生命周期这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 liveness 表：每个 checkpoint 的 `defined_at`、`last_used_at`、是否 state、是否允许复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 12 章实践：IR、Pass、Lowering 与 Executable Plan

本章对应原书中的第三册“AI 编译器细化：IR、pass、lowering 与 executable plan”。不要把它当作概念复述，本章要把 typed graph、pass、fusion、layout propagation、backend capability、fallback reason 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `run_inference`，核心命令或测试入口是 `lcqi_cli`。

Listing 12.1 第 12 章实践：IR、Pass、Lowering 与 Executable Plan 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / run_inference
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 tiny MLP 写成三节点 IR：linear、relu、linear。读者要先手写预期结果，再运行项目观察输出。动作是：说明哪些 pass 可以融合，哪些必须保留为 oracle checkpoint。如果输出里没有 `backend` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：IR 的价值不是画图，而是让 shape、layout、量化、后端选择和 fallback 诊断有稳定承载物这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 executable plan 草图：节点、输入输出 tensor、layout、kernel、fallback reason 和 profiler event。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

后端优化、工业专项与验收

第 13 章实践：CPU 后端、微内核与性能证据

本章对应原书中的第三册“CPU 后端优化细讲”和“CPU decode 实战”。不要把它当作概念复述，本章要把 scalar baseline、packed weight、AVX2、objdump、perf boundary、shape sweep 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `avx2`，核心命令或测试入口是 `lcqi_bench`。

Listing 13.1 第 13 章实践：CPU 后端、微内核与性能证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / avx2
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：对同一 kernel 保存 benchmark CSV 和反汇编摘要。读者要先手写预期结果，再运行项目观察输出。动作是：解释速度差来自地址流、SIMD lane、packing、load 或 reduction 哪一项。如果输出里没有 `average_us` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：CPU 后端优化必须同时有 oracle、反汇编、shape sweep 和降级边界；只有耗时表不够这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 CPU kernel evidence：命令、机器、编译器、输入 shape、backend、checksum、反汇编摘要和未验证 PMU 边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 14 章实践：GPU 后端边界、copy 与同步成本

本章对应原书中的第三册“GPU 硬件基础”和“GPU 后端优化细讲”。不要把它当作概念复述，本章要把 device buffer、stream、event、kernel launch、copy、sync、fallback 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `StepTrace`，核心命令或测试入口是 `lcqi_trace`。

Listing 14.1 第 14 章实践：GPU 后端边界、copy 与同步成本的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / StepTrace
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：当前项目没有 GPU kernel，也要写清 adapter 合同。读者要先手写预期结果，再运行项目观察输出。动作是：把 CPU trace 字段扩展成 GPU StepTrace 需要哪些字段。如果输出里没有 `copy_time_ns` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：GPU 快不快不能只看 kernel；copy、sync、sampler、fallback 和 batch wait 必须进入端到端 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 GPU adapter 设计：buffer owner、stream、event、H2D/D2H 字节、sync 点、fallback 和 oracle 对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 15 章实践：工业 LLM 专项能力对照

本章对应原书中的第三册“工业 LLM 专项：投机解码、MoE、LoRA、多 GPU 与源码对照”。不要把它当作概念复述，本章要把 speculative decoding、MoE routing、LoRA adapter、multi-GPU、source reading 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `bandwidth_limit`，核心命令或测试入口是 `lcqi_ledger`。

Listing 15.1 第 15 章实践：工业 LLM 专项能力对照的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_ledger / bandwidth_limit
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：选择一个专项，不写泛泛介绍。读者要先手写预期结果，再运行项目观察输出。动作是：把它接到现有 trace、memory、backend 或 serving 字段上。如果输出里没有 `tokens/s` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：工业专项不是功能清单；必须说明它购买什么能力、增加什么状态、破坏哪些假设这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交专项设计卡：动机、状态新增、trace 字段、回归测试、性能账本和不能验证的部分。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 16 章实践：工程验收、回归门禁与最终项目

本章对应原书中的第三册“工程验收与回归体系”。不要把它当作概念复述，本章要把 reference、unit test、trace golden、benchmark gate、SLO gate、coverage、CI 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `lcqi_tests`，核心命令或测试入口是 `ctest`。

Listing 16.1 第 16 章实践：工程验收、回归门禁与最终项目的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↩
    ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: ctest / lcqi_tests
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把所有前面报告合成最终验收包。读者要先手写预期结果，再运行项目观察输出。动作是：运行 `ctest` 并说明每个 test 保护哪条合同。如果输出里没有 `100% tests passed` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：实践卷最终目标是长期维护：结果正确、trace 稳定、性能可解释、服务指标可回归、失败边界不伪装这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交最终项目：README、命令记录、报告索引、测试结果、benchmark 表、trace 摘要、风险清单和下一阶段计划。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：工具、Smoke、Benchmark 与回归测试

工具不是附属品

LCQI 的工程证据不只在 C++ 源码里。smoke 脚本决定真实模型怎样下载、校验和运行；benchmark 程序决定性能口径；trace、ledger 和 serving probe 决定报告字段；CTest 决定哪些行为能长期回归。读者必须把这些文件当作工程合同的一部分。

Benchmark、Trace、Ledger 与 Serving Probe

`bench.cpp` 输出 int8 kernel benchmark。它服务性能观察，但必须和 max diff 对照一起看。

Listing 16.2 src/bench.cpp

```cpp
#include <lcqi/int8_kernels.hpp>

#include <algorithm>
#include <cstdlib>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

namespace {

constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
constexpr int LCQI_REPEAT_ARG_INDEX = 1;
constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;

const char* target_arch() noexcept {
#if defined(__x86_64__) || defined(_M_X64)
    return LCQI_TARGET_ARCH_X86_64;
#elif defined(__i386__) || defined(_M_IX86)
    return LCQI_TARGET_ARCH_I386;
#else
    return LCQI_TARGET_ARCH_UNKNOWN;
#endif
}

std::int32_t parse_repeat(int argc, char** argv) {
    if (argc <= LCQI_REPEAT_ARG_INDEX) {
        return LCQI_DEFAULT_REPEAT;
    }
    const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[
    ↪ LCQI_REPEAT_ARG_INDEX]));
    if (repeat <= 0) {
```

```
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
    return {
        {128, 128, 16, repeat},
        {256, 256, 16, repeat},
        {512, 512, 16, repeat},
        {513, 257, 16, repeat},
        {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
                                  repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
        {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
         std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
    };
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::cout
            << "target_arch,input_size,output_size,output_block,repeat,backend,available,"
               "average_us,max_abs_diff,checksum\n";
        for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(repeat)) {
            const lcqi::KernelBenchmarkResult result =
                lcqi::benchmark_linear_i8_case(benchmark_case);
            for (const lcqi::KernelBackendBenchmark& backend : result.backends) {
                std::cout << target_arch() << ','
                          << result.benchmark_case.input_size << ','
                          << result.benchmark_case.output_size << ','
                          << result.benchmark_case.output_block_size << ','
                          << result.benchmark_case.repeat << ','
                          << backend.name << ','
                          << (backend.available ? 1 : 0) << ','
                          << backend.average_us << ','
                          << backend.max_abs_diff << ','
                          << backend.checksum << '\n';
            }
        }
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

`trace.cpp` 输出 reference decoder checkpoint。它用于定位错误边界，不应该被简化成只打印最终 token。

Listing 16.3 src/trace.cpp

```cpp
#include <lcqi/reference_decoder.hpp>
#include <lcqi/tokenizer.hpp>

#include <algorithm>
#include <array>
#include <cstdlib>
#include <exception>
#include <filesystem>
#include <iostream>
#include <span>
#include <sstream>
#include <string>
#include <string_view>

namespace {

constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;

std::filesystem::path tokenizer_fixture_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_tokenizer.txt";
}

std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel& tokenizer,
                                                     std::string_view prompt) {
    std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer, prompt);
    if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
        full_tokens.front() != LCQI_TRACE_BOS_ID ||
        full_tokens.back() != LCQI_TRACE_EOS_ID) {
        throw std::runtime_error("unexpected tokenizer fixture ids");
    }

    std::vector<std::int32_t> decoder_tokens(
        full_tokens.begin() + 1,
        full_tokens.end() - 1);
    if (!std::equal(decoder_tokens.begin(),
                    decoder_tokens.end(),
                    LCQI_TRACE_TOKEN_IDS.begin(),
                    LCQI_TRACE_TOKEN_IDS.end())) {
        throw std::runtime_error("unexpected decoder token ids");
    }
```

```
    return decoder_tokens;
}

std::string join_ints(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (index != 0) {
            stream << 'x';
        }
        stream << values[index];
    }
    return stream.str();
}

std::string join_int_values(std::span<const std::int32_t> values) {
    std::ostringstream stream;
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (index != 0) {
            stream << ';';
        }
        stream << values[index];
    }
    return stream.str();
}

std::int32_t checksum_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result += value;
    }
    return result;
}

std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
    std::int32_t result = 0;
    for (const std::int32_t value : values) {
        result = std::max(result, std::abs(value));
    }
    return result;
}

std::string join_floats(std::span<const float> values) {
    std::ostringstream stream;
    const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
    for (std::size_t index = 0; index < count; ++index) {
        if (index != 0) {
            stream << ';';
        }
        stream << values[index];
    }
    if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
        stream << ";...";
```

```
    }
    return stream.str();
}

void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
                     const lcqi::TokenizerModel& tokenizer,
                     std::span<const std::int32_t> full_tokens) {
    std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values\n";
    std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
    std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
              << ',' << full_tokens.size() << ",1,i32,contiguous,"
              << checksum_ints(full_tokens) << ','
              << max_abs_ints(full_tokens) << ','
              << join_int_values(full_tokens) << '\n';
    std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,u64,fnv1a,"
              << tokenizer_contract_hash(tokenizer) << ",0,"
              << tokenizer.chat_template_hash << '\n';
    for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints) {
        std::cout << checkpoint.name << ','
                  << checkpoint.token_position << ','
                  << checkpoint.layer_id << ','
                  << join_ints(checkpoint.shape) << ','
                  << join_ints(checkpoint.stride) << ','
                  << checkpoint.dtype << ','
                  << checkpoint.layout << ','
                  << checkpoint.checksum << ','
                  << checkpoint.max_abs << ','
                  << join_floats(checkpoint.values) << '\n';
    }
    std::cout << "sampler_argmax,"
              << LCQI_TRACE_TOKEN_IDS.size() - 1 << ','
              << LCQI_TRACE_METADATA_LAYER
              << ',' << trace.result.logits.size() << ",1,f32,argmax,"
              << "0,0,token=" << trace.result.predicted_token << '\n';
    std::cout << "generated_token,"
              << LCQI_TRACE_TOKEN_IDS.size() << ','
              << LCQI_TRACE_METADATA_LAYER
              << ",scalar,scalar,i32,generated,"
              << trace.result.predicted_token << ','
              << trace.result.predicted_token << ','
              << trace.result.predicted_token << '\n';
}

}  // namespace

int main() {
    try {
        const lcqi::TokenizerModel tokenizer =
```

```
            lcqi::load_tokenizer(tokenizer_fixture_path());
        const std::vector<std::int32_t> full_tokens =
            lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
        const std::vector<std::int32_t> decoder_tokens =
            decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
        const lcqi::ReferenceDecoderModel model = lcqi::↩
    ↪ make_tiny_reference_decoder_model();
        const lcqi::ReferenceDecodeTraceResult trace =
            lcqi::run_reference_decode_with_trace(model, decoder_tokens);
        print_trace_csv(trace, tokenizer, full_tokens);
        return EXIT_SUCCESS;
    } catch (const std::exception& error) {
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

`ledger.cpp` 输出 7B 规模账本。它是估算，不是实测吞吐；读者要区分 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限。

Listing 16.4 src/ledger.cpp

```
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <iomanip>
#include <iostream>
#include <string_view>

namespace {

constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
constexpr double LCQI_FLOPS_PER_MAC = 2.0;
constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;

struct DTypeLedger {
    std::string_view name;
    double weight_gb_per_token = 0.0;
    double kv_element_bytes = 0.0;
};

constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
    {"fp16", 14.0, 2.0},
    {"int8", 7.0, 2.0},
    {"int4", 3.7, 2.0},
};
```

```cpp

constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
    50.0,
    100.0,
    200.0,
    600.0,
    1000.0,
    1600.0,
};

double linear_macs_per_layer() noexcept {
    const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
    const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
    const double qkv = hidden * (hidden + 2.0 * kv_width);
    const double output_projection = hidden * hidden;
    const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
    return qkv + output_projection + mlp;
}

double attention_macs_per_layer() noexcept {
    return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
           LCQI_SEVENB_HEAD_DIM;
}

double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
    return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
}

double kv_read_gb_per_decode_token(double element_bytes) noexcept {
    const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_GB;
}

double kv_capacity_mib(double element_bytes, std::int32_t context_length) ↩
    ↪ noexcept {
    const double bytes = static_cast<double>(context_length) *
                         kv_bytes_per_token_all_layers(element_bytes);
    return bytes / LCQI_BYTES_PER_MIB;
}

void print_model_rows() {
    const double linear_macs = linear_macs_per_layer();
    const double attention_macs = attention_macs_per_layer();
    const double total_flops =
        (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * ↩
    ↪ LCQI_FLOPS_PER_MAC;
    std::cout << "section,item,value,unit,notes\n";
    std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B ↩
    ↪ fixture\n";
    std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
```

```
81      std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,GQA↩
    ↪  query heads\n";
82      std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,GQA KV ↩
    ↪ heads\n";
83      std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per ↩
    ↪ head\n";
84      std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs,↩
    ↪ decode token\n";
85      std::cout << "compute,attention_macs_per_layer," << attention_macs
86                << ",macs,context=4096\n";
87      std::cout << "compute,total_decode_flops_per_token," << total_flops
88                << ",flops,linear plus attention\n";
89  }
90
91  void print_kv_rows() {
92      for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
93          std::cout << "kv,fp16_capacity_context_" << context << ','
94                    << kv_capacity_mib(2.0, context)
95                    << ",MiB,single session\n";
96      }
97  }
98
99  void print_dtype_rows() {
100     for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
101         const double kv_read_gb = kv_read_gb_per_decode_token(dtype.↩
    ↪ kv_element_bytes);
102         const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
103         std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
104                   << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
105         std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
106                   << kv_read_gb << ",GB/token,context=4096\n";
107         std::cout << "decode_bytes," << dtype.name << "_total_gb,"
108                   << total_gb << ",GB/token,weight plus KV\n";
109         for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
110             std::cout << "bandwidth_limit," << dtype.name << "_at_"
111                       << static_cast<std::int32_t>(bandwidth) << "_gbps,"
112                       << bandwidth / total_gb
113                       << ",tokens/s,bandwidth-only upper bound\n";
114         }
115     }
116 }
117
118 }  // namespace
119
120 int main() {
121     try {
122         std::cout << std::fixed << std::setprecision(3);
123         print_model_rows();
124         print_kv_rows();
125         print_dtype_rows();
126         return EXIT_SUCCESS;
127     } catch (const std::exception& error) {
```

```
        std::cerr << "error: " << error.what() << '\n';
        return EXIT_FAILURE;
    }
}
```

serving_probe.cpp 固定短请求、RAG 请求、取消请求和超长请求。它把 TTFT、TPOT、拒绝和取消回收变成可观察字段。

Listing 16.5 src/serving_probe.cpp

```
#include <algorithm>
#include <cstdlib>
#include <cstdint>
#include <exception>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <string_view>
#include <vector>

namespace {

constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;

struct ProbeRequest {
    std::string_view request_id;
    std::int32_t prompt_tokens = 0;
    std::int32_t max_new_tokens = 0;
    bool cancel_after_prefill = false;
};

struct ProbeResult {
    std::string_view request_id;
    std::int32_t accepted = 0;
    std::string_view rejection_reason;
    std::int32_t prompt_tokens = 0;
    std::int32_t reserved_tokens = 0;
    double kv_reserved_mib = 0.0;
    double queue_wait_ms = 0.0;
    double prefill_ms = 0.0;
```

```
    double ttft_ms = 0.0;
    double decode_step_p95_ms = 0.0;
    double tpot_ms = 0.0;
    std::int32_t completion_tokens = 0;
    std::string_view finish_reason;
    double cancel_reclaim_ms = 0.0;
};

const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
    {"req_short", 32, 4, false},
    {"req_rag", 512, 8, false},
    {"req_cancel", 128, 16, true},
    {"req_too_long", 4096, 512, false},
};

double kv_mib_for_tokens(std::int32_t tokens) noexcept {
    const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT * 2.0 *
                         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
                         LCQI_PROBE_KV_ELEMENT_BYTES;
    return bytes / LCQI_BYTES_PER_MIB;
}

double percentile(std::vector<double> values, double fraction) {
    if (values.empty()) {
        return 0.0;
    }
    std::sort(values.begin(), values.end());
    const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
    const std::size_t index =
        std::min(max_index,
                 static_cast<std::size_t>(
                     std::ceil(fraction * static_cast<double>(max_index))));
    return values[index];
}

std::vector<ProbeResult> run_probe() {
    std::vector<ProbeResult> results;
    double reserved_kv_mib = 0.0;
    std::int32_t accepted_before = 0;
    for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
        ProbeResult result;
        result.request_id = request.request_id;
        result.prompt_tokens = request.prompt_tokens;
        result.reserved_tokens = request.prompt_tokens + request.max_new_tokens;
        result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);
        const double static_session_mib =
            result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
            LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
        if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB) {
```

```
            result.accepted = 0;
            result.rejection_reason = "memory_budget_exceeded";
            result.finish_reason = "rejected";
            results.push_back(result);
            continue;
        }

        reserved_kv_mib += result.kv_reserved_mib;
        result.accepted = 1;
        result.rejection_reason = "none";
        result.queue_wait_ms = static_cast<double>(accepted_before) *
                               LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
        result.prefill_ms = static_cast<double>(request.prompt_tokens) *
                            LCQI_PROBE_PREFILL_MS_PER_TOKEN;
        result.ttft_ms = result.queue_wait_ms + result.prefill_ms + LCQI_PROBE_DECODE_STEP_MS;
        result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
                                    static_session_mib / 512.0;
        result.tpot_ms = result.decode_step_p95_ms;
        if (request.cancel_after_prefill) {
            result.completion_tokens = 0;
            result.finish_reason = "cancelled";
            result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
            reserved_kv_mib -= result.kv_reserved_mib;
        } else {
            result.completion_tokens = request.max_new_tokens;
            result.finish_reason = "max_tokens";
        }
        ++accepted_before;
        results.push_back(result);
    }
    return results;
}

void print_request_rows(const std::vector<ProbeResult>& results) {
    std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,"
                 "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,"
                 "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,"
                 "metric_value\n";
    for (const ProbeResult& result : results) {
        std::cout << "request," << result.request_id << ','
                  << result.accepted << ','
                  << result.rejection_reason << ','
                  << result.prompt_tokens << ','
                  << result.reserved_tokens << ','
                  << result.kv_reserved_mib << ','
                  << result.queue_wait_ms << ','
                  << result.prefill_ms << ','
                  << result.ttft_ms << ','
```

```
                  << result.decode_step_p95_ms << ','
                  << result.tpot_ms << ','
                  << result.completion_tokens << ','
                  << result.finish_reason << ','
                  << result.cancel_reclaim_ms << ",,\n";
    }
}

void print_summary_row(const std::vector<ProbeResult>& results) {
    std::vector<double> ttft_values;
    std::vector<double> queue_values;
    double accepted = 0.0;
    double rejected = 0.0;
    double cancelled = 0.0;
    double reserved_peak_mib = 0.0;
    double running_reserved_mib = 0.0;
    for (const ProbeResult& result : results) {
        if (result.accepted == 0) {
            rejected += 1.0;
            continue;
        }
        accepted += 1.0;
        ttft_values.push_back(result.ttft_ms);
        queue_values.push_back(result.queue_wait_ms);
        running_reserved_mib += result.kv_reserved_mib;
        reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
        if (result.finish_reason == "cancelled") {
            cancelled += 1.0;
            running_reserved_mib -= result.kv_reserved_mib;
        }
    }
    const double total = accepted + rejected;
    const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "accepted_count," << accepted << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
              << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled
    ↪ << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejected_count," << rejected << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "rejection_rate," << rejection_rate << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n'
    ↪ ;
    std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
              << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
}

}  // namespace
```

```
191
192 int main() {
193     try {
194         std::cout << std::fixed << std::setprecision(3);
195         const std::vector<ProbeResult> results = run_probe();
196         print_request_rows(results);
197         print_summary_row(results);
198         return EXIT_SUCCESS;
199     } catch (const std::exception& error) {
200         std::cerr << "error: " << error.what() << '\n';
201         return EXIT_FAILURE;
202     }
203 }
```

onednn_bench.cpp 是可选外部库对标入口。它只在环境安装 oneDNN 时构建，不能把不可用平台写成已验证性能。

Listing 16.6 src/onednn_bench.cpp

```
1  #include <dnnl.hpp>
2
3  #include <chrono>
4  #include <cstdint>
5  #include <cstdlib>
6  #include <iostream>
7  #include <numeric>
8  #include <stdexcept>
9  #include <string>
10 #include <unordered_map>
11 #include <vector>
12
13 namespace {
14
15 constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
16 constexpr std::int64_t LCQI_ONEDNN_K = 4096;
17 constexpr std::int64_t LCQI_ONEDNN_O = 4096;
18 constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
19 constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
20 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
21 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
22 constexpr float LCQI_INPUT_SCALE = 0.125F;
23 constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
24 constexpr std::int32_t LCQI_INPUT_CENTER = 8;
25 constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
26 constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
27 constexpr float LCQI_WEIGHT_SCALE = 0.03125F;
28
29 std::int32_t parse_repeat(int argc, char** argv) {
30     if (argc <= LCQI_REPEAT_ARG_INDEX) {
31         return LCQI_ONEDNN_DEFAULT_REPEAT;
32     }
33     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↩
    ↪ LCQI_REPEAT_ARG_INDEX]));
```

```
    if (repeat <= 0) {
        throw std::runtime_error("repeat must be positive");
    }
    return repeat;
}

std::vector<float> make_input() {
    std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
    for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
        input[static_cast<std::size_t>(index)] =
            static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER) *
            LCQI_INPUT_SCALE;
    }
    return input;
}

std::vector<float> make_weights_kn() {
    std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * LCQI_ONEDNN_O));
    for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
        for (std::int64_t out = 0; out < LCQI_ONEDNN_O; ++out) {
            const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
            const float value =
                static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
                                   LCQI_WEIGHT_CENTER) *
                LCQI_WEIGHT_SCALE;
            weights[static_cast<std::size_t>(k * LCQI_ONEDNN_O + out)] = value;
        }
    }
    return weights;
}

float checksum(const std::vector<float>& values) {
    return std::accumulate(values.begin(), values.end(), 0.0F);
}

}  // namespace

int main(int argc, char** argv) {
    try {
        const std::int32_t repeat = parse_repeat(argc, argv);
        std::vector<float> input = make_input();
        std::vector<float> weights = make_weights_kn();
        std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * LCQI_ONEDNN_O),
                                  0.0F);

        dnnl::engine engine(dnnl::engine::kind::cpu, 0);
        dnnl::stream stream(engine);

        const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH,
```

```
    const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_O};
    const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, LCQI_ONEDNN_O};

    const dnnl::memory::desc source_desc(
        source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);
    const dnnl::memory::desc weight_desc(
        weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);
    const dnnl::memory::desc output_desc(
        output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag::ab);

    dnnl::memory source_memory(source_desc, engine, input.data());
    dnnl::memory weight_memory(weight_desc, engine, weights.data());
    dnnl::memory output_memory(output_desc, engine, output.data());

    dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, weight_desc, output_desc);
    dnnl::matmul matmul(matmul_desc);
    const std::unordered_map<int, dnnl::memory> arguments = {
        {DNNL_ARG_SRC, source_memory},
        {DNNL_ARG_WEIGHTS, weight_memory},
        {DNNL_ARG_DST, output_memory},
    };

    for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
        matmul.execute(stream, arguments);
        stream.wait();
    }

    const auto begin = std::chrono::steady_clock::now();
    for (std::int32_t index = 0; index < repeat; ++index) {
        matmul.execute(stream, arguments);
        stream.wait();
    }
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed = end - begin;
    const double average_us =
        elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>(repeat);

    std::cout << "library,input_size,output_size,dtype,repeat,average_us,checksum\n";
    std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_O
              << ",f32," << repeat << ',' << average_us << ','
              << checksum(output) << '\n';
    return EXIT_SUCCESS;
} catch (const std::exception& error) {
    std::cerr << "error: " << error.what() << '\n';
```

```
        return EXIT_FAILURE;
    }
}
```

外部模型 Smoke 与对比脚本

`run_smollm2_small_smoke.py` 下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，构建固定 commit 的 `llama-simple-chat`，在 CPU 上运行一次真实开源 small 模型生成，并输出可复查报告。它是外部真实模型 smoke，不替代 LCQI tiny golden trace。

Listing 16.7 tools/run_smollm2_small_smoke.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import os
import re
import shlex
import shutil
import subprocess
import sys
from dataclasses import dataclass
from pathlib import Path


MODEL_SOURCE_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"
MODEL_GGUF_REPO_ID = "bartowski/SmolLM2-135M-Instruct-GGUF"
MODEL_FILE_NAME = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
MODEL_URL = (
    "https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF"
    f"/resolve/main/{MODEL_FILE_NAME}"
)
MODEL_EXPECTED_BYTES = 105454432
MODEL_EXPECTED_SHA256 = (
    "2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d"
)
LLAMA_CPP_REPO_URL = "https://github.com/ggml-org/llama.cpp.git"
LLAMA_CPP_COMMIT = "b3fed31b99f9bd37725833674252bccb429bb183"
LLAMA_TARGET = "llama-simple-chat"
DEFAULT_PROMPT = "What is 2+3? Answer in one short sentence."
DEFAULT_CONTEXT_TOKENS = 512
DEFAULT_TIMEOUT_SECONDS = 180
DEFAULT_BUILD_JOBS = 2


@dataclass(frozen=True)
class CommandResult:
    args: list[str]
    returncode: int
```

```
    stdout: str
    stderr: str


def script_path() -> Path:
    return Path(__file__).resolve()


def volume_dir() -> Path:
    return script_path().parents[1]


def repo_dir() -> Path:
    return script_path().parents[3]


def default_cache_dir() -> Path:
    explicit_cache = os.environ.get("LCQI_SMOLLM2_CACHE_DIR")
    if explicit_cache:
        return Path(explicit_cache).expanduser()
    xdg_cache = os.environ.get("XDG_CACHE_HOME")
    cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "
    ↪ .cache"
    return cache_root / "lcqi-smollm2-small-smoke"


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-smollm2-small-model-smoke.txt"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def require_tool(name: str) -> str:
    path = shutil.which(name)
    if path is None:
        raise RuntimeError(f"required tool not found in PATH: {name}")
    return path


def run_command(
    args: list[str],
    *,
    cwd: Path | None = None,
    input_text: str | None = None,
    timeout_seconds: int | None = None,
) -> CommandResult:
    try:
        completed = subprocess.run(
            args,
            cwd=str(cwd) if cwd else None,
```

```
            input=input_text,
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return CommandResult(args=args, returncode=124, stdout=stdout, stderr=stderr)

    return CommandResult(
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
    )


def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
        f"{action} failed with exit code {result.returncode}\n"
        f"command: {command_line(result.args)}\n"
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
    )


def tail(text: str, max_chars: int = 4000) -> str:
    if len(text) <= max_chars:
        return text
    return text[-max_chars:]


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def verify_model_file(path: Path) -> tuple[bool, str, int]:
    if not path.exists():
        return False, "", 0
    actual_bytes = path.stat().st_size
    actual_sha = sha256_file(path)
    return (
        actual_bytes == MODEL_EXPECTED_BYTES
```

```
143         and actual_sha == MODEL_EXPECTED_SHA256,
144         actual_sha,
145         actual_bytes,
146     )
147
148
149 def download_model(model_path: Path, *, force_download: bool, no_download: bool↩
    ↪ ) -> None:
150     model_path.parent.mkdir(parents=True, exist_ok=True)
151
152     if force_download and model_path.exists():
153         model_path.unlink()
154
155     is_valid, actual_sha, actual_bytes = verify_model_file(model_path)
156     if is_valid:
157         print(f"[lcqi-smoke] model cache hit: {model_path}")
158         return
159
160     if model_path.exists():
161         print(
162             "[lcqi-smoke] cached model hash mismatch, redownloading: "
163             f"bytes={actual_bytes}, sha256={actual_sha}"
164         )
165         model_path.unlink()
166
167     if no_download:
168         raise RuntimeError(
169             f"model file is missing or invalid and --no-download was set: {↩
    ↪ model_path}"
170         )
171
172     require_tool("curl")
173     part_path = model_path.with_suffix(model_path.suffix + ".part")
174     if part_path.exists():
175         part_path.unlink()
176
177     print(f"[lcqi-smoke] downloading small GGUF model: {MODEL_GGUF_REPO_ID}/{↩
    ↪ MODEL_FILE_NAME}")
178     result = run_command(
179         [
180             "curl",
181             "-L",
182             "--fail",
183             "--retry",
184             "3",
185             "-o",
186             str(part_path),
187             MODEL_URL,
188         ]
189     )
190     ensure_success(result, "download SmolLM2 GGUF")
191
```

```
    downloaded_ok, downloaded_sha, downloaded_bytes = verify_model_file(part_path)
    if not downloaded_ok:
        raise RuntimeError(
            "downloaded model did not match expected identity: "
            f"bytes={downloaded_bytes}, sha256={downloaded_sha}"
        )
    part_path.replace(model_path)
    print(f"[lcqi-smoke] model verified: sha256={MODEL_EXPECTED_SHA256}")


def ensure_llama_simple_chat(cache_dir: Path, *, jobs: int, skip_build: bool) -> Path:
    cached_binary = cache_dir / "llama.cpp" / "build" / "bin" / LLAMA_TARGET
    source_dir = cache_dir / "llama.cpp"
    build_dir = source_dir / "build"
    if cached_binary.exists() and os.access(cached_binary, os.X_OK) and is_pinned_llama_checkout(source_dir):
        print(f"[lcqi-smoke] llama.cpp binary cache hit: {cached_binary}")
        return cached_binary

    if skip_build:
        raise RuntimeError(f"llama.cpp binary is missing and --skip-build was set: {cached_binary}")

    require_tool("git")
    require_tool("cmake")

    cache_dir.mkdir(parents=True, exist_ok=True)

    if not (source_dir / ".git").exists():
        print(f"[lcqi-smoke] cloning llama.cpp into cache: {source_dir}")
        result = run_command(
            [
                "git",
                "clone",
                "--filter=blob:none",
                "--no-checkout",
                LLAMA_CPP_REPO_URL,
                str(source_dir),
            ]
        )
        ensure_success(result, "clone llama.cpp")

    print(f"[lcqi-smoke] checking out pinned llama.cpp commit: {LLAMA_CPP_COMMIT}")
    fetch = run_command(
        ["git", "-C", str(source_dir), "fetch", "--depth", "1", "origin", LLAMA_CPP_COMMIT]
    )
    ensure_success(fetch, "fetch pinned llama.cpp commit")
    checkout = run_command(["git", "-C", str(source_dir), "checkout", "--detach
```

```
    ↪ ", LLAMA_CPP_COMMIT])
    ensure_success(checkout, "checkout pinned llama.cpp commit")

    print(f"[lcqi-smoke] configuring llama.cpp build: {build_dir}")
    configure = run_command(
        [
            "cmake",
            "-S",
            str(source_dir),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
            "-DLLAMA_BUILD_TESTS=OFF",
            "-DLLAMA_BUILD_EXAMPLES=ON",
            "-DLLAMA_BUILD_SERVER=OFF",
            "-DGGML_NATIVE=OFF",
        ]
    )
    ensure_success(configure, "configure llama.cpp")

    print(f"[lcqi-smoke] building {LLAMA_TARGET} with -j{jobs}")
    build = run_command(
        ["cmake", "--build", str(build_dir), "--target", LLAMA_TARGET, "-j", ↩
    ↪ str(jobs)]
    )
    ensure_success(build, f"build {LLAMA_TARGET}")

    if not cached_binary.exists():
        raise RuntimeError(f"expected binary was not produced: {cached_binary}"↩
    ↪ )
    return cached_binary


def is_pinned_llama_checkout(source_dir: Path) -> bool:
    if not (source_dir / ".git").exists():
        return False
    result = run_command(["git", "-C", str(source_dir), "rev-parse", "HEAD"])
    if result.returncode != 0:
        return False
    return result.stdout.strip() == LLAMA_CPP_COMMIT


def strip_ansi(text: str) -> str:
    return re.sub(r"\x1b\[[0-?]*[ -/]*[@-~]", "", text)


def extract_response(stdout: str) -> str:
    cleaned = strip_ansi(stdout).replace("\r", "")
    chunks = [chunk.strip() for chunk in re.split(r"(?:^|\n)> ?", cleaned) if ↩
    ↪ chunk.strip()]
    if not chunks:
        return ""
```

```
    return chunks[0]


def run_model_smoke(
    llama_binary: Path,
    model_path: Path,
    *,
    prompt: str,
    context_tokens: int,
    timeout_seconds: int,
) -> tuple[CommandResult, str]:
    command = [
        str(llama_binary),
        "-m",
        str(model_path),
        "-ngl",
        "0",
        "-c",
        str(context_tokens),
    ]
    print("[lcqi-smoke] running real small model generation")
    result = run_command(
        command,
        input_text=prompt.rstrip("\n") + "\n\n",
        timeout_seconds=timeout_seconds,
    )
    ensure_success(result, "run small model smoke")

    response = extract_response(result.stdout)
    if not response:
        raise RuntimeError(
            "small model smoke exited successfully but produced no assistant text\n"
            f"stdout:\n{result.stdout}\n"
            f"stderr:\n{result.stderr}"
        )
    return result, response


def current_repo_commit() -> str:
    result = run_command(["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"])
    if result.returncode != 0:
        return "unknown"
    return result.stdout.strip()


def current_uname() -> str:
    result = run_command(["uname", "-a"])
    if result.returncode != 0:
        return "unknown"
    return result.stdout.strip()
```

```python

def write_report(
    report_path: Path,
    *,
    cache_dir: Path,
    llama_binary: Path,
    model_path: Path,
    prompt: str,
    result: CommandResult,
    response: str,
) -> None:
    report_path.parent.mkdir(parents=True, exist_ok=True)
    clean_stdout = strip_ansi(result.stdout).strip()
    report = "\n".join(
        [
            "LCQI SmolLM2 Small Model Smoke",
            "",
            f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
            f"repo_commit={current_repo_commit()}",
            f"host={current_uname()}",
            f"python={sys.version.split()[0]}",
            f"cache_dir={cache_dir}",
            "",
            f"model_source_id={MODEL_SOURCE_ID}",
            f"model_gguf_repo_id={MODEL_GGUF_REPO_ID}",
            f"model_file={MODEL_FILE_NAME}",
            f"model_url={MODEL_URL}",
            f"model_expected_bytes={MODEL_EXPECTED_BYTES}",
            f"model_expected_sha256={MODEL_EXPECTED_SHA256}",
            f"model_path={model_path}",
            "",
            f"llama_cpp_repo={LLAMA_CPP_REPO_URL}",
            f"llama_cpp_commit={LLAMA_CPP_COMMIT}",
            f"llama_target={LLAMA_TARGET}",
            f"llama_binary={llama_binary}",
            "",
            f"prompt={prompt}",
            f"command={command_line(result.args)}",
            f"exit_code={result.returncode}",
            "",
            "validation=PASS: model hash matched, llama-simple-chat exited 0, "
            "assistant response was non-empty",
            "",
            "assistant_response:",
            response,
            "",
            "stdout_clean_tail:",
            tail(clean_stdout, 2000),
            "",
            "stderr_tail:",
            tail(result.stderr.strip(), 2000),
```

```
            "",
        ]
    )
    report_path.write_text(report, encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description=(
            "Run a real local small open-source model smoke for CPU Volume 3. "
            "The script caches llama.cpp and the GGUF model outside the Git repository."
        )
    )
    parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
    parser.add_argument("--model-path", type=Path, default=None)
    parser.add_argument("--llama-bin", type=Path, default=None)
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--ctx", type=int, default=DEFAULT_CONTEXT_TOKENS)
    parser.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--force-download", action="store_true")
    parser.add_argument("--no-download", action="store_true")
    parser.add_argument("--skip-build", action="store_true")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    cache_dir = args.cache_dir.expanduser().resolve()
    model_path = (
        args.model_path.expanduser().resolve()
        if args.model_path
        else cache_dir / "models" / MODEL_FILE_NAME
    )
    report_path = args.report.expanduser().resolve()

    try:
        download_model(
            model_path,
            force_download=args.force_download,
            no_download=args.no_download,
        )
        if args.llama_bin:
            llama_binary = args.llama_bin.expanduser().resolve()
            if not llama_binary.exists() or not os.access(llama_binary, os.X_OK):
                raise RuntimeError(f"--llama-bin is not executable: {llama_binary}")
        else:
            llama_binary = ensure_llama_simple_chat(
```

```
                cache_dir,
                jobs=args.jobs,
                skip_build=args.skip_build,
            )

        result, response = run_model_smoke(
            llama_binary,
            model_path,
            prompt=args.prompt,
            context_tokens=args.ctx,
            timeout_seconds=args.timeout,
        )
        write_report(
            report_path,
            cache_dir=cache_dir,
            llama_binary=llama_binary,
            model_path=model_path,
            prompt=args.prompt,
            result=result,
            response=response,
        )
    except RuntimeError as exc:
        print(f"[lcqi-smoke] ERROR: {exc}", file=sys.stderr)
        return 1

    print(f"report={report_path}")
    print(f"model_sha256={MODEL_EXPECTED_SHA256}")
    print(f"assistant_response={response}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

run_gpt2_smoke.py 下载标准 GPT-2 的 config.json、vocab.json、merges.txt 和 model.safetensors，再调用 LCQI 自研 lcqi_gpt2 reference path 生成 token。它证明自研 safetensors、BPE、GPT-2 forward、KV cache decode 和 greedy generation 已连成闭环。

Listing 16.8 tools/run_gpt2_smoke.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import os
import shlex
import subprocess
import sys
import urllib.request
from dataclasses import dataclass
from pathlib import Path
```

```
14
15
16 MODEL_REPO_ID = "openai-community/gpt2"
17 MODEL_REVISION = "main"
18 MODEL_FILE_IDENTITIES = {
19     "config.json": (
20         665,
21         "0daed7749b4f02b8f76240d5444551d7b08712dab4d0adb8239c56ba823bb7b4",
22     ),
23     "vocab.json": (
24         1042301,
25         "196139668be63f3b5d6574427317ae82f612a97c5d1cdaf36ed2256dbf636783",
26     ),
27     "merges.txt": (
28         456318,
29         "1ce1664773c50f3e0cc8842619a93edc4624525b728b188a9e0be33b7726adc5",
30     ),
31     "model.safetensors": (
32         548105171,
33         "248dfc3911869ec493c76e65bf2fcf7f615828b0254c12b473182f0f81d3a707",
34     ),
35 }
36 DEFAULT_PROMPT = "Hello, my name is"
37 DEFAULT_MAX_NEW_TOKENS = 1
38 DEFAULT_TIMEOUT_SECONDS = 300
39 DEFAULT_BUILD_JOBS = 2
40
41
42 @dataclass(frozen=True)
43 class CommandResult:
44     args: list[str]
45     returncode: int
46     stdout: str
47     stderr: str
48
49
50 def script_path() -> Path:
51     return Path(__file__).resolve()
52
53
54 def volume_dir() -> Path:
55     return script_path().parents[1]
56
57
58 def repo_dir() -> Path:
59     return script_path().parents[3]
60
61
62 def default_cache_dir() -> Path:
63     explicit_cache = os.environ.get("LCQI_GPT2_CACHE_DIR")
64     if explicit_cache:
65         return Path(explicit_cache).expanduser()
```

```
    xdg_cache = os.environ.get("XDG_CACHE_HOME")
    cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / ".cache"
    return cache_root / "lcqi-gpt2-smoke" / MODEL_REPO_ID.replace("/", "--")


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-release"


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-smoke.txt"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def run_command(
    args: list[str],
    *,
    cwd: Path | None = None,
    timeout_seconds: int | None = None,
) -> CommandResult:
    try:
        completed = subprocess.run(
            args,
            cwd=str(cwd) if cwd else None,
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return CommandResult(args=args, returncode=124, stdout=stdout, stderr=stderr)
    return CommandResult(
        args=args,
        returncode=completed.returncode,
        stdout=completed.stdout,
        stderr=completed.stderr,
    )


def ensure_success(result: CommandResult, action: str) -> None:
    if result.returncode == 0:
        return
    raise RuntimeError(
        f"{action} failed with exit code {result.returncode}\n"
        f"command: {command_line(result.args)}\n"
```

```
        f"stdout tail:\n{tail(result.stdout)}\n"
        f"stderr tail:\n{tail(result.stderr)}"
    )


def tail(text: str, max_chars: int = 4000) -> str:
    if len(text) <= max_chars:
        return text
    return text[-max_chars:]


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def model_url(file_name: str) -> str:
    return (
        f"https://huggingface.co/{MODEL_REPO_ID}/resolve/"
        f"{MODEL_REVISION}/{file_name}"
    )


def download_file(url: str, target: Path, timeout_seconds: int) -> None:
    part_path = target.with_suffix(target.suffix + ".part")
    if part_path.exists():
        part_path.unlink()
    request = urllib.request.Request(url, headers={"User-Agent": "lcqi-gpt2-
    ↪ smoke"})
    with urllib.request.urlopen(request, timeout=timeout_seconds) as response:
        with part_path.open("wb") as output:
            while True:
                block = response.read(1024 * 1024)
                if not block:
                    break
                output.write(block)
    part_path.replace(target)


def file_identity_matches(path: Path, expected_bytes: int, expected_sha256: str
    ↪ ) -> bool:
    return (
        path.exists()
        and path.stat().st_size == expected_bytes
        and sha256_file(path) == expected_sha256
    )


def ensure_model_files(cache_dir: Path, *, no_download: bool, timeout_seconds: 
```

```
    ↪ int) -> None:
    cache_dir.mkdir(parents=True, exist_ok=True)
    missing = [
        name
        for name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.↩
    ↪ items()
        if not file_identity_matches(cache_dir / name, expected_bytes, ↩
    ↪ expected_sha256)
    ]
    if not missing:
        print(f"[lcqi-gpt2] model cache hit: {cache_dir}")
        return
    if no_download:
        raise RuntimeError(
            f"missing GPT-2 model files and --no-download was set: {missing}"
        )
    for file_name in missing:
        target = cache_dir / file_name
        if target.exists():
            target.unlink()
        print(f"[lcqi-gpt2] downloading {MODEL_REPO_ID}/{file_name}")
        download_file(model_url(file_name), target, timeout_seconds)
        expected_bytes, expected_sha256 = MODEL_FILE_IDENTITIES[file_name]
        if not file_identity_matches(target, expected_bytes, expected_sha256):
            raise RuntimeError(
                f"downloaded GPT-2 file identity mismatch: {file_name}"
            )


def build_lcqi_gpt2(build_dir: Path, jobs: int) -> Path:
    configure = run_command(
        [
            "cmake",
            "-S",
            str(volume_dir()),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
        timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
    )
    ensure_success(configure, "configure LCQI")
    build = run_command(
        [
            "cmake",
            "--build",
            str(build_dir),
            "--target",
            "lcqi_gpt2",
            "-j",
            str(jobs),
        ],
```

```
        timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
    )
    ensure_success(build, "build lcqi_gpt2")
    binary = build_dir / "labs" / "linux_cpu_inference" / "lcqi_gpt2"
    if not binary.exists():
        raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
    return binary


def write_report(
    report_path: Path,
    *,
    cache_dir: Path,
    binary: Path,
    prompt: str,
    max_new_tokens: int,
    engine: str,
    result: CommandResult,
) -> None:
    report_path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"model_repo_id={MODEL_REPO_ID}",
        f"model_revision={MODEL_REVISION}",
        f"model_cache_dir={cache_dir}",
    ]
    for file_name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items():
        path = cache_dir / file_name
        lines.append(f"file.{file_name}.bytes={path.stat().st_size}")
        lines.append(f"file.{file_name}.sha256={sha256_file(path)}")
        lines.append(f"file.{file_name}.expected_bytes={expected_bytes}")
        lines.append(f"file.{file_name}.expected_sha256={expected_sha256}")
    lines.extend(
        [
            f"binary={binary}",
            f"prompt={prompt}",
            f"max_new_tokens={max_new_tokens}",
            f"engine={engine}",
            f"command={command_line(result.args)}",
            f"exit_code={result.returncode}",
            "stdout_begin",
            result.stdout.rstrip(),
            "stdout_end",
            "stderr_begin",
            tail(result.stderr).rstrip(),
            "stderr_end",
        ]
    )
    passed = result.returncode == 0 and "generated_text " in result.stdout
    lines.append(f"validation={'PASS' if passed else 'FAIL'}")
    report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
```

```python


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Run LCQI self-owned GPT-2 smoke on a HuggingFace safetensors directory."
    )
    parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
    parser.add_argument("--engine", choices=("cached", "full"), default="cached")
    parser.add_argument("--benchmark", action="store_true")
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--no-download", action="store_true")
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    try:
        if args.max_new_tokens < 0:
            raise RuntimeError("--max-new-tokens cannot be negative")
        ensure_model_files(
            args.cache_dir,
            no_download=args.no_download,
            timeout_seconds=args.timeout_seconds,
        )
        binary = build_lcqi_gpt2(args.build_dir, args.jobs)
        command = [
            str(binary),
            "--engine",
            args.engine,
            str(args.cache_dir),
            args.prompt,
            str(args.max_new_tokens),
        ]
        if args.benchmark:
            command.insert(1, "--benchmark")
        result = run_command(command, cwd=repo_dir(), timeout_seconds=args.timeout_seconds)
        write_report(
            args.report,
            cache_dir=args.cache_dir,
            binary=binary,
            prompt=args.prompt,
            max_new_tokens=args.max_new_tokens,
```

```
            engine=args.engine,
            result=result,
        )
        print(f"[lcqi-gpt2] report: {args.report}")
        print(tail(result.stdout, max_chars=2000))
        return 0 if result.returncode == 0 else result.returncode
    except Exception as error:
        print(f"[lcqi-gpt2] error: {error}", file=sys.stderr)
        return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

run_gpt2_benchmark_compare.py 用同一个 GPT-2 F32 safetensors 模型分别运行 LCQI full-prefix 与 cached-KV 路径，并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考。它的报告说明算法差距、kernel 差距和模型口径差异。

Listing 16.9 tools/run_gpt2_benchmark_compare.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import os
import re
import shlex
import subprocess
import sys
from dataclasses import dataclass
from pathlib import Path

import run_gpt2_smoke


DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_MAX_NEW_TOKENS = 4
DEFAULT_BUILD_JOBS = 2
DEFAULT_TIMEOUT_SECONDS = 300
DEFAULT_SMOLLM2_CACHE_DIR = Path.home() / ".cache" / "lcqi-smollm2-small-smoke"
SMOLLM2_GGUF = "SmolLM2-135M-Instruct-Q4_K_M.gguf"


@dataclass(frozen=True)
class EngineRun:
    name: str
    command: list[str]
    returncode: int
    stdout: str
    stderr: str
    metrics: dict[str, str]

```

```python

def volume_dir() -> Path:
    return Path(__file__).resolve().parents[1]


def repo_dir() -> Path:
    return Path(__file__).resolve().parents[3]


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-benchmark-compare.txt"


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-release"


def command_line(args: list[str]) -> str:
    return " ".join(shlex.quote(arg) for arg in args)


def tail(text: str, max_chars: int = 4000) -> str:
    return text if len(text) <= max_chars else text[-max_chars:]


def run_command(args: list[str], *, timeout_seconds: int) -> tuple[int, str, str]:
    try:
        completed = subprocess.run(
            args,
            cwd=repo_dir(),
            text=True,
            capture_output=True,
            timeout=timeout_seconds,
            check=False,
        )
    except subprocess.TimeoutExpired as exc:
        stdout = exc.stdout if isinstance(exc.stdout, str) else ""
        stderr = exc.stderr if isinstance(exc.stderr, str) else ""
        return 124, stdout, stderr
    return completed.returncode, completed.stdout, completed.stderr


def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    for line in stdout.splitlines():
        if not line:
            continue
        key, _, value = line.partition(" ")
        if key in {
            "engine",
            "generated_text",
```

```
            "benchmark_load_ms",
            "benchmark_tokenize_ms",
            "benchmark_prefill_ms",
            "benchmark_decode_ms",
            "benchmark_generate_ms",
            "benchmark_total_ms",
            "benchmark_prompt_tokens",
            "benchmark_generated_tokens",
            "benchmark_prefill_steps",
            "benchmark_decode_steps",
            "benchmark_generate_tokens_per_second",
            "benchmark_decode_tokens_per_second",
            "benchmark_kv_cache_bytes",
        }:
            metrics[key] = value
    return metrics


def run_lcqi(
    binary: Path,
    model_dir: Path,
    *,
    engine: str,
    prompt: str,
    max_new_tokens: int,
    timeout_seconds: int,
) -> EngineRun:
    command = [
        str(binary),
        "--benchmark",
        "--engine",
        engine,
        str(model_dir),
        prompt,
        str(max_new_tokens),
    ]
    returncode, stdout, stderr = run_command(command, timeout_seconds=↩
    ↪ timeout_seconds)
    return EngineRun(
        name=f"lcqi_{engine}",
        command=command,
        returncode=returncode,
        stdout=stdout,
        stderr=stderr,
        metrics=parse_lcqi_metrics(stdout),
    )


def llama_bench_binary(cache_dir: Path) -> Path:
    return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"


```

```python
def smollm2_model_path(cache_dir: Path) -> Path:
    return cache_dir / "models" / SMOLLM2_GGUF


def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
    metrics: dict[str, str] = {}
    lines = [line.strip() for line in stdout.splitlines() if line.strip()]
    for line in lines:
        if not line.startswith("|"):
            continue
        columns = [column.strip() for column in line.strip("|").split("|")]
        if len(columns) != 7 or columns[0] == "model" or columns[0].startswith("-"):
            continue
        metrics["llama_model"] = columns[0]
        metrics["llama_size"] = columns[1]
        metrics["llama_params"] = columns[2]
        metrics["llama_backend"] = columns[3]
        metrics["llama_threads"] = columns[4]
        test_name = columns[5]
        tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
        if test_name.startswith("pp"):
            metrics["llama_prefill_test"] = test_name
            metrics["llama_prefill_tps"] = tokens_per_second
        elif test_name.startswith("tg"):
            metrics["llama_decode_test"] = test_name
            metrics["llama_decode_tps"] = tokens_per_second
    return metrics


def run_llama_bench(cache_dir: Path, *, timeout_seconds: int) -> EngineRun | None:
    binary = llama_bench_binary(cache_dir)
    model = smollm2_model_path(cache_dir)
    if not binary.exists() or not os.access(binary, os.X_OK) or not model.exists():
        return None
    command = [
        str(binary),
        "-m",
        str(model),
        "-ngl",
        "0",
        "-p",
        "5",
        "-n",
        "4",
        "-r",
        "1",
    ]
    returncode, stdout, stderr = run_command(command, timeout_seconds=timeout_seconds)
```

```
    return EngineRun(
        name="llama_cpp_smollm2_q4_k_m",
        command=command,
        returncode=returncode,
        stdout=stdout,
        stderr=stderr,
        metrics=parse_llama_bench_metrics(stdout),
    )


def write_report(path: Path, runs: list[EngineRun], *, prompt: str, max_new_tokens: int) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        "LCQI GPT-2 Benchmark Compare",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"repo_commit={current_commit()}",
        f"working_tree_dirty={working_tree_dirty()}",
        f"python={sys.version.split()[0]}",
        f"prompt={prompt}",
        f"max_new_tokens={max_new_tokens}",
        "",
        "scope=LCQI full_prefix and cached_kv run the same GPT-2 F32 safetensors model. "
        "llama.cpp uses SmolLM2-135M-Instruct Q4_K_M as an external mature-engine reference, "
        "so its numbers are engineering context, not same-model quality ranking.",
        "",
    ]
    for run in runs:
        lines.extend(
            [
                f"[{run.name}]",
                f"returncode={run.returncode}",
                f"command={command_line(run.command)}",
            ]
        )
        for key in sorted(run.metrics):
            lines.append(f"{key}={run.metrics[key]}")
        lines.extend(
            [
                "stdout_tail:",
                tail(run.stdout.strip(), 2000),
                "stderr_tail:",
                tail(run.stderr.strip(), 2000),
                "",
            ]
        )
    while lines and lines[-1] == "":
        lines.pop()
```

```
232     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
233
234
235 def current_commit() -> str:
236     result = subprocess.run(
237         ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
238         text=True,
239         capture_output=True,
240         check=False,
241     )
242     return result.stdout.strip() if result.returncode == 0 else "unknown"
243
244
245 def working_tree_dirty() -> str:
246     result = subprocess.run(
247         ["git", "-C", str(repo_dir()), "status", "--porcelain"],
248         text=True,
249         capture_output=True,
250         check=False,
251     )
252     if result.returncode != 0:
253         return "unknown"
254     return "true" if result.stdout.strip() else "false"
255
256
257 def parse_args() -> argparse.Namespace:
258     parser = argparse.ArgumentParser(
259         description="Compare LCQI GPT-2 full-prefix and KV-cache paths, with optional llama.cpp context."
260     )
261     parser.add_argument("--cache-dir", type=Path, default=run_gpt2_smoke.default_cache_dir())
262     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
263     parser.add_argument("--report", type=Path, default=default_report_path())
264     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
265     parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
266     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
267     parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
268     parser.add_argument("--no-download", action="store_true")
269     parser.add_argument("--skip-llama", action="store_true")
270     parser.add_argument("--smollm2-cache-dir", type=Path, default=DEFAULT_SMOLLM2_CACHE_DIR)
271     return parser.parse_args()
272
273
274 def main() -> int:
275     args = parse_args()
276     if args.max_new_tokens < 0:
277         print("[lcqi-compare] --max-new-tokens cannot be negative", file=sys.stderr)
```

```
278         return 1
279     try:
280         run_gpt2_smoke.ensure_model_files(
281             args.cache_dir,
282             no_download=args.no_download,
283             timeout_seconds=args.timeout_seconds,
284         )
285         binary = run_gpt2_smoke.build_lcqi_gpt2(args.build_dir, args.jobs)
286         runs = [
287             run_lcqi(
288                 binary,
289                 args.cache_dir,
290                 engine="full",
291                 prompt=args.prompt,
292                 max_new_tokens=args.max_new_tokens,
293                 timeout_seconds=args.timeout_seconds,
294             ),
295             run_lcqi(
296                 binary,
297                 args.cache_dir,
298                 engine="cached",
299                 prompt=args.prompt,
300                 max_new_tokens=args.max_new_tokens,
301                 timeout_seconds=args.timeout_seconds,
302             ),
303         ]
304         if not args.skip_llama:
305             llama_run = run_llama_bench(
306                 args.smollm2_cache_dir.expanduser().resolve(),
307                 timeout_seconds=args.timeout_seconds,
308             )
309             if llama_run is not None:
310                 runs.append(llama_run)
311         write_report(args.report, runs, prompt=args.prompt, max_new_tokens=args
    ↪ .max_new_tokens)
312     except Exception as error:
313         print(f"[lcqi-compare] error: {error}", file=sys.stderr)
314         return 1
315 
316     for run in runs:
317         print(f"{run.name}: returncode={run.returncode}")
318         for key in sorted(run.metrics):
319             print(f"  {key}={run.metrics[key]}")
320     print(f"report={args.report}")
321     return 0 if all(run.returncode == 0 for run in runs) else 1
322 
323 
324 if __name__ == "__main__":
325     raise SystemExit(main())
```

`run_gpt2_optimization_ab.py` 专门比较 LCQI 自身改动前后。它从 baseline commit 建临时 worktree，baseline 和当前工作区都通过实践卷 CMake 入口构建 Release，再多轮运行同一个

GPT-2 F32 模型。报告中的 `median/min/max` 用来约束机器波动，`generated_text` 用来确认优化前后输出没有漂移，`benchmark_worker_count` 用来确认 cached decoder 到底是串行、自动线程数还是固定线程数。

Listing 16.10 tools/run_gpt2_optimization_ab.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import shutil
import statistics
import subprocess
import sys
import tempfile
from dataclasses import dataclass
from pathlib import Path


DEFAULT_BASELINE_REF = "4febf3f"
DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_MAX_NEW_TOKENS = 4
DEFAULT_ROUNDS = 3
DEFAULT_BUILD_JOBS = 2
DEFAULT_TIMEOUT_SECONDS = 300

METRIC_KEYS = [
    "benchmark_prefill_ms",
    "benchmark_decode_ms",
    "benchmark_generate_ms",
    "benchmark_worker_count",
    "benchmark_generate_tokens_per_second",
    "benchmark_decode_tokens_per_second",
]


@dataclass(frozen=True)
class RunSample:
    variant: str
    engine: str
    round_id: int
    metrics: dict[str, float]
    generated_text: str


def script_path() -> Path:
    return Path(__file__).resolve()


def repo_dir() -> Path:
    return script_path().parents[3]

```

```
48
49 def volume_dir() -> Path:
50     return script_path().parents[1]
51
52
53 def practice_dir(root: Path) -> Path:
54     return root / "books" / "cpu-volume-3-practice"
55
56
57 def default_report_path() -> Path:
58     return volume_dir() / "results" / "lcqi-gpt2-optimization-ab.txt"
59
60
61 def default_model_dir() -> Path:
62     return Path.home() / ".cache" / "lcqi-gpt2-smoke" / "openai-community--gpt2↩
   ↪ "
63
64
65 def run_command(
66     args: list[str],
67     *,
68     cwd: Path,
69     timeout_seconds: int,
70 ) -> subprocess.CompletedProcess[str]:
71     return subprocess.run(
72         args,
73         cwd=cwd,
74         text=True,
75         capture_output=True,
76         timeout=timeout_seconds,
77         check=False,
78     )
79
80
81 def ensure_success(
82     completed: subprocess.CompletedProcess[str],
83     action: str,
84 ) -> None:
85     if completed.returncode == 0:
86         return
87     command = (
88         " ".join(completed.args)
89         if isinstance(completed.args, list)
90         else str(completed.args)
91     )
92     raise RuntimeError(
93         f"{action} failed with exit code {completed.returncode}\n"
94         f"command: {command}\n"
95         f"stdout tail:\n{completed.stdout[-2000:]}\n"
96         f"stderr tail:\n{completed.stderr[-2000:]}"
97     )
98
```

```python

def git_text(args: list[str]) -> str:
    completed = run_command(["git", *args], cwd=repo_dir(), timeout_seconds=60)
    ensure_success(completed, "git command")
    return completed.stdout.strip()


def working_tree_dirty() -> bool:
    completed = run_command(
        ["git", "status", "--porcelain"],
        cwd=repo_dir(),
        timeout_seconds=60,
    )
    ensure_success(completed, "git status")
    return bool(completed.stdout.strip())


def build_binary(root: Path, build_dir: Path, jobs: int, timeout_seconds: int) -> Path:
    configure = run_command(
        [
            "cmake",
            "-S",
            str(practice_dir(root)),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
        cwd=root,
        timeout_seconds=timeout_seconds,
    )
    ensure_success(configure, f"configure {root}")
    build = run_command(
        [
            "cmake",
            "--build",
            str(build_dir),
            "--target",
            "lcqi_gpt2",
            "-j",
            str(jobs),
        ],
        cwd=root,
        timeout_seconds=timeout_seconds,
    )
    ensure_success(build, f"build {root}")
    binary = build_dir / "linux_cpu_inference" / "lcqi_gpt2"
    if not binary.exists():
        raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
    return binary


```

```python
def parse_metrics(stdout: str) -> tuple[dict[str, float], str]:
    metrics: dict[str, float] = {}
    generated_text = ""
    for line in stdout.splitlines():
        key, _, value = line.partition(" ")
        if key in METRIC_KEYS:
            metrics[key] = float(value)
        elif key == "generated_text":
            generated_text = value
    metrics.setdefault("benchmark_worker_count", 1.0)
    missing = sorted(set(METRIC_KEYS) - metrics.keys())
    if missing:
        raise RuntimeError(f"missing benchmark metrics: {missing}")
    return metrics, generated_text


def run_sample(
    binary: Path,
    *,
    variant: str,
    engine: str,
    round_id: int,
    model_dir: Path,
    prompt: str,
    max_new_tokens: int,
    threads: int,
    timeout_seconds: int,
) -> RunSample:
    command = [
        str(binary),
        "--benchmark",
        "--engine",
        engine,
    ]
    if variant == "current":
        command.extend(["--threads", str(threads)])
    command.extend([
        str(model_dir),
        prompt,
        str(max_new_tokens),
    ])
    completed = run_command(
        command,
        cwd=repo_dir(),
        timeout_seconds=timeout_seconds,
    )
    ensure_success(completed, f"run {variant} {engine}")
    metrics, generated_text = parse_metrics(completed.stdout)
    return RunSample(
        variant=variant,
        engine=engine,
        round_id=round_id,
```

```
        metrics=metrics,
        generated_text=generated_text,
    )


def summarize(samples: list[RunSample]) -> list[str]:
    lines: list[str] = []
    variants = sorted({sample.variant for sample in samples})
    engines = sorted({sample.engine for sample in samples})
    for variant in variants:
        for engine in engines:
            selected = [
                sample for sample in samples
                if sample.variant == variant and sample.engine == engine
            ]
            if not selected:
                continue
            lines.append(f"[{variant} {engine}]")
            for key in METRIC_KEYS:
                values = [sample.metrics[key] for sample in selected]
                lines.append(
                    f"{key}=median:{statistics.median(values):.6g},"
                    f"min:{min(values):.6g},max:{max(values):.6g}"
                )
            texts = sorted({sample.generated_text for sample in selected})
            lines.append(f"generated_text={texts[0] if len(texts) == 1 else texts}")
            lines.append("")
    return lines


def write_report(
    path: Path,
    samples: list[RunSample],
    *,
    baseline_ref: str,
    baseline_commit: str,
    current_commit: str,
    prompt: str,
    max_new_tokens: int,
    rounds: int,
    current_threads: int,
) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        "LCQI GPT-2 Optimization A/B",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"baseline_ref={baseline_ref}",
        f"baseline_commit={baseline_commit}",
        f"current_commit={current_commit}",
        f"current_working_tree_dirty={str(working_tree_dirty()).lower()}",
```

```
253         f"prompt={prompt}",
254         f"max_new_tokens={max_new_tokens}",
255         f"rounds={rounds}",
256         f"baseline_threads=1",
257         f"current_threads={current_threads}",
258         "",
259         "scope=Both variants are built through books/cpu-volume-3-practice with "
260         "CMAKE_BUILD_TYPE=Release and run on the same GPT-2 F32 safetensors model. "
261         "The llama.cpp comparison remains in lcqi-gpt2-benchmark-compare.txt; this "
262         "report isolates LCQI before/after code changes.",
263         "",
264         "[samples]",
265     ]
266     for sample in samples:
267         metric_text = ",".join(
268             f"{key}:{sample.metrics[key]:.6g}" for key in METRIC_KEYS
269         )
270         lines.append(
271             f"round={sample.round_id},variant={sample.variant},engine={sample.engine},"
272             f"{metric_text},generated_text={sample.generated_text}"
273         )
274     lines.append("")
275     lines.extend(summarize(samples))
276     while lines and lines[-1] == "":
277         lines.pop()
278     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
279
280
281 def parse_args() -> argparse.Namespace:
282     parser = argparse.ArgumentParser(description="Run LCQI GPT-2 before/after A/B benchmark.")
283     parser.add_argument("--baseline-ref", default=DEFAULT_BASELINE_REF)
284     parser.add_argument("--model-dir", type=Path, default=default_model_dir())
285     parser.add_argument("--report", type=Path, default=default_report_path())
286     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
287     parser.add_argument("--max-new-tokens", type=int, default=DEFAULT_MAX_NEW_TOKENS)
288     parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
289     parser.add_argument("--threads", type=int, default=0)
290     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
291     parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
292     return parser.parse_args()
293
294
295 def main() -> int:
296     args = parse_args()
297     if args.rounds <= 0:
```

```
        print("[lcqi-ab] --rounds must be positive", file=sys.stderr)
        return 1
    if args.max_new_tokens < 0:
        print("[lcqi-ab] --max-new-tokens cannot be negative", file=sys.stderr)
        return 1
    if args.threads < 0:
        print("[lcqi-ab] --threads cannot be negative", file=sys.stderr)
        return 1
    if not args.model_dir.exists():
        print(f"[lcqi-ab] model directory not found: {args.model_dir}", file=sys.stderr)
        return 1

    baseline_commit = git_text(["rev-parse", "--short", args.baseline_ref])
    current_commit = git_text(["rev-parse", "--short", "HEAD"])
    with tempfile.TemporaryDirectory(prefix="lcqi-gpt2-ab-") as tmp:
        tmp_path = Path(tmp)
        baseline_root = tmp_path / "baseline"
        current_build = tmp_path / "current-build"
        baseline_build = tmp_path / "baseline-build"
        ensure_success(
            run_command(
                ["git", "worktree", "add", "--detach", str(baseline_root), args.baseline_ref],
                cwd=repo_dir(),
                timeout_seconds=args.timeout_seconds,
            ),
            "create baseline worktree",
        )
        try:
            baseline_binary = build_binary(
                baseline_root,
                baseline_build,
                args.jobs,
                args.timeout_seconds,
            )
            current_binary = build_binary(
                repo_dir(),
                current_build,
                args.jobs,
                args.timeout_seconds,
            )

            samples: list[RunSample] = []
            for round_id in range(1, args.rounds + 1):
                for variant, binary in [
                    ("baseline", baseline_binary),
                    ("current", current_binary),
                ]:
                    for engine in ["full", "cached"]:
                        sample = run_sample(
                            binary,
```

```
348                             variant=variant,
349                             engine=engine,
350                             round_id=round_id,
351                             model_dir=args.model_dir,
352                             prompt=args.prompt,
353                             max_new_tokens=args.max_new_tokens,
354                             threads=1 if variant == "baseline" else args.threads,
355                             timeout_seconds=args.timeout_seconds,
356                         )
357                         samples.append(sample)
358                         print(
359                             f"round={round_id} variant={variant} engine={engine} "
360                             f"generate_ms={sample.metrics['benchmark_generate_ms']:.3f} "
361                             f"workers={sample.metrics['benchmark_worker_count']:.0f} "
362                             f"gen_tps={sample.metrics['benchmark_generate_tokens_per_second']:.5f}"
363                         )
364             write_report(
365                 args.report,
366                 samples,
367                 baseline_ref=args.baseline_ref,
368                 baseline_commit=baseline_commit,
369                 current_commit=current_commit,
370                 prompt=args.prompt,
371                 max_new_tokens=args.max_new_tokens,
372                 rounds=args.rounds,
373                 current_threads=args.threads,
374             )
375         finally:
376             subprocess.run(
377                 ["git", "worktree", "remove", "--force", str(baseline_root)],
378                 cwd=repo_dir(),
379                 text=True,
380                 capture_output=True,
381                 check=False,
382             )
383     print(f"report={args.report}")
384     return 0
385
386
387 if __name__ == "__main__":
388     raise SystemExit(main())
```

`run_gpt2_hotspot_profile.py` 专门收集 cached decode 内部热点。它构建 Release `lcqi_gpt2`，用 `--profile-hotspots` 输出 `hotspot_lm_head_pct`、`hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`hotspot_qkv_projection_pct`、`hotspot_attention_projection_pct`、`hotspot_attention_pct` 和 `benchmark_worker_count`，再按 token 数和轮次计算中位数。这个脚本回答“优化应该打哪里”，不是回答“某次提交快了多少”。本轮线程

优化就是靠它发现第一次 auto 阈值漏掉了 attention output projection，随后把行数和列数阈值一起纳入并行判定。

Listing 16.11 tools/run_gpt2_hotspot_profile.py

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import datetime as dt
import statistics
import subprocess
from dataclasses import dataclass
from pathlib import Path


DEFAULT_PROMPT = "Hello, my name is"
DEFAULT_TOKEN_COUNTS = "4,16"
DEFAULT_ROUNDS = 3
DEFAULT_BUILD_JOBS = 2
DEFAULT_TIMEOUT_SECONDS = 900

METRIC_PREFIXES = ("benchmark_", "hotspot_")


@dataclass(frozen=True)
class HotspotSample:
    max_new_tokens: int
    round_id: int
    metrics: dict[str, float]
    generated_text: str


def script_path() -> Path:
    return Path(__file__).resolve()


def repo_dir() -> Path:
    return script_path().parents[3]


def volume_dir() -> Path:
    return script_path().parents[1]


def practice_dir() -> Path:
    return repo_dir() / "books" / "cpu-volume-3-practice"


def default_build_dir() -> Path:
    return volume_dir() / "build" / "lcqi-hotspot-profile"


```

```python
def default_model_dir() -> Path:
    return Path.home() / ".cache" / "lcqi-gpt2-smoke" / "openai-community--gpt2"


def default_report_path() -> Path:
    return volume_dir() / "results" / "lcqi-gpt2-hotspot-profile.txt"


def run_command(
    args: list[str],
    *,
    cwd: Path,
    timeout_seconds: int,
) -> subprocess.CompletedProcess[str]:
    return subprocess.run(
        args,
        cwd=cwd,
        text=True,
        capture_output=True,
        timeout=timeout_seconds,
        check=False,
    )


def ensure_success(completed: subprocess.CompletedProcess[str], action: str) -> None:
    if completed.returncode == 0:
        return
    command = (
        " ".join(completed.args)
        if isinstance(completed.args, list)
        else str(completed.args)
    )
    raise RuntimeError(
        f"{action} failed with exit code {completed.returncode}\n"
        f"command: {command}\n"
        f"stdout tail:\n{completed.stdout[-2000:]}\n"
        f"stderr tail:\n{completed.stderr[-2000:]}"
    )


def build_binary(build_dir: Path, jobs: int, timeout_seconds: int) -> Path:
    configure = run_command(
        [
            "cmake",
            "-S",
            str(practice_dir()),
            "-B",
            str(build_dir),
            "-DCMAKE_BUILD_TYPE=Release",
        ],
```

```
        cwd=repo_dir(),
        timeout_seconds=timeout_seconds,
    )
    ensure_success(configure, "configure hotspot profile build")
    build = run_command(
        [
            "cmake",
            "--build",
            str(build_dir),
            "--target",
            "lcqi_gpt2",
            "-j",
            str(jobs),
        ],
        cwd=repo_dir(),
        timeout_seconds=timeout_seconds,
    )
    ensure_success(build, "build lcqi_gpt2")
    binary = build_dir / "linux_cpu_inference" / "lcqi_gpt2"
    if not binary.exists():
        raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
    return binary


def parse_numeric_list(value: str) -> list[int]:
    result: list[int] = []
    for part in value.split(","):
        stripped = part.strip()
        if not stripped:
            continue
        parsed = int(stripped)
        if parsed <= 0:
            raise RuntimeError("--token-counts entries must be positive")
        result.append(parsed)
    if not result:
        raise RuntimeError("--token-counts must contain at least one positive integer")
    return result


def parse_stdout(stdout: str) -> tuple[dict[str, float], str]:
    metrics: dict[str, float] = {}
    generated_text = ""
    for line in stdout.splitlines():
        key, _, value = line.partition(" ")
        if key.startswith(METRIC_PREFIXES):
            metrics[key] = float(value)
        elif key == "generated_text":
            generated_text = value
    required = {
        "benchmark_generate_ms",
        "benchmark_worker_count",
```

```
        "hotspot_total_step_ms",
        "hotspot_lm_head_pct",
        "hotspot_mlp_fc_pct",
        "hotspot_mlp_projection_pct",
        "hotspot_qkv_projection_pct",
        "hotspot_attention_pct",
    }
    missing = sorted(required - metrics.keys())
    if missing:
        raise RuntimeError(f"missing hotspot metrics: {missing}")
    if not generated_text:
        raise RuntimeError("missing generated_text")
    return metrics, generated_text


def run_sample(
    binary: Path,
    *,
    model_dir: Path,
    prompt: str,
    max_new_tokens: int,
    round_id: int,
    threads: int,
    timeout_seconds: int,
) -> HotspotSample:
    completed = run_command(
        [
            str(binary),
            "--profile-hotspots",
            "--engine",
            "cached",
            "--threads",
            str(threads),
            str(model_dir),
            prompt,
            str(max_new_tokens),
        ],
        cwd=repo_dir(),
        timeout_seconds=timeout_seconds,
    )
    ensure_success(completed, f"run hotspot profile tokens={max_new_tokens}")
    metrics, generated_text = parse_stdout(completed.stdout)
    return HotspotSample(
        max_new_tokens=max_new_tokens,
        round_id=round_id,
        metrics=metrics,
        generated_text=generated_text,
    )


def summarize(samples: list[HotspotSample]) -> list[str]:
    lines: list[str] = []
```

```
    metric_keys = [
        "benchmark_load_ms",
        "benchmark_generate_ms",
        "benchmark_generate_tokens_per_second",
        "benchmark_decode_tokens_per_second",
        "hotspot_total_step_ms",
        "benchmark_worker_count",
        "hotspot_lm_head_pct",
        "hotspot_mlp_fc_pct",
        "hotspot_mlp_projection_pct",
        "hotspot_qkv_projection_pct",
        "hotspot_attention_projection_pct",
        "hotspot_attention_pct",
    ]
    for token_count in sorted({sample.max_new_tokens for sample in samples}):
        selected = [sample for sample in samples if sample.max_new_tokens == token_count]
        lines.append(f"[summary max_new_tokens={token_count}]")
        for key in metric_keys:
            values = [sample.metrics[key] for sample in selected if key in sample.metrics]
            if not values:
                continue
            lines.append(
                f"{key}=median:{statistics.median(values):.6g},"
                f"min:{min(values):.6g},max:{max(values):.6g}"
            )
        texts = sorted({sample.generated_text for sample in selected})
        lines.append(f"generated_text={texts[0] if len(texts) == 1 else texts}")
        lines.append("")
    return lines


def write_report(
    path: Path,
    samples: list[HotspotSample],
    *,
    model_dir: Path,
    prompt: str,
    token_counts: list[int],
    rounds: int,
    threads: int,
) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    lines = [
        "LCQI GPT-2 Hotspot Profile",
        "",
        f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
        f"model_dir={model_dir}",
        f"prompt={prompt}",
        f"token_counts={','.join(str(value) for value in token_counts)}",
```

```
        f"rounds={rounds}",
        f"requested_threads={threads}",
        "engine=cached",
        "build=CMAKE_BUILD_TYPE=Release through books/cpu-volume-3-practice",
        "",
        "[samples]",
    ]
    for sample in samples:
        metric_text = ",".join(
            f"{key}:{sample.metrics[key]:.6g}"
            for key in sorted(sample.metrics)
            if key.startswith(METRIC_PREFIXES)
        )
        lines.append(
            f"round={sample.round_id},max_new_tokens={sample.max_new_tokens},"
            f"{metric_text},generated_text={sample.generated_text}"
        )
    lines.append("")
    lines.extend(summarize(samples))
    while lines and lines[-1] == "":
        lines.pop()
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run LCQI GPT-2 cached hotspot profile.")
    parser.add_argument("--model-dir", type=Path, default=default_model_dir())
    parser.add_argument("--build-dir", type=Path, default=default_build_dir())
    parser.add_argument("--report", type=Path, default=default_report_path())
    parser.add_argument("--prompt", default=DEFAULT_PROMPT)
    parser.add_argument("--token-counts", default=DEFAULT_TOKEN_COUNTS)
    parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
    parser.add_argument("--threads", type=int, default=0)
    parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
    parser.add_argument("--timeout-seconds", type=int, default=DEFAULT_TIMEOUT_SECONDS)
    return parser.parse_args()


def main() -> int:
    args = parse_args()
    if args.rounds <= 0:
        print("[lcqi-hotspot] --rounds must be positive")
        return 1
    if args.threads < 0:
        print("[lcqi-hotspot] --threads cannot be negative")
        return 1
    if not args.model_dir.exists():
        print(f"[lcqi-hotspot] model directory not found: {args.model_dir}")
        return 1
    token_counts = parse_numeric_list(args.token_counts)
```

```
    binary = build_binary(args.build_dir, args.jobs, args.timeout_seconds)
    samples: list[HotspotSample] = []
    for round_id in range(1, args.rounds + 1):
        for token_count in token_counts:
            sample = run_sample(
                binary,
                model_dir=args.model_dir,
                prompt=args.prompt,
                max_new_tokens=token_count,
                round_id=round_id,
                threads=args.threads,
                timeout_seconds=args.timeout_seconds,
            )
            samples.append(sample)
            print(
                f"round={round_id} max_new_tokens={token_count} "
                f"generate_ms={sample.metrics['benchmark_generate_ms']:.3f} "
                f"workers={sample.metrics['benchmark_worker_count']:.0f} "
                f"lm_head_pct={sample.metrics['hotspot_lm_head_pct']:.3f} "
                f"mlp_pct={sample.metrics['hotspot_mlp_fc_pct'] + sample.metrics['hotspot_mlp_projection_pct']:.3f}"
            )
    write_report(
        args.report,
        samples,
        model_dir=args.model_dir,
        prompt=args.prompt,
        token_counts=token_counts,
        rounds=args.rounds,
        threads=args.threads,
    )
    print(f"report={args.report}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`run_lcqi_benchmark.sh` 是本地 benchmark 收集脚本。它适合在固定机器上反复运行，把同一组 shape 和 backend 的结果落到报告文件。

Listing 16.12 tools/run_lcqi_benchmark.sh

```
#!/usr/bin/env bash
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
VOLUME_DIR="$(cd "${SCRIPT_DIR}/.." && pwd)"
REPO_DIR="$(cd "${VOLUME_DIR}/../.." && pwd)"
BUILD_DIR="${VOLUME_DIR}/build/lcqi-release"
RESULT_DIR="${VOLUME_DIR}/results"
REPEAT="${1:-200}"
STAMP="$(date -u +%Y%m%dT%H%M%SZ)"
```

```
CSV_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.csv"
META_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.meta.txt"
PERF_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.perf.txt"

mkdir -p "${RESULT_DIR}"

cmake -S "${VOLUME_DIR}" -B "${BUILD_DIR}" -DCMAKE_BUILD_TYPE=Release
cmake --build "${BUILD_DIR}"
ctest --test-dir "${BUILD_DIR}" --output-on-failure

{
  echo "timestamp_utc=${STAMP}"
  echo "repeat=${REPEAT}"
  echo "commit=$(git -C "${REPO_DIR}" rev-parse --short HEAD)"
  echo "uname=$(uname -a)"
  echo "compiler=$(${CXX:-c++} --version | sed -n '1p')"
  echo "cmake=$(cmake --version | sed -n '1p')"
  echo "bench=${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench"
} > "${META_PATH}"

"${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" > "${CSV_PATH}"

if command -v perf >/dev/null 2>&1; then
  perf stat \
    -e cycles,instructions,cache-misses,branches,branch-misses \
    "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" \
    > /dev/null 2> "${PERF_PATH}" || true
else
  echo "perf_unavailable=1" > "${PERF_PATH}"
fi

echo "csv=${CSV_PATH}"
echo "meta=${META_PATH}"
echo "perf=${PERF_PATH}"
```

完整测试

`lcqi_tests.cpp` 把 safetensors、tokenizer、tiny MLP、int8 kernel、reference decoder、GPT-2 tiny path、trace、ledger 和 serving probe 串成验收网。GPT-2 优化不是只看速度：测试会把 full-prefix logits、旧 cached logits、优化 decoder logits、强制两 worker 的并行 decoder logits 和 logits-free greedy token 放在同一个 tiny prompt 下对齐，确认 predicted token、logits、KV filled token 数和 generated ids 不漂移。新增优化或 shape 时，必须先扩展这里的 golden 和边界样例。

Listing 16.13 tests/lcqi_tests.cpp

```
#include <lcqi/gpt2_reference.hpp>
#include <lcqi/inference.hpp>
#include <lcqi/int8_kernels.hpp>
#include <lcqi/model.hpp>
#include <lcqi/reference_decoder.hpp>
#include <lcqi/safetensors.hpp>
#include <lcqi/tokenizer.hpp>
```

```cpp

#include <array>
#include <cmath>
#include <cstring>
#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <span>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

namespace {

constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ↩
    ↪ LCQI_REFERENCE_DECODER_LOGITS{
    0.352516979F,
    -0.126884326F,
    0.0797837451F,
    -0.29797405F,
    0.127030417F,
};
constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 13452902845388333734ULL;
constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
```

```
59 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
60 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
61 constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
62 constexpr std::int32_t LCQI_GPT2_PREDICTED_TOKEN = 3;
63 constexpr std::size_t LCQI_GPT2_VOCAB_SIZE = 6;
64 constexpr std::array<float, LCQI_GPT2_VOCAB_SIZE> LCQI_GPT2_LOGITS{
65     0.407358F,
66     -0.504049F,
67     -0.107834F,
68     0.840337F,
69     -0.450123F,
70     -0.156763F,
71 };
72 constexpr std::int32_t LCQI_GPT2_GENERATED_LENGTH = 5;
73 constexpr std::int32_t LCQI_GPT2_TOKENIZER_HELLO_ID = 7;
74 constexpr std::uint16_t LCQI_TEST_F16_ONE = 0x3C00U;
75 constexpr std::uint16_t LCQI_TEST_F16_NEGATIVE_TWO = 0xC000U;
76 constexpr std::uint16_t LCQI_TEST_BF16_ONE_POINT_FIVE = 0x3FC0U;
77 constexpr std::uint16_t LCQI_TEST_BF16_NEGATIVE_HALF = 0xBF00U;
78
79 struct KernelShapeCase {
80     std::int32_t input_size = 0;
81     std::int32_t output_size = 0;
82     std::int32_t output_block_size = 0;
83 };
84
85 struct RawTensorFixture {
86     std::string name;
87     std::string dtype;
88     std::vector<std::int64_t> shape;
89     std::vector<std::uint8_t> bytes;
90 };
91
92 struct FloatTensorFixture {
93     std::string name;
94     std::vector<std::int64_t> shape;
95     std::vector<float> values;
96 };
97
98 constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
99     {17, 19, 8},
100     {33, 7, 8},
101     {64, 32, 16},
102     {65, 31, 16},
103 }};
104
105 void require(bool condition, const char* message) {
106     if (!condition) {
107         throw std::runtime_error(message);
108     }
109 }
110
```

```cpp
void require_close(float actual, float expected, const char* message) {
    if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
        throw std::runtime_error(message);
    }
}

std::filesystem::path test_model_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_mlp_i8.txt";
}

std::filesystem::path test_tokenizer_path() {
    return std::filesystem::path(__FILE__).parent_path().parent_path() /
           "models" / "tiny_tokenizer.txt";
}

std::filesystem::path temp_file_path(const std::string& name) {
    return std::filesystem::temp_directory_path() / name;
}

void write_le_u64(std::ofstream& output, std::uint64_t value) {
    for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
        const char byte = static_cast<char>(
            (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
        output.write(&byte, 1);
    }
}

void append_le_u16(std::vector<std::uint8_t>& output, std::uint16_t value) {
    output.push_back(static_cast<std::uint8_t>(value & LCQI_BYTE_MASK));
    output.push_back(static_cast<std::uint8_t>((value >> LCQI_TEST_BYTE_BITS) &
    ↪  LCQI_BYTE_MASK));
}

void append_le_f32(std::vector<std::uint8_t>& output, float value) {
    std::uint32_t bits = 0;
    std::memcpy(&bits, &value, sizeof(bits));
    for (std::int32_t i = 0; i < 4; ++i) {
        output.push_back(static_cast<std::uint8_t>(
            (bits >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
    }
}

void write_safetensors_fixture(const std::filesystem::path& path,
                               const std::string& header,
                               std::uint64_t payload_bytes) {
    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create safetensors test fixture");
    }
    write_le_u64(output, static_cast<std::uint64_t>(header.size()));
    output.write(header.data(), static_cast<std::streamsize>(header.size()));
```

```
    for (std::uint64_t i = 0; i < payload_bytes; ++i) {
        const char byte = static_cast<char>(i & 0xFFU);
        output.write(&byte, 1);
    }
}

std::string json_shape(std::span<const std::int64_t> shape) {
    std::ostringstream stream;
    stream << '[';
    for (std::size_t index = 0; index < shape.size(); ++index) {
        if (index != 0) {
            stream << ',';
        }
        stream << shape[index];
    }
    stream << ']';
    return stream.str();
}

void write_safetensors_raw_fixture(const std::filesystem::path& path,
                                   std::span<const RawTensorFixture> tensors) {
    std::ostringstream header;
    header << "{\"__metadata__\":{\"format\":\"lcqi-test\"}";
    std::uint64_t offset = 0;
    for (const RawTensorFixture& tensor : tensors) {
        header << ",\"" << tensor.name << "\":{\"dtype\":\"" << tensor.dtype
               << "\",\"shape\":" << json_shape(tensor.shape)
               << ",\"data_offsets\":[" << offset << ','
               << offset + tensor.bytes.size() << "]}";
        offset += tensor.bytes.size();
    }
    header << '}';

    std::ofstream output(path, std::ios::binary);
    if (!output) {
        throw std::runtime_error("cannot create safetensors raw test fixture");
    }
    const std::string header_text = header.str();
    write_le_u64(output, static_cast<std::uint64_t>(header_text.size()));
    output.write(header_text.data(), static_cast<std::streamsize>(header_text.
    ↪ size()));
    for (const RawTensorFixture& tensor : tensors) {
        output.write(reinterpret_cast<const char*>(tensor.bytes.data()),
                     static_cast<std::streamsize>(tensor.bytes.size()));
    }
}

std::vector<std::uint8_t> f32_payload(std::span<const float> values) {
    std::vector<std::uint8_t> bytes;
    bytes.reserve(values.size() * 4);
    for (const float value : values) {
        append_le_f32(bytes, value);
```

```
    }
    return bytes;
}

std::vector<float> transpose_linear_to_hf_conv1d(const lcqi::Gpt2LinearF32& linear) {
    std::vector<float> transposed(
        static_cast<std::size_t>(linear.input_size) *
            static_cast<std::size_t>(linear.output_size),
        0.0F);
    for (std::int32_t out = 0; out < linear.output_size; ++out) {
        for (std::int32_t in = 0; in < linear.input_size; ++in) {
            transposed[static_cast<std::size_t>(in) *
                           static_cast<std::size_t>(linear.output_size) +
                       static_cast<std::size_t>(out)] =
                linear.weights[static_cast<std::size_t>(out) *
                                   static_cast<std::size_t>(linear.input_size) +
                               static_cast<std::size_t>(in)];
        }
    }
    return transposed;
}

void add_f32_tensor(std::vector<RawTensorFixture>& tensors,
                    std::string name,
                    std::vector<std::int64_t> shape,
                    std::span<const float> values) {
    tensors.push_back(RawTensorFixture{
        std::move(name),
        "F32",
        std::move(shape),
        f32_payload(values),
    });
}

void write_text_file(const std::filesystem::path& path, const std::string& text) {
    std::ofstream output(path);
    if (!output) {
        throw std::runtime_error("cannot create text fixture");
    }
    output << text;
}

void write_tiny_gpt2_safetensors(const std::filesystem::path& path,
                                 const lcqi::Gpt2ReferenceModel& model) {
    std::vector<RawTensorFixture> tensors;
    add_f32_tensor(tensors,
                   "transformer.wte.weight",
                   {model.config.vocab_size, model.config.hidden_size},
                   model.token_embedding);
```

```
    add_f32_tensor(tensors,
                   "transformer.wpe.weight",
                   {model.config.max_positions, model.config.hidden_size},
                   model.position_embedding);
    for (std::int32_t layer_id = 0; layer_id < model.config.layer_count; ++layer_id) {
        const lcqi::Gpt2LayerWeightsF32& layer =
            model.layers[static_cast<std::size_t>(layer_id)];
        const std::string prefix = "transformer.h." + std::to_string(layer_id) + ".";
        add_f32_tensor(tensors, prefix + "ln_1.weight", {model.config.hidden_size}, layer.ln_1_weight);
        add_f32_tensor(tensors, prefix + "ln_1.bias", {model.config.hidden_size}, layer.ln_1_bias);
        const std::vector<float> c_attn_hf = transpose_linear_to_hf_conv1d(layer.c_attn);
        add_f32_tensor(tensors,
                       prefix + "attn.c_attn.weight",
                       {model.config.hidden_size,
                        model.config.hidden_size * 3},
                       c_attn_hf);
        add_f32_tensor(tensors,
                       prefix + "attn.c_attn.bias",
                       {model.config.hidden_size * 3},
                       layer.c_attn.bias);
        const std::vector<float> c_proj_hf = transpose_linear_to_hf_conv1d(layer.c_proj);
        add_f32_tensor(tensors,
                       prefix + "attn.c_proj.weight",
                       {model.config.hidden_size, model.config.hidden_size},
                       c_proj_hf);
        add_f32_tensor(tensors,
                       prefix + "attn.c_proj.bias",
                       {model.config.hidden_size},
                       layer.c_proj.bias);
        add_f32_tensor(tensors, prefix + "ln_2.weight", {model.config.hidden_size}, layer.ln_2_weight);
        add_f32_tensor(tensors, prefix + "ln_2.bias", {model.config.hidden_size}, layer.ln_2_bias);
        const std::vector<float> c_fc_hf = transpose_linear_to_hf_conv1d(layer.c_fc);
        add_f32_tensor(tensors,
                       prefix + "mlp.c_fc.weight",
                       {model.config.hidden_size, model.config.intermediate_size},
                       c_fc_hf);
        add_f32_tensor(tensors,
                       prefix + "mlp.c_fc.bias",
                       {model.config.intermediate_size},
                       layer.c_fc.bias);
        const std::vector<float> mlp_proj_hf =
            transpose_linear_to_hf_conv1d(layer.mlp_c_proj);
```

```
304         add_f32_tensor(tensors,
305                        prefix + "mlp.c_proj.weight",
306                        {model.config.intermediate_size, model.config.↩
    ↪ hidden_size},
307                        mlp_proj_hf);
308         add_f32_tensor(tensors,
309                        prefix + "mlp.c_proj.bias",
310                        {model.config.hidden_size},
311                        layer.mlp_c_proj.bias);
312     }
313     add_f32_tensor(tensors,
314                    "transformer.ln_f.weight",
315                    {model.config.hidden_size},
316                    model.final_ln_weight);
317     add_f32_tensor(tensors,
318                    "transformer.ln_f.bias",
319                    {model.config.hidden_size},
320                    model.final_ln_bias);
321     write_safetensors_raw_fixture(path, tensors);
322 }
323
324 void test_tiny_mlp() {
325     const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
326     require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
327     require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch")↩
    ↪ ;
328     require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch")↩
    ↪ ;
329
330     const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
331     const lcqi::InferenceResult result = lcqi::run_inference(model, input);
332
333     require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected ↩
    ↪ predicted class");
334     require(result.logits.size() == static_cast<std::size_t>(↩
    ↪ LCQI_TEST_OUTPUT_SIZE),
335             "unexpected logits size");
336     require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
337     require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");
338
339     std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), ↩
    ↪ 0.0F);
340     lcqi::linear_i8(model.hidden, input, hidden);
341     lcqi::relu_inplace(hidden);
342     require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
343     require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
344     require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
345 }
346
347 void test_tokenizer_contract() {
348     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↩
    ↪ test_tokenizer_path());
```

```
    require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
    require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
    require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
    require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
            "tokenizer template hash mismatch");

    const std::vector<std::int32_t> token_ids =
        lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
    const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
    require(token_ids == expected, "tokenizer prompt ids mismatch");
    require(lcqi::tokenizer_contract_hash(tokenizer) == LCQI_TOKENIZER_HASH_EXPECTED,
            "tokenizer contract hash mismatch");
}

void test_tokenizer_rejects_unknown_without_unk() {
    const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(test_tokenizer_path());
    bool threw = false;
    try {
        static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"));
    } catch (const std::runtime_error&) {
        threw = true;
    }
    require(threw, "tokenizer should reject unknown token when unk id is absent");
}

void test_safetensors_manifest_contract() {
    const std::filesystem::path path =
        temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
    const std::string header =
        R"({"__metadata__":{"format":"lcqi-fixture"},"model.embed_tokens.weight":{"dtype":"F32","shape":[2,3],"data_offsets":[0,24]},"model.layers.0.attn.q_proj.weight":{"dtype":"F16","shape":[4,3],"data_offsets":[24,48]}})";
    write_safetensors_fixture(path, header, LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);

    const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(path);
    std::filesystem::remove(path);

    require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
            "safetensors header size mismatch");
    require(manifest.data_start_offset ==
                static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
                    static_cast<std::uint64_t>(header.size()),
            "safetensors data start mismatch");
    require(manifest.metadata.size() == 1, "safetensors metadata count mismatch");
```

```
391     require(manifest.metadata[0].first == "format" &&
392                 manifest.metadata[0].second == "lcqi-fixture",
393             "safetensors metadata mismatch");
394     require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");
395 
396     const lcqi::SafeTensorEntry* embedding =
397         manifest.find_tensor("model.embed_tokens.weight");
398     require(embedding != nullptr, "safetensors embedding tensor missing");
399     require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
400     require(embedding->shape == std::vector<std::int64_t>({2, 3}),
401             "safetensors embedding shape mismatch");
402     require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
403                 embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
404             "safetensors embedding offset mismatch");
405     require(embedding->byte_size() ==
406                 LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
407             "safetensors embedding byte size mismatch");
408 
409     const lcqi::SafeTensorEntry* q_proj =
410         manifest.find_tensor("model.layers.0.attn.q_proj.weight");
411     require(q_proj != nullptr, "safetensors q projection tensor missing");
412     require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
413     require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
414                 q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
415             "safetensors q projection offset mismatch");
416 }
417 
418 void test_safetensors_rejects_bad_offsets() {
419     const std::filesystem::path path =
420         temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
421     const std::string header =
422         R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,32]}})";
423     write_safetensors_fixture(path, header, 4);
424 
425     bool threw = false;
426     try {
427         static_cast<void>(lcqi::load_safetensors_manifest(path));
428     } catch (const std::runtime_error&) {
429         threw = true;
430     }
431     std::filesystem::remove(path);
432     require(threw, "safetensors parser should reject offsets beyond payload");
433 }
434 
435 void test_safetensors_rejects_bad_dtype_shape_size() {
436     const std::filesystem::path path =
437         temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
438     const std::string header =
439         R"({"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,8]}})";
440     write_safetensors_fixture(path, header, 8);
441 
442     bool threw = false;
```

```
443     try {
444         static_cast<void>(lcqi::load_safetensors_manifest(path));
445     } catch (const std::runtime_error&) {
446         threw = true;
447     }
448     std::filesystem::remove(path);
449     require(threw, "safetensors parser should reject dtype/shape byte mismatch");
450 }
451
452 void test_safetensors_reads_float_tensors() {
453     const std::filesystem::path path =
454         temp_file_path("lcqi_safetensors_float_read.safetensors");
455     std::vector<std::uint8_t> f16_bytes;
456     append_le_u16(f16_bytes, LCQI_TEST_F16_ONE);
457     append_le_u16(f16_bytes, LCQI_TEST_F16_NEGATIVE_TWO);
458     std::vector<std::uint8_t> bf16_bytes;
459     append_le_u16(bf16_bytes, LCQI_TEST_BF16_ONE_POINT_FIVE);
460     append_le_u16(bf16_bytes, LCQI_TEST_BF16_NEGATIVE_HALF);
461
462     const std::array<RawTensorFixture, 3> tensors{{
463         {"f32", "F32", {2}, f32_payload(std::array<float, 2>{1.25F, -2.5F})},
464         {"f16", "F16", {2}, f16_bytes},
465         {"bf16", "BF16", {2}, bf16_bytes},
466     }};
467     write_safetensors_raw_fixture(path, tensors);
468
469     const lcqi::SafeTensorFile file = lcqi::load_safetensors_file(path);
470     std::filesystem::remove(path);
471     const std::vector<float> f32 = lcqi::read_safetensor_f32_tensor(file, "f32");
472     const std::vector<float> f16 = lcqi::read_safetensor_f32_tensor(file, "f16");
473     const std::vector<float> bf16 = lcqi::read_safetensor_f32_tensor(file, "bf16");
474
475     require_close(f32[0], 1.25F, "safetensors F32 value 0 mismatch");
476     require_close(f32[1], -2.5F, "safetensors F32 value 1 mismatch");
477     require_close(f16[0], 1.0F, "safetensors F16 value 0 mismatch");
478     require_close(f16[1], -2.0F, "safetensors F16 value 1 mismatch");
479     require_close(bf16[0], 1.5F, "safetensors BF16 value 0 mismatch");
480     require_close(bf16[1], -0.5F, "safetensors BF16 value 1 mismatch");
481 }
482
483 void test_gpt2_tiny_forward_and_generation() {
484     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model();
485     const std::vector<std::int32_t> tokens{1, 2, 3};
486     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens);
487
488     require(result.logits.size() == LCQI_GPT2_VOCAB_SIZE, "GPT-2 logits size
```

```
    mismatch");
    require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
            "GPT-2 predicted token mismatch");
    for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
        require_close(result.logits[index], LCQI_GPT2_LOGITS[index], "GPT-2 logit mismatch");
    }

    const std::vector<std::int32_t> generated =
        lcqi::gpt2_generate_greedy(model, tokens, 2);
    const std::vector<std::int32_t> expected{1, 2, 3, 3, 3};
    require(generated == expected, "GPT-2 greedy generation mismatch");
    require(static_cast<std::int32_t>(generated.size()) == LCQI_GPT2_GENERATED_LENGTH,
            "GPT-2 generated length mismatch");

    lcqi::Gpt2KvCache cache(model.config);
    lcqi::Gpt2ForwardResult cached_result;
    for (const std::int32_t token : tokens) {
        cached_result = lcqi::run_gpt2_forward_cached(model, cache, token);
    }
    require(cache.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
            "GPT-2 KV cache filled token count mismatch");
    require(cache.byte_size() > 0, "GPT-2 KV cache byte size should be non-zero");
    require(cached_result.predicted_token == result.predicted_token,
            "cached GPT-2 predicted token mismatch");
    require(cached_result.logits.size() == result.logits.size(),
            "cached GPT-2 logits size mismatch");
    for (std::size_t index = 0; index < result.logits.size(); ++index) {
        require_close(cached_result.logits[index],
                      result.logits[index],
                      "cached GPT-2 logit mismatch");
    }

    lcqi::Gpt2CachedGreedyDecoder decoder_with_logits(model);
    lcqi::Gpt2ForwardResult decoder_result;
    for (const std::int32_t token : tokens) {
        decoder_result = decoder_with_logits.step_with_logits(token);
    }
    require(decoder_with_logits.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
            "GPT-2 optimized decoder filled token count mismatch");
    require(decoder_with_logits.kv_cache_bytes() == cache.byte_size(),
            "GPT-2 optimized decoder KV byte size mismatch");
    require(decoder_result.predicted_token == result.predicted_token,
            "optimized cached GPT-2 predicted token mismatch");
    require(decoder_result.logits.size() == result.logits.size(),
            "optimized cached GPT-2 logits size mismatch");
    for (std::size_t index = 0; index < result.logits.size(); ++index) {
        require_close(decoder_result.logits[index],
                      result.logits[index],
```

```
                    "optimized cached GPT-2 logit mismatch");
    }

    lcqi::Gpt2ExecutionOptions parallel_options;
    parallel_options.worker_count = 2;
    parallel_options.parallel_min_rows = 1;
    lcqi::Gpt2CachedGreedyDecoder parallel_decoder(model, nullptr, ↩
    ↪ parallel_options);
    lcqi::Gpt2ForwardResult parallel_result;
    for (const std::int32_t token : tokens) {
        parallel_result = parallel_decoder.step_with_logits(token);
    }
    require(parallel_decoder.worker_count() == parallel_options.worker_count,
            "parallel GPT-2 decoder worker count mismatch");
    require(parallel_result.predicted_token == result.predicted_token,
            "parallel cached GPT-2 predicted token mismatch");
    require(parallel_result.logits.size() == result.logits.size(),
            "parallel cached GPT-2 logits size mismatch");
    for (std::size_t index = 0; index < result.logits.size(); ++index) {
        require_close(parallel_result.logits[index],
                      result.logits[index],
                      "parallel cached GPT-2 logit mismatch");
    }

    lcqi::Gpt2CachedGreedyDecoder greedy_decoder(model);
    std::int32_t greedy_predicted = 0;
    for (const std::int32_t token : tokens) {
        greedy_predicted = greedy_decoder.step(token);
    }
    require(greedy_predicted == result.predicted_token,
            "optimized logits-free GPT-2 predicted token mismatch");

    lcqi::Gpt2HotspotProfile hotspot_profile;
    lcqi::Gpt2CachedGreedyDecoder profiled_decoder(model, &hotspot_profile);
    std::int32_t profiled_predicted = 0;
    for (const std::int32_t token : tokens) {
        profiled_predicted = profiled_decoder.step(token);
    }
    require(profiled_predicted == result.predicted_token,
            "profiled GPT-2 predicted token mismatch");
    require(hotspot_profile.decoder_steps == static_cast<std::int64_t>(tokens.↩
    ↪ size()),
            "profiled GPT-2 decoder step count mismatch");
    require(hotspot_profile.layer_steps ==
                static_cast<std::int64_t>(tokens.size()) * model.config.↩
    ↪ layer_count,
            "profiled GPT-2 layer step count mismatch");
    require(hotspot_profile.total_step_ms >= hotspot_profile.lm_head_ms,
            "profiled GPT-2 total time should include lm head time");

    const std::vector<std::int32_t> cached_generated =
        lcqi::gpt2_generate_greedy_cached(model, tokens, 2);
```

```
585     require(cached_generated == expected, "cached GPT-2 greedy generation ↩
    ↪ mismatch");
586 }
587
588 void test_gpt2_byte_bpe_tokenizer() {
589     const std::filesystem::path vocab_path = temp_file_path("lcqi_gpt2_vocab.↩
    ↪ json");
590     const std::filesystem::path merges_path = temp_file_path("lcqi_gpt2_merges.↩
    ↪ txt");
591     const std::string space_marker = "\xC4\xA0";
592     const std::string vocab =
593         "{\"Hello\":7,\",\":8,\"" + space_marker + "world\":9,\"!\":10}";
594     const std::string merges =
595         "#version: 0.2\n"
596         "H e\n"
597         "He l\n"
598         "Hel l\n"
599         "Hell o\n" +
600         space_marker + " w\n" +
601         space_marker + "w o\n" +
602         space_marker + "wo r\n" +
603         space_marker + "wor l\n" +
604         space_marker + "worl d\n";
605     write_text_file(vocab_path, vocab);
606     write_text_file(merges_path, merges);
607
608     const lcqi::Gpt2Tokenizer tokenizer =
609         lcqi::load_gpt2_tokenizer(vocab_path, merges_path, -1, -1);
610     std::filesystem::remove(vocab_path);
611     std::filesystem::remove(merges_path);
612
613     const std::vector<std::int32_t> ids =
614         lcqi::gpt2_encode(tokenizer, "Hello, world!");
615     const std::vector<std::int32_t> expected{
616         LCQI_GPT2_TOKENIZER_HELLO_ID,
617         8,
618         9,
619         10,
620     };
621     require(ids == expected, "GPT-2 BPE ids mismatch");
622     require(lcqi::gpt2_decode(tokenizer, ids) == "Hello, world!",
623             "GPT-2 BPE decode mismatch");
624 }
625
626 void test_gpt2_loads_hf_style_tiny_checkpoint() {
627     const std::filesystem::path directory =
628         temp_file_path("lcqi_gpt2_tiny_checkpoint_dir");
629     std::filesystem::remove_all(directory);
630     std::filesystem::create_directories(directory);
631
632     const lcqi::Gpt2ReferenceModel original = lcqi::↩
    ↪ make_tiny_gpt2_reference_model();
```

```
633     write_text_file(
634         directory / "config.json",
635         R"({"model_type":"gpt2","n_embd":4,"n_head":2,"n_layer":1,"n_positions"↩
    ↪ :8,"n_ctx":8,"vocab_size":6,"n_inner":8,"layer_norm_epsilon":0.00001,"↩
    ↪ activation_function":"gelu_new","bos_token_id":0,"eos_token_id":5})");
636     write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
637 
638     const lcqi::Gpt2ReferenceModel loaded = lcqi::load_gpt2_from_directory(↩
    ↪ directory);
639     const std::vector<std::int32_t> tokens{1, 2, 3};
640     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(loaded, ↩
    ↪ tokens);
641     std::filesystem::remove_all(directory);
642 
643     require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
644             "loaded GPT-2 predicted token mismatch");
645     for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
646         require_close(result.logits[index],
647                       LCQI_GPT2_LOGITS[index],
648                       "loaded GPT-2 logit mismatch");
649     }
650 }
651 
652 void test_gpt2_loader_rejects_bad_shape() {
653     const std::filesystem::path directory =
654         temp_file_path("lcqi_gpt2_bad_shape_dir");
655     std::filesystem::remove_all(directory);
656     std::filesystem::create_directories(directory);
657 
658     const lcqi::Gpt2ReferenceModel original = lcqi::↩
    ↪ make_tiny_gpt2_reference_model();
659     write_text_file(
660         directory / "config.json",
661         R"({"model_type":"gpt2","n_embd":5,"n_head":1,"n_layer":1,"n_positions"↩
    ↪ :8,"n_ctx":8,"vocab_size":6,"n_inner":8,"layer_norm_epsilon":0.00001,"↩
    ↪ activation_function":"gelu_new","bos_token_id":0,"eos_token_id":5})");
662     write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
663 
664     bool threw = false;
665     try {
666         static_cast<void>(lcqi::load_gpt2_from_directory(directory));
667     } catch (const std::runtime_error&) {
668         threw = true;
669     }
670     std::filesystem::remove_all(directory);
671     require(threw, "GPT-2 loader should reject bad tensor shape");
672 }
673 
674 void test_reference_rms_norm() {
675     const std::vector<float> input{3.0F, 4.0F};
676     const std::vector<float> weight{1.0F, 0.5F};
677     std::vector<float> output(2, 0.0F);
```

```
678     lcqi::rms_norm(input, weight, 0.0F, output);
679     require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
680     require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
681 }
682
683 void test_reference_rope() {
684     std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
685     lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
686     require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
687     require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
688     require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine ↩
        ↪ mismatch");
689     require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
690 }
691
692 void test_reference_kv_attention() {
693     lcqi::DecoderConfig config;
694     config.hidden_size = 4;
695     config.query_heads = 2;
696     config.kv_heads = 1;
697     config.head_dim = 2;
698     config.intermediate_size = 4;
699     config.vocab_size = 4;
700     config.max_context = 4;
701
702     lcqi::ReferenceKVCache cache(config, 1);
703     const std::vector<float> key{1.0F, 0.0F};
704     const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, ↩
        ↪ LCQI_ATTENTION_EXPECTED_1};
705     cache.append(0, 0, key, value);
706     require(cache.filled_tokens() == 1, "KV filled token count mismatch");
707     require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");
708
709     const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
710     std::vector<float> output(4, 0.0F);
711     lcqi::attention_decode(config, cache, 0, 0, query, output);
712     require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0↩
        ↪  mismatch");
713     require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1↩
        ↪  mismatch");
714     require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0↩
        ↪  mismatch");
715     require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1↩
        ↪  mismatch");
716 }
717
718 void test_reference_decoder_end_to_end() {
719     const lcqi::ReferenceDecoderModel model = lcqi::↩
        ↪ make_tiny_reference_decoder_model();
720     const std::vector<std::int32_t> tokens{1, 2, 3};
721     const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model↩
        ↪ , tokens);
```

```
722     require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
723             "reference decoder logits size mismatch");
724     require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
725             "reference decoder predicted token mismatch");
726     for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ↩
    ↪ ++index) {
727         require_close(result.logits[index],
728                       LCQI_REFERENCE_DECODER_LOGITS[index],
729                       "reference decoder golden logit mismatch");
730     }
731 }
732
733 void test_reference_decoder_trace_contract() {
734     const lcqi::ReferenceDecoderModel model = lcqi::↩
    ↪ make_tiny_reference_decoder_model();
735     const std::vector<std::int32_t> tokens{1, 2, 3};
736     const lcqi::ReferenceDecodeTraceResult trace =
737         lcqi::run_reference_decode_with_trace(model, tokens);
738
739     require(trace.result.predicted_token == ↩
    ↪ LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
740             "reference trace predicted token mismatch");
741     require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
742
743     const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
744     require(first.name == "embedding", "first checkpoint should be embedding");
745     require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
746             "first checkpoint token position mismatch");
747     require(first.dtype == "f32", "first checkpoint dtype mismatch");
748     require(first.layout == "row_major", "first checkpoint layout mismatch");
749     require(first.shape.size() == 1 &&
750                 first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
751             "first checkpoint shape mismatch");
752     require(first.stride.size() == 1 && first.stride[0] == 1,
753             "first checkpoint stride mismatch");
754
755     bool saw_logits = false;
756     bool saw_kv_slot = false;
757     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↩
    ↪  {
758         if (checkpoint.name == "kv_cache_key_slot") {
759             saw_kv_slot = true;
760             require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch"↩
    ↪ );
761             require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout↩
    ↪  mismatch");
762         }
763         if (checkpoint.name == "logits") {
764             saw_logits = true;
765             require(checkpoint.token_position == ↩
    ↪ LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
766                     "logits checkpoint token position mismatch");
```

```
767             require(checkpoint.shape.size() == 1 &&
768                         checkpoint.shape[0] ==
769                             static_cast<std::int32_t>(↩
    ↪ LCQI_REFERENCE_DECODER_VOCAB_SIZE),
770                     "logits checkpoint shape mismatch");
771             require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.↩
    ↪ size(),
772                     "logits checkpoint value size mismatch");
773             for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.↩
    ↪ size(); ++index) {
774                 require_close(checkpoint.values[index],
775                               LCQI_REFERENCE_DECODER_LOGITS[index],
776                               "trace logits checkpoint mismatch");
777             }
778         }
779     }
780     require(saw_kv_slot, "KV checkpoint missing");
781     require(saw_logits, "logits checkpoint missing");
782 }
783
784 void test_packed_i8_kernel_matches_scalar() {
785     for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
786         const lcqi::QuantizedLinearLayer layer =
787             lcqi::make_deterministic_i8_layer(shape_case.input_size,
788                                               shape_case.output_size,
789                                               0.03125F);
790         const lcqi::PackedLinearI8 packed =
791             lcqi::pack_linear_i8(layer, shape_case.output_block_size);
792         const std::vector<float> input = lcqi::make_deterministic_input(layer.↩
    ↪ input_size);
793         std::vector<float> scalar_output(static_cast<std::size_t>(layer.↩
    ↪ output_size), 0.0F);
794         std::vector<float> packed_output(static_cast<std::size_t>(layer.↩
    ↪ output_size), 0.0F);
795         lcqi::linear_i8(layer, input, scalar_output);
796         lcqi::linear_i8_packed(packed, input, packed_output);
797         require(lcqi::max_abs_diff(scalar_output, packed_output) <= ↩
    ↪ LCQI_KERNEL_MAX_DIFF,
798                 "packed int8 kernel output differs from scalar");
799
800         const lcqi::PackedLinearI8 simd_packed =
801             lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
802         if (lcqi::linear_i8_packed_avx2_available()) {
803             std::vector<float> avx2_output(static_cast<std::size_t>(layer.↩
    ↪ output_size), 0.0F);
804             lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
805             require(lcqi::max_abs_diff(scalar_output, avx2_output) <= ↩
    ↪ LCQI_KERNEL_MAX_DIFF,
806                     "AVX2 int8 kernel output differs from scalar");
807         }
808     }
809 }
```

```
810
811 }  // namespace
812
813 int main() {
814     try {
815         test_tiny_mlp();
816         test_tokenizer_contract();
817         test_tokenizer_rejects_unknown_without_unk();
818         test_safetensors_manifest_contract();
819         test_safetensors_rejects_bad_offsets();
820         test_safetensors_rejects_bad_dtype_shape_size();
821         test_safetensors_reads_float_tensors();
822         test_gpt2_tiny_forward_and_generation();
823         test_gpt2_byte_bpe_tokenizer();
824         test_gpt2_loads_hf_style_tiny_checkpoint();
825         test_gpt2_loader_rejects_bad_shape();
826         test_reference_rms_norm();
827         test_reference_rope();
828         test_reference_kv_attention();
829         test_reference_decoder_end_to_end();
830         test_reference_decoder_trace_contract();
831         test_packed_i8_kernel_matches_scalar();
832
833         std::cout << "lcqi tests passed\n";
834         return EXIT_SUCCESS;
835     } catch (const std::exception& error) {
836         std::cerr << "lcqi test failure: " << error.what() << "\n";
837         return EXIT_FAILURE;
838     }
839 }
```

实践与代码总索引

本卷已经把实践任务、代码走读路线和完整代码清单合并到同一套内容目录。目录中不再存在一个独立的“源码卷部分”：构建入口、调用链、公共合同、tiny MLP、int8 kernel、reference decoder、GPT-2、工具脚本和测试文件都插入到相应主题之后，共同解释同一份 `books/cpu-volume-3/labs/linux_cpu_inference` 源码。

源码阅读任务

> 读者不需要再在实践卷和源码卷之间来回切换，也不需要先读完 16 个实践章再去末尾翻代码。实践章节提出任务，紧邻的代码走读解释完整实现，测试和 benchmark 章节给出回归证据；三者已经合并到这一本实践与代码卷里，共同构成一条可复查的工程证据链。

一、实践主题到代码走读的合并位置

第 1 章之后 插入“代码走读：构建入口、项目边界与调用链总图”，先把 CMake、README、模型资产、target、CLI、benchmark、trace、ledger、serving probe 和测试入口放到一张可复现地图里。

第 4 章之后 插入“代码走读：公共合同、Tiny MLP 与 Int8 Kernel”，把模型、推理、tokenizer、safetensors、reference decoder、GPT-2 头文件，以及 tiny MLP、基础 CLI、scalar/packed/AVX2 kernel 连起来。

第 10 章之后 插入“代码走读：Reference Decoder、Tokenizer、Safetensors 与 GPT-2”，把真实模型加载、byte-level BPE、safetensors、GPT-2 forward、cached KV 和 greedy generation 放在模型导入主题内部讲清楚。

第 16 章之后 插入“代码走读：工具、Smoke、Benchmark 与回归测试”，把 benchmark、trace、ledger、serving probe、SmolLM2 smoke、GPT-2 smoke、benchmark compare 和 CTest 回归证据集中收束。

二、最小复现命令

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
make cpu3p-pdf
make cpu3p-epub
```

完成这组命令后，读者应同时检查相邻实践章节的作业结果、对应代码走读、测试断言和报告字段，确认代码、测试、报告字段和章节讲解是一条证据链。

术语表

术语	实践含义
manifest	模型资产清单，记录 tensor name、dtype、shape、offset、checksum 和 tokenizer/配置版本。
shape verifier	在 kernel 运行前检查 shape、dtype、layout、量化 descriptor 和 state edge 是否匹配。
reference decoder	以清楚正确为目标的 decoder 实现，用来生成 trace 和 golden。
oracle	比较候选实现与 reference 的判定器，可使用精确相等或误差界。
trace checkpoint	推理链路中的可复查节点，至少包含 shape、stride、dtype、layout、checksum 和摘要值。
KV cache	attention 历史 K/V 状态，跨 token 存活，不能被普通 activation planner 复用。
packing	把权重改写成更适合热循环和 SIMD 的布局。
backend	执行计划落到机器的实现，例如 scalar、AVX2、AVX-512、GPU 或库调用。
fallback	后端能力不满足时显式降级到可验证路径，并记录原因。
SLO	服务目标，例如 TTFT、TPOT、queue wait、拒绝率和取消回收时间。